Laboratorios da materia Estatística

Intelixencia Artificial

Beatriz Pateiro López

Táboa de contidos

Prefacio 5

Introdución a R 6

Instalación de R . . . 8
Documentación de R . . . 8
Primeiros pasos en R . . . 8
R como calculadora científica . . . 9
Clases de obxectos . . . 9
Vectores . . . 10
Construción de vectores . . . 10
Selección de compoñentes . . . 12
Comandos útiles para traballar con vectores . . . 13
Operacións con vectores . . . 14
Matrices . . . 15
Construción de matrices . . . 15
Comandos para matrices . . . 16
Operacións con matrices . . . 17
Factores, data frames e listas . . . 18
Factores . . . 18
Data frames . . . 18
Listas . . . 19
Gráficos . . . 19
Gráficos de alto nivel . . . 20
Gráficos de baixo nivel . . . 26
Funcións . . . 28
Librarías . . . 29
Importación de datos . . . 29
Lectura de datos dun ficheiro de texto . . . 30
Acceso ás bases de datos de R . . . 31
EXERCICIOS . . . 31

1 Estatística descritiva 33

1.1 Táboas de frecuencias . . . 33
1.2 Representacións gráficas . . . 35
1.3 Medidas características . . . 41
1.4 Tipificación e transformación de datos en R . . . 44
1.4.1 Tipificar datos . . . 44
1.4.2 Transformación de datos . . . 45

1.5 Estatística descritiva bidimensional . . . 47
1.5.1 Táboas de continxencia . . . 47
1.5.2 Boxplot para unha variable continua segundo unha categórica . . . 47
1.5.3 Análise de dúas variables cuantitativas . . . 48
1.6 Táboa resumo de procedementos e funcións . . . 50
1.7 EXERCICIOS . . . 51

2 Fundamentos de probabilidade 52
2.1 A función `sample()` . . . 52
2.2 Reproducibilidade . . . 55
2.3 EXERCICIOS . . . 55

3 Variables aleatorias 56
3.1 Distribucións en R . . . 56
3.2 Principais modelos de variables aleatorias discretas . . . 57
3.2.1 Distribucións de Bernoulli e Binomial . . . 57
3.2.2 Distribucións xeométrica e binomial negativa . . . 59
3.2.3 Distribución de Poisson . . . 60
3.2.4 Distribución hiperxeométrica . . . 61
3.3 Principais modelos de variables aleatorias continuas . . . 62
3.3.1 Distribución uniforme . . . 62
3.3.2 Distribución normal . . . 63
3.3.3 Distribución exponencial . . . 66
3.3.4 Distribución gamma . . . 67
3.3.5 Distribución beta . . . 68
3.4 Teorema Central do Límite . . . 70
3.5 EXERCICIOS . . . 76

4 Introdución á inferencia estatística e á estimación de parámetros 77
4.1 Estimación puntual . . . 77
4.1.1 Estimación puntual da media e a varianza . . . 77
4.1.2 Estimación puntual dunha proporción . . . 82
4.2 Intervalos de confianza . . . 84
4.2.1 Intervalo de confianza para a media dunha poboación normal . . . 84
4.2.2 Intervalo de confianza para a diferenza de medias de poboacións normais 87
4.2.3 Intervalo de confianza para a varianza dunha poboación normal . . . 92
4.2.4 Intervalo de confianza para o cociente de varianzas de poboacións normais 93
4.2.5 Intervalo de confianza para unha proporción . . . 94
4.2.6 Intervalo de confianza para a diferenza de proporcións . . . 94
4.3 EXERCICIOS . . . 96

5 Contrastes de hipóteses 97
5.1 Conceptos básicos . . . 97

5.2 Contrastes para a media dunha poboación normal . . . 98
5.2.1 Contrastes para a media con varianza coñecida . . . 98
5.2.2 Contrastes para a media con varianza descoñecida . . . 102
5.3 Contrastes para a diferenza de medias . . . 108
5.3.1 Mostras independentes, varianzas descoñecidas e iguais . . . 108
5.3.2 Mostras independentes, varianzas descoñecidas e distintas . . . 110
5.3.3 Mostras emparelladas . . . 111
5.4 Contrastes para a razón de varianzas de poboacións normais . . . 113
5.5 Contrastes para unha proporción . . . 115
5.6 Contrastes para a diferenza de proporcións . . . 116
5.7 EXERCICIOS . . . 117

6 Introdución á regresión linear **118**
6.1 O modelo de regresión lineal simple . . . 118
6.2 Estimación do modelo de regresión en R . . . 119
6.3 Predición . . . 123
6.4 EXERCICIOS . . . 123

Prefacio

Este documento recompila o material de laboratorio da materia Estatística do Grao en Intelixencia Artificial. Está pensado para complemetar ao material teórico e poñer en práctica e aplicar os conceptos desenvolvidos no curso: estatística descritiva, probabilidade e inferencia estatística.

Neste curso empregamos R como linguaxe de referencia para a análise de datos. R é unha ferramenta de código aberto amplamente utilizada nos ámbitos da estatística, a aprendizaxe automática e a ciencia de datos, tanto na investigación como noutros ámbitos profesionais. A súa potencia, flexibilidade e a extensa comunidade internacional que a mantén fan del unha elección ideal para a análise estatística.

Emprego de ferramentas asistidas por IA

Este material foi redactado combinando experiencia docente e recursos tecnolóxicos actuais. Puntualmente, empregouse ChatGPT —unha ferramenta de intelixencia artificial desenvolvida por OpenAI— para apoiar a redacción ou facilitar algunhas traducións automáticas, sempre baixo supervisión e revisión dos resultados. A responsabilidade final sobre os contidos, interpretacións e adaptacións é exclusivamente da autora.

Agradecementos e suxestións

Agradezo calquera suxestión que axude a mellorar este documento. Se atopas erratas, erros, ou consideras que algunha sección se podería explicar mellor, non dubides en escribirme a beatriz.pateiro@usc.es.



Esta obra está baixo una Licencia Creative Commons Atribución-NonComercial-Compartir igual 4.0 Internacional

Introdución a R

R é unha linguaxe de programación deseñada para a manipulación de datos, o cálculo matemático, a modelización estatística e a creación de gráficos. Tamén se pode ver como un ecosistema de software libre porque non se limita á linguaxe de programación. R abrangue un conxunto de librarías (**contributed packages**) en continuo desenvolvemento, que agrupan funcións orientadas a obxectivos específicos, así como documentación, entornos de desenvolvemento e comunidades que interactúan entre si.

No contexto da IA, R utilízase nas fases de exploración, preparación e análise de datos, que son pasos clave antes de aplicar calquera modelo de aprendizaxe. Grazas á súa ampla colección de paquetes, R permite limpar datos, tratar valores perdidos, transformar variables e realizar análises estatísticas exploratorias que axudan a comprender a estrutura da información dispoñible. Ademais, R ofrece ferramentas específicas para aprendizaxe automática e modelización predictiva. Por exemplo, algoritmos de aprendizaxe automática moi extendidos como GBM, XGBoost e LightGBM dispoñen de implementacións en forma de paquetes de R, o que permite aplicar estes métodos de maneira eficiente en problemas reais de análise de datos. Do mesmo xeito, R conta cun amplo ecosistema de paquetes no ámbito da intelixencia artificial explicable (XAI), que permiten interpretar modelos de aprendizaxe automática. Tamén conta con librarías que facilitan a súa integración con APIs de Large Language Models (LLMs), proporcionando complementos relevantes.

> **Emprego de ferramentas asistidas por IA na aprendizaxe**
>
> Facer uso de LLMs como ChatGPT e Copilot para programar é, en xeral, un bo recurso **cando xa sabes o que estás facendo**. Non obstante, empregar este tipo de ferramentas cando se está aprendendo unha linguaxe de programación como R é realmente **prexudicial para a aprendizaxe**, especialmente se só copias e pegas o que che devolven. En primeiro lugar porque podes recibir código que é incorrecto. Se non coñeces suficientemente ben a linguaxe coa que estás a traballar, non entenderás o que estás recibindo nin serás capaz de corrixilo. Por outro lado, aínda que a resposta sexa correcta, estas ferramentas tenden a proporcionar solucións que empregan funcións coas que nunca traballarías se estás escribindo código en R por primeira vez. Imos mostrar un exemplo. R inclúe un conxunto de datos chamado `mtcars`, que contén información sobre automóviles, como o consumo de combustible (`mpg`) e o número de marchas (`gear`). Ao facer a seguinte consulta a ChatGPT *'Dame un código R que, no conxunto de datos mtcars, calcule a media da variable mpg en función de gear'* obtivéronse dúas posibles solucións:

```r
# Calcular a media de mpg en función de gear
media_mpg_por_gear <- aggregate(mpg ~ gear, data = mtcars, FUN = mean)

# Mostrar o resultado
media_mpg_por_gear
```

```r
# Alternativamente, usando dplyr
library(dplyr)

media_mpg_por_gear <- mtcars %>%
  group_by(gear) %>%
  summarise(media_mpg = mean(mpg))

# Mostrar o resultado
media_mpg_por_gear
```

Se ben as dúas opcións son correctas, ambas teñen as súas dificultades para alguén que está aprendendo R. A primeira solución usa `aggregate()` e a fórmula `mpg ~ gear`. Para un principiante, a notación con `~` pode resultar pouco intuitiva e esixirá comprender como funcionan as fórmulas en R. A segunda solución, que é máis actual e a que se adoita empregar en análise de datos moderna, require cargar unha libraría externa (`dplyr`) e usar a sintaxe de tubería `%>%`, que tamén pode non ser inmediata para quen está empezando. A solución máis sinxela para un principiante sería empregar una función básica como `tapply()`, que permite calcular a media ou outras medidas estatísticas de forma directa sen precisar fórmulas complexas ou paquetes adicionais.

```r
media_mpg_por_gear <- tapply(mtcars$mpg, mtcars$gear, mean)
media_mpg_por_gear
```

Como resumo, o coñecemento previo é esencial se queres usar LLMs para calquera tarefa, en particular, en programación. Mesmo así, os LLM son útiles sobre todo para pedir explicacións, pero non axudan tanto a aprender pois xeran respostas que simplemente parecen correctas. A recomendación é non facer uso de LLMs para a aprendizaxe de R nin para a resolución dos exercicios do curso (**as respostas que parezan xeradas por este tipo de ferramentas serán sinaladas e pedirase xustificación sobre elas**). Lectura recomendada: AI, LLMs, BS, and vibe coding

Instalación de R

Toda a información relativa a R e á súa instalación pode atoparse na súa páxina web (http://www.r-project.org/) no denominado CRAN (Comprehensive R Archive Network).

O sistema R consta de dúas partes fundamentais:

- O sistema base (necesario para executar R por primeira vez).
- A colección de paquetes adicionais (**contributed packages**).

Para instalar R, simplemente escolle unha localización no teu ordenador e executa o ficheiro de instalación correspondente ao teu sistema operativo.

Unha vez instalado R, podes executalo e comezar a traballar. Se o prefires, tamén podes empregar RStudio (https://posit.co/download/rstudio-desktop/) como o programa principal para acceder a R. RStudio é un entorno de desenvolvemento integrado (IDE) para R. Permite integrar gráficos, táboas e documentación nun único contorno, facendo que a análise de datos sexa máis accesible.

Documentación de R

Para máis información, pódese recorrer tanto á axuda de R que se instala co propio programa (desde o menú Axuda), como aos manuais dispoñibles na súa páxina web. Existen diferentes formas de documentación en R:

- Manuais electrónicos: https://cran.r-project.org/manuals.html
- Unha ampla colección de libros e outros manuais sobre R:
  - https://www.r-project.org/doc/bib/R-publications.html
  - https://www.r-project.org/other-docs.html
- R FAQ (preguntas máis frecuentes): https://cran.r-project.org/doc/FAQ/R-FAQ.html

Primeiros pasos en R

Ao executar R, verás que dentro da xanela (R GUI) aparece unha barra de menú, unha barra de ferramentas e a **consola de R**. A consola de R é o lugar no que teclearemos os comandos de R e obteremos os resultados do programa. Outra opción é crear un *script* (un arquivo de texto), escribir e gardar nel as funcións ou secuencias de comandos de interese. Despois, poderán executarse copiándoas na consola.

O primeiro que vemos na consola é información básica sobre R e o símbolo `>` (prompt), que indica que R está á espera de que introduzamos un comando.

R como calculadora científica

R permite realizar cálculos aritméticos e, nese sentido, podería utilizarse como unha potente calculadora. Os operadores aritméticos básicos son `+`, `-`, `*`, `/`, `^` para elevar a unha potencia, `%%` para o resto da división enteira, e `%/%` para o cociente da división enteira. Se nunha mesma expresión aparecen varios operadores binarios, a orde priorizada de execución é: exponenciación, produto/división e suma/resta. Para modificala, debemos recorrer ao uso de parénteses.

```
2 + 4 * 5 - 10 / 2^2
(2 + 4) * 5 - (10 / 2)^2
```

Obsérvase que para a separación decimal, R utiliza o punto. Este feito pode dar lugar a problemas, por exemplo, ao importar datos de Excel onde se emprega a coma para separar a parte enteira da decimal.

Ademais das operacións elementais, poden realizarse outro tipo de operacións como logaritmos, exponenciais, raíces cadradas, funcións trigonométricas, etc. Nestes casos, debe aplicarse a función correspondente (`log()`, `exp()`, `sqrt()`, `sin()`, `cos()`, `tan()`, ...) sobre o valor co que se opera.

```
exp(2)
sqrt(144)
cos(pi / 2)
```

En R, os valores máis infinito e menos infinito están representados por `Inf` e `-Inf`. Se realizamos operacións para as cales a matemática non ten solución, o resultado é `NaN` (Not a Number).

```
1 / 0
-2 * Inf
sqrt(-2)
0 / 0
```

Clases de obxectos

Nos exemplos que puxemos ata agora, todos os resultados foron numéricos. Porén, en R podemos traballar con diferentes obxectos: `numeric`, `logical`, `character`, `list`, `matrix`, `array`, `factor` ou `data.frame`, entre outros. Para coñecer o tipo ou clase dun obxecto, utilízase a función `class()`.

```r
a <- 2
b <- "ola"
c <- TRUE
class(a)
class(b)
class(c)
```

Debemos ter en conta que o mesmo operador/función pode ter efectos distintos segundo o tipo de obxecto ao que se aplique. Por exemplo, se facemos `2*c`, a variable lóxica codifícase primeiro cos valores `TRUE = 1`, `FALSE = 0` e despois opera como se fose unha variable numérica.

```r
2 * c
class(2 * c)
```

Nas seguintes seccións imos presentar os principais tipos de obxectos que se usan en R e amosar como se crean, manipulan e combinan para realizar operacións. Aprender a traballar con estes obxectos é fundamental, porque constitúen a base de calquera análise ou modelo en R.

Vectores

R opera con diferentes estructuras de datos. A máis simple é o vector, que é una colección de elementos do mesmo tipo.

Construción de vectores

En R dispoñemos de varios mecanismos para construír vectores. A forma máis común de crear un vector é usando o comando de concatenación `c()`.

```r
x <- c(3.4, 23.1, 5.9, 6.7, 7)
x
class(x)
```

Ademais de numéricos, os vectores poden ser lóxicos, de tipo carácter ou factores dependendo do tipo de variables que os formen.

```r
y <- c("pera", "amorodo", "uva")
y
class(y)
```

```r
z <- c(TRUE, TRUE, FALSE)
z
class(z)
```

En R, os vectores poden conter valores ausentes, indicados co símbolo `NA`. Isto é útil cando se traballa con datos incompletos ou cando algunhas observacións non están dispoñibles. Por exemplo, ao crear un vector con `c()` podemos incluír `NA` directamente:

```r
x2 <- c(3.4, 23.1, 5.9, NA, 6.7, 7)
x2
class(x2)
```

Se desexamos crear un vector sen asignarlle valores, podemos usar `vector()`, especificando a lonxitude do mesmo. Tamén podemos empregar funcións específicas dependendo da clase, como `numeric()` ou `logical()`.

```r
vector(mode = "numeric", length = 5)
numeric(5)
vector(mode = "logical", length = 5)
logical(5)
```

En R é habitual traballar con secuencias de números, tanto para crear vectores de datos como para definir índices ou intervalos de valores. R proporciona distintos mecanismos sinxelos para xerar este tipo de secuencias, dependendo de se queremos números enteiros consecutivos ou valores igualmente espazados nun intervalo dado. Para xerar secuencias de número enteiros podemos facer o seguinte.

```r
1:5
-3:3
```

Cando se busca ter secuencias equiespaciadas de números dentro dun intervalo, utilízase a función `seq()` indicando a lonxitude desexada da secuencia ou o espazo a deixar entre dous números consecutivos.

```r
seq(-5, 5, length = 5)
seq(-5, 5, by = 2)
```

> **Emparellamento de argumentos en R**
>
> Cando escribimos `seq(1, 10, length = 7)`, R asume que o primeiro valor introducido corresponde ao valor inicial da secuencia e que o segundo valor corresponde ao valor final. Isto coñécese como emparellamento posicional (positional matching).
> Isto implica que **non é necesario especificar o nome dos argumentos** sempre que se introduzan na mesma orde na que aparecen definidos na función.
> Porén, cando unha función ten moitos argumentos e non coñecemos a orde exacta na que están definidos, o máis recomendable é **indicar explicitamente o nome do argumento** que estamos a pasar. En xeral, abonda con escribir a parte inicial do nome do argumento, sempre que non se produza ambigüidade con outros argumentos. Este comportamento denomínase emparellamento parcial de nomes (partial matching of names).

Se os vectores que desexamos xerar toman en todas as súas componentes o mesmo valor (repeticións), usamos o comando `rep()`, indicando a lonxitude de vector desexada.

```
rep(9, 4)
```

En calquera dos casos anteriores, para coñecer que parámetros e opcións ten un comando podemos utilizar `args(nome_comando)`. Se necesitamos máis información sobre o seu uso, podemos recorrer á axuda de que dispón R a través de `help(nome_comando)`. Como exemplo, podemos ver que argumentos ten a función `numeric` mediante `args(numeric)` ou acceder á axuda de `seq()` empregando `help(seq)` ou `?seq`.

Selección de compoñentes

Diferentes subconjuntos de elementos dun vector poden ser seleccionados incluíndo, entre corchetes despois do nome de vector, un vector índice: unha expresión lóxica ou unha secuencia numérica que indica os elementos a tomar.

```
x[2]
x[c(2, 4)]
x[2:4]
```

Uso de condicións lóxicas para seleccionar subconxuntos de datos

Na práctica, é habitual traballar con subconxuntos de datos do conxunto orixinal que cumpran certos criterios. Por exemplo, pode interesarnos coñecer os elementos de `x` que son maiores que 5. Podemos seleccionar subconxuntos utilizando condicións lóxicas en R.

O uso de operadores lóxicos con vectores é **extremadamente útil**. Un operador lóxico permite comprobar se se cumpre ou non unha determinada condición. Se se usa nun vector (por exemplo, nun conxunto grande de medidas), podemos obter o número de casos que cumpren a condición.

A seguinte táboa mostra os operadores lóxicos en R.

Operador	Descrición
`<`	menor que
`<=`	menor ou igual que
`>`	maior que
`>=`	maior ou igual que
`==`	igual a
`!=`	non igual a

```
x > 5
x[x > 5]
```

Os operadores da seguinte táboa pódense utilizar para facer comparacións entre vectores lóxicos en R.

Operador	Descrición
`&`	**e** lóxico
`|`	**ou** lóxico
`!`	**non** lóxico

```
x > 5 & x < 10
x[x > 5 & x < 10]
```

Comandos útiles para traballar con vectores

Existen comandos en R que están especialmente pensados para traballar con vectores. Por exemplo, para coñecer a lonxitude dun vector, emprégase a función `length()`.

```
length(x)
```

Se o que nos interesa é ordenar os valores que forman o vector de forma crecente ou decrecente, podemos utilizar a función `sort()`.

```r
sort(x)
sort(x, decreasing = TRUE)
```

A continuación recóllense algúns destes comandos.

Función R	Descrición
`sum(x)`	Suma dos elementos de `x`
`cumsum(x)`	Vector coa suma acumulada até cada compoñente de `x`
`prod(x)`	Produto dos elementos de `x`
`cumprod(x)`	Vector co produto acumulado de `x`
`max(x)`	Máximo de `x`
`min(x)`	Mínimo de `x`
`length(x)`	Lonxitude de `x`
`sort(x)`	Ordena os elementos de `x` de menor a maior
`mean(x)`	Media aritmética de `x`
`union(x, y)`	Unión dos vectores `x` e `y` sen repetir elementos
`intersect(x, y)`	Elementos comúns sen repetir en `x` e `y`
`setdiff(x, y)`	Elementos de `x` que non están en `y`

Se realizamos operacións con vectores que conteñen `NA`, é importante ter en conta que moitas funcións, como `mean()`, devolverán tamén `NA` a menos que se especifique o argumento `na.rm = TRUE` para ignorar os valores ausentes.

```r
mean(x2)
mean(x2, na.rm = TRUE)
```

Operacións con vectores

En xeral, as mesmas operacións que se podían levar a cabo con números poden ser realizadas con vectores.

Operacións compoñente a compoñente

En R podemos realizar as operacións básicas sobre vectores, compoñente a compoñente.

```r
x <- c(1, 5, 7, 11)
z <- c(1, -2, 3, -4)
x + z
x * z
```

```
x / z
(2 * x - z)^2
```

A diferenza do que ocorre con outras linguaxes de programación, é necesario estar moi atentos a que as lonxitudes dos vectores coincidan, pois en caso contrario R non devolve un erro (aínda que si unha mensaxe de aviso) e seguirá operando coma se nada ocorrese.

```
aux <- c(20, 40, 60)
x + aux
```

Vemos que na cuarta componente do vector suma reciclou a primeira compoñente do vector `aux`, realizando a operación $20 + 11 = 31$. Do mesmo xeito, pódense levar a cabo operacións de vectores con números. En tal caso, a operación aplícase a cada compoñente do vector.

```
x^2
seq(-5, 5, length = 20) + 5
```

Operacións vectoriais

O produto de dous vectores, tal e como se coñece en matemáticas $\sum_i x_i z_i$ (produto escalar), lévase a cabo co operador `%*%`

```
x %*% z
```

Matrices

Construción de matrices

As matrices son as xeneralizaciones bidimensionales dos vectores. O comando `matrix()` é o empregado comunmente á hora de xerar unha matriz. Nos parámetros do devandito comando é necesario indicar as compoñentes da matriz e o número de filas e columnas. Outros comandos útiles para construír matrices son `rbind()` e `cbind()` que enlazan vectores por filas ou por columnas para construír unha matriz.

```
matrix(c(1, 2, 3, 7), nrow = 2, ncol = 2)
matrix(1:6, nrow = 2, ncol = 3)
cbind(c(1, 2), c(3, 7))
rbind(c(1, 3, 5), c(2, 4, 6))
```

Nas bases de datos coas que se adoita traballar tanto en Estatística como noutras ciencias experimentais, xeralmente as filas representan a individuos e as columnas ás variables en estudo. Como consecuencia, pode ocorrer que, traballando con matrices, interésenos asignar a determinadas filas ou columnas nomes específicos. Isto faise en R a través dos comandos `colnames()` e `rownames()`.

```
A <- cbind(c(1, 2), c(3, 7))
colnames(A)
colnames(A) <- c("Variable1", "Variable2")
A
colnames(A) <- paste("Variable", 1:2, sep = "")
A
```

Comandos para matrices

Existen comandos en R que están especialmente pensados para traballar con matrices. Existen multitude de comandos deseñados para traballar sobre matrices: `diag()` para extraer a diagonal, `det()` para o determinante, `solve()` que nos serve para obter a inversa (a condición de que a matriz sexa non singular), etc. Na seguinte táboa amósanse algúns dos máis empregados.

Función en R	Descrición
`dim(A)`	dimensión de `A`
`nrow(A)`	número de filas de `A`
`ncol(A)`	número de columnas de `A`
`rownames(A)`	nomes das filas de `A`
`colnames(A)`	nomes das columnas de `A`
`t(A)`	trasposta da matriz `A`
`solve(A)`	inversa da matriz `A`
`rowMeans(A)`	medias por filas
`rowSums(A)`	sumas por filas
`colMeans(A)`	medias por columnas
`colSums(A)`	sumas por columnas

```
diag(A)
t(A)
det(A)
solve(A)
A %*% solve(A)
solve(A) %*% A
```

Ademais, `eigen()` permítenos obter autovalores e autovectores asociados a unha determinada matriz.

```
eigen(A)
```

Operacións con matrices

Operacións compoñente a compoñente

Os operadores de cálculo comúns pódense utilizar para operar compoñente a compoñente tamén con matrices. Aquí, a diferenza do que ocorría cos vectores, os desamaños dimensionales serán reportados como erros.

```
B <- rbind(c(1, 3, 5), c(2, 4, 6))
A + B
C <- matrix(c(5, 3, 7, 8), nrow = 2, ncol = 2)
A + C
A * C
```

Operacións matriciais

A partir dun produto matricial de vectores (posto que un vector pódese considerar como unha matriz cunha única fila ou columna) tamén se pode xerar unha matriz. O comando `t()` utilízase para obter a trasposta dunha matriz (ou dun vector).

```
x <- c(1, 2, 3)
y <- c(4, 5, 6)
t(x)
x %*% t(x)
x %*% t(y)
```

O produto de matrices comunmente coñecido en matemáticas realízase, do mesmo xeito que o produto vectorial que acabamos de ver, utilizando o operador `%*%`.

```
A %*% B
```

Factores, data frames e listas

Factores

Un factor en R é un obxecto que representa variables categóricas, ordenadas ou non, e almacena tanto os valores como os niveis posibles. Pódense crear con `factor()`.

```
sexo <- factor(c("M", "F", "F", "M", "F"))
sexo
levels(sexo)
```

Data frames

Outra clase de obxectos coa que deberemos afacernos a traballar son os data frames. Un data frame é unha estrutura de datos tabular que serve para almacenar información en filas e columnas. O que diferencia aos data frames das matrices é que cada columna pode ter un tipo distinto de dato. O comando xeral para a construción de data frames é `data.frame()`.

```
cbind(1, 1:5)
L3 <- LETTERS[1:3]
L3
df <- data.frame(cbind(1, 1:5), "Letras" = sample(L3, 5, replace = TRUE))
df
```

Unha vez creado un data frame, podemos acceder ás súas columnas, filas ou elementos concretos de varias formas:

```
df$Letras # Columna "Letras"
df[[2]] # Segunda columna
df[, 2] # Segunda columna
df[2:3, ] # Filas 2 e 3
df[1, 3] # Elemento da fila 1, columna 3
df[5, "Letras"] # Elemento fila 5, columna "Letras"
```

É común que, por esixencias de determinadas funcións de R, vexámonos en ocasións obrigados a converter matrices en data frames. Analogamente ao que ocorría cos factores, a orde a executar é `as.data.frame()`.

```
matriz4x4 <- matrix(1:16, nrow = 4, ncol = 4)
as.data.frame(matriz4x4)
```

A pesar de que a contorna que nos ofrece o programa R non é a primeira vista tan atractivo como poida ser o doutros programas comerciais, R tamén nos ofrece a posibilidade de ver os datos de matrices ou data frames nun formato de celas, tal e como ocorre nestes últimos. Isto conséguese utilizando o comando `fix()`.

```
fix(matriz4x4)
```

Para que podamos continuar utilizando a consola de comandos, será necesario que de cada vez pechemos a pantalla co formato de celas.

Listas

Unha lista en R é un obxecto consistente dunha colección ordenada doutros obxectos coñecidos como compoñentes da lista. Dita compoñentes poden ser de diferente clase, dimensión, ... O comando utilizado para xerar unha lista é `list()` e os seus parámetros, as compoñentes da lista.

```
Lista <- list(marido = "Juan Pedro", esposa = "Josefina", nfillos = 3, idades = c(16, 11, 4)
```

Para acceder individualmente ás compoñentes dunha lista existen dúas vías. A primeira é a través do número de compoñente, para o que temos que utilizar o dobre corchete a continuación do nome da lista. A segunda é a través do nome da compoñente, escribíndoo a continuación do nome da lista precedido polo carácter `$`.

```
Lista[[2]]
Lista[[5]]
Lista$marido
Lista$idades
```

Gráficos

Os gráficos representan unha compoñente moi importante en R. Mediante gráficos é posible representar unha gran cantidade de funcións estatísticas. Os comandos gráficos en R divídense basicamente en 3 grupos:

- **Gráficos de alto nivel.** Crean un novo gráfico en pantalla; opcionalmente inclúen etiquetas, títulos, lenda, ...
- **Gráficos de baixo nivel.** Engaden información a gráficos xa existentes. Por exemplo, puntos, liñas, ...

- **Gráficos interactivos.** Permiten ao usuario engadir ou eliminar información dun gráfico de forma interactiva, así como usar o rato como punteiro ou localizador.

A totalidade de comandos e funcións gráficas que veremos aquí están incluídos nos paquetes que por defecto veñen instalados en R. Existen paquetes gráficos opcionais (e tamén gratuítos) que poden ser instalados en R, tales como `ggplot2`, `maps`, `grid` ou `lattice`, que provén ao usuario con funcionalidades máis avanzadas.

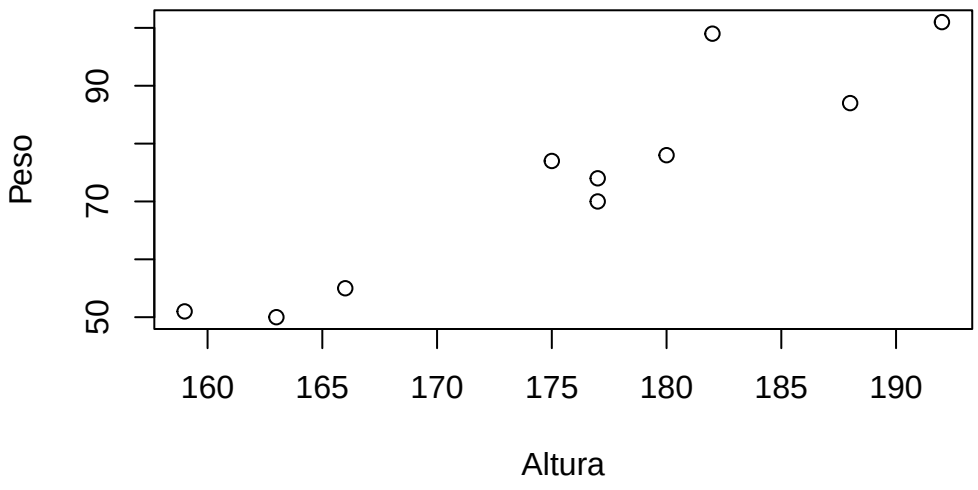
Gráficos de alto nivel

Os gráficos de alto nivel están deseñados para xerar un gráfico completo dos datos pasados como argumentos á función. Eixos, etiquetas e títulos son xerados automaticamente a non ser que o usuario esixa o contrario. Os gráficos de alto nivel sempre comezan un novo gráfico, borrando o actual se non se lles indica o contrario. En Windows, se queremos abrir unha nova xanela gráfica sen borrar a anterior, empregamos a orde `windows()`. Noutros sistemas operativos existen funcións análogas, como `quartz()` en macOS ou `x11()` en Linux.

A función `plot()`

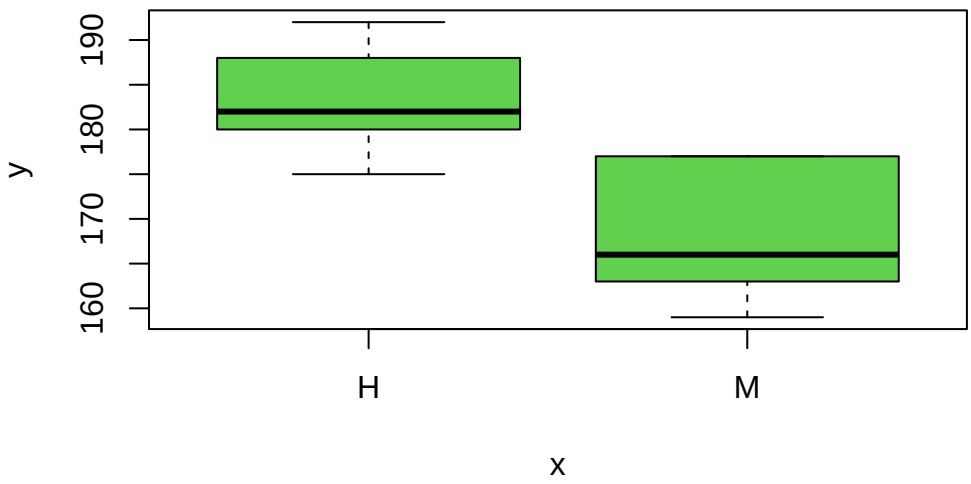
Sen dúbida a función gráfica máis utilizada é `plot()`. É unha función xenérica, na que o gráfico producido depende da clase do primeiro argumento da función. Vexamos un exemplo do seu uso, onde se xera un gráfico de dispersión do peso en función da altura nun grupo de 10 individuos.

```
Altura <- c(175, 177, 192, 159, 166, 188, 180, 182, 177, 163)
Peso <- c(77, 70, 101, 51, 55, 87, 78, 99, 74, 50)
plot(Altura, Peso)
```

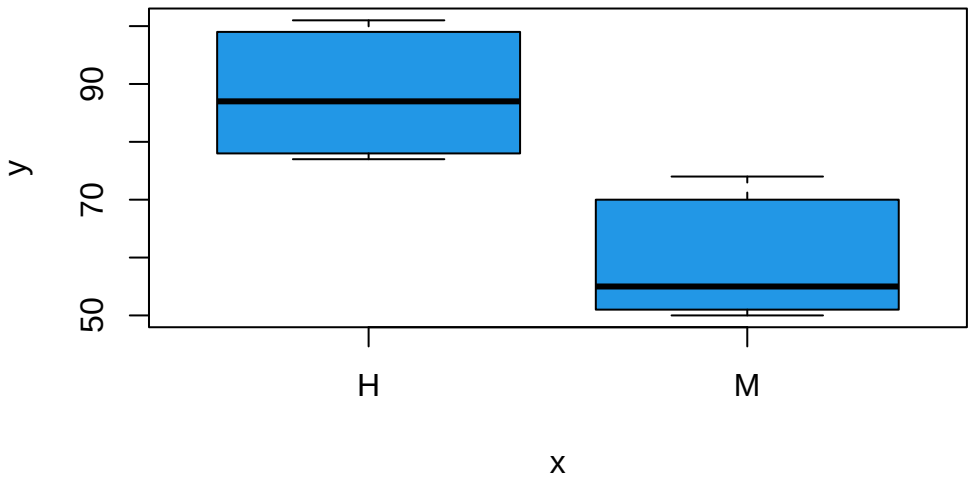


Se o primeiro dato que se lle pasa á función `plot()` é un `factor` (por exemplo, o sexo de cada un dos individuos), o gráfico que se xera é un diagrama de caixa (boxplot) do segundo dato (`Altura` ou `Peso`) para cada nivel do factor `Sexo`. Vexamos a continuación como se xerarían dous boxplots de cada nivel de `Sexo` para a `Altura` e o `Peso` respectivamente.

```
Sexo <- as.factor(c("H", "M", "H", "M", "M", "H", "H", "H", "M", "M"))
plot(Sexo, Altura, col = 3)
```




```
plot(Sexo, Peso, col = 4)
```

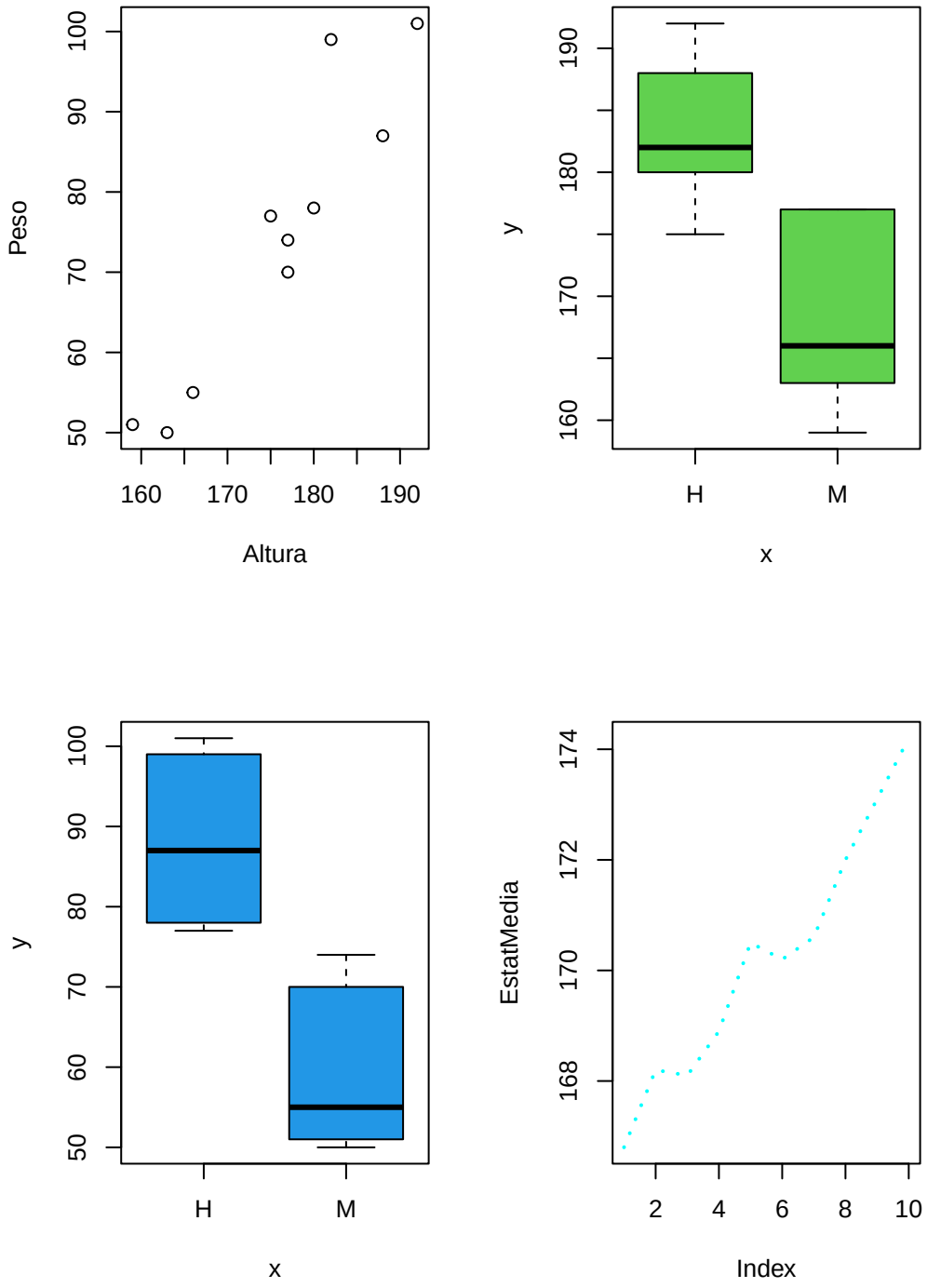


Utilizamos o parámetro `col`, para indicar a R a cor en que queremos o gráfico. Isto pódese facer numericamente ou ben indicando directamente a cor cunha orde do tipo `col="blue"`.

En ocasións pode interesarnos xerar unha única pantalla gráfica que conteña multitude de gráficas diferentes. Supoñamos, por exemplo, que na mesma pantalla pretendemos incluír as tres

gráficas xeradas anteriormente e ademais unha gráfica que represente a evolución da estatura media na última década (unha serie temporal).

```
par(mfrow = c(2, 2))
plot(Altura, Peso)
plot(Sexo, Altura, col = 3)
plot(Sexo, Peso, col = 4)
EstatMedia <- c(166.8, 168.2, 168.1, 168.9, 170.5, 170.2, 170.6, 172, 173.1, 174.2)
plot(EstatMedia, type = "l", lwd = 2, col = "cyan", lty = 3)
```



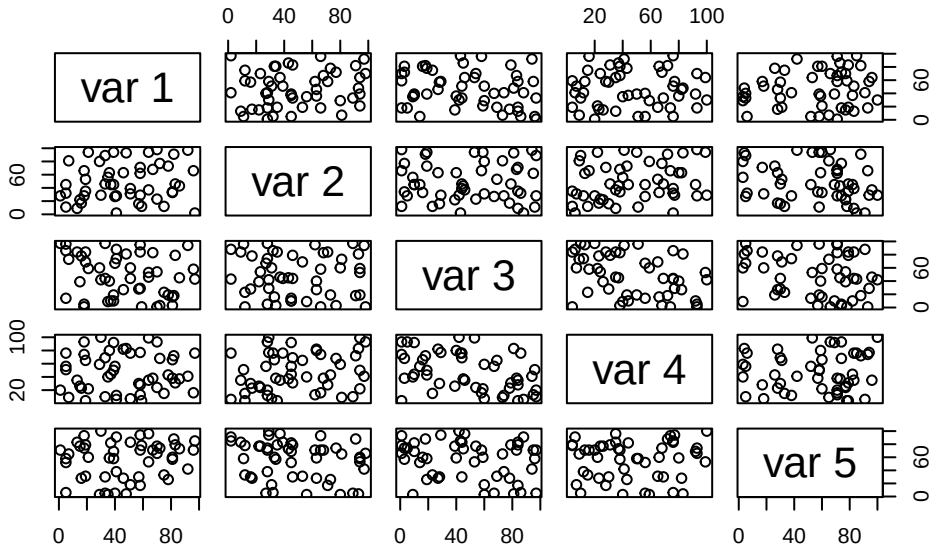
Aquí, coa orde `par(mfrow=c(2,2))` estamos a indicar a R que divida a pantalla de gráficos en 2 filas e 2 columnas, de modo que podamos incluír dentro $2 \times 2 = 4$ gráficos. Ademais, no último dos gráficos, estamos a usar parámetros que descoñeciamos: `type="l"` sérvenos para indicar que queremos que os puntos do gráfico se unan a través de liñas, outras opcións son

`type="p"` (para puntos) ou `type="b"` (para puntos e liñas); o parámetro `lty` sérvenos para indicar o tipo de liña desexado, por exemplo `lty=1` é a liña sólida habitual e `lty=3` é a liña descontinua; co parámetro `lwd` indicamos o ancho de liña desexado.

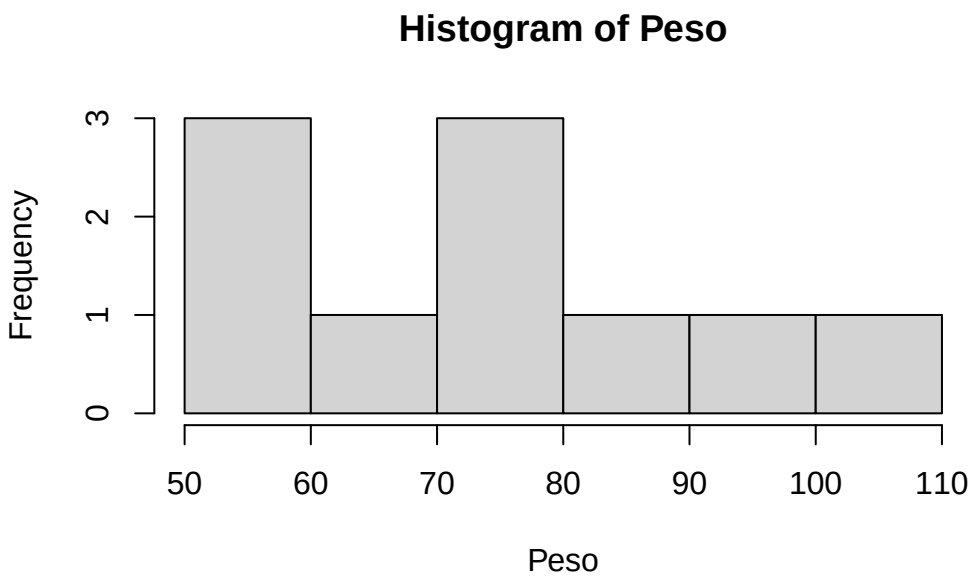
Outros comandos gráficos

Aínda que a función `plot()` é a máis importante e a máis comunmente usada, existen outras funcións que permiten levar a cabo gráficos de alto nivel: `pairs()` debuxa gráficos de dispersión 2 a 2 entre todas as columnas que conforman un data frame ou unha matriz numérica; `hist()` debuxa o histograma dunha variable; `image()` crea un rectángulo a partir dunha paleta de cores onde a cor cambia en función dos datos do terceiro argumento; `contour()` crea un gráfico de contorno en función dos valores do terceiro argumento e `persp()` é o equivalente 3D ao gráfico de dispersión 2D que debuxamos antes con `plot()`. A continuación móstranse uns breves exemplos de uso dos devanditos comandos.

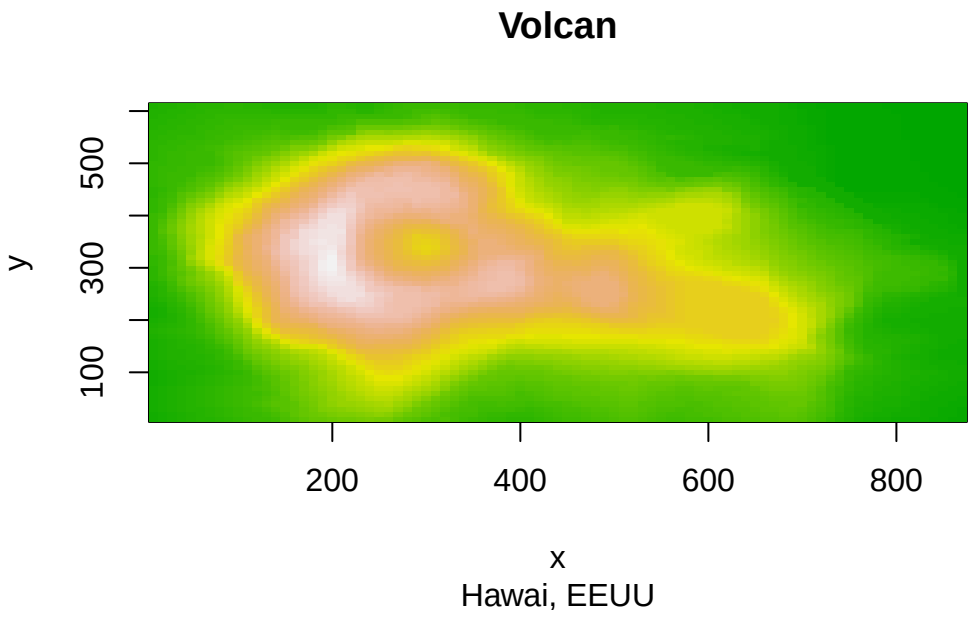
```
X <- matrix(sample(1:100, size = 50 * 5, replace = TRUE), nrow = 50, ncol = 5)
pairs(X)
```



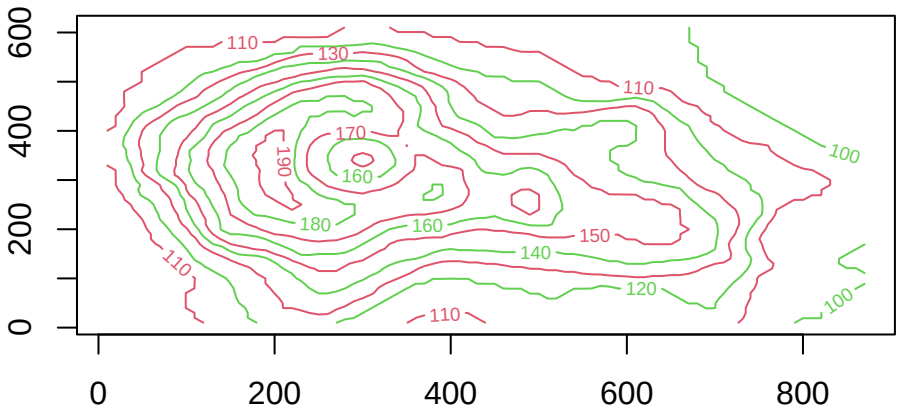

```
hist(Peso)
```



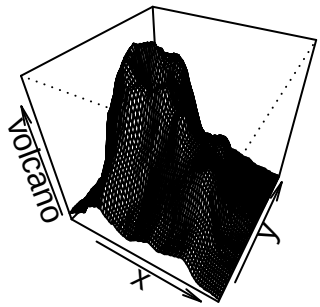

```r
x <- 10 * (1:nrow(volcano))
y <- 10 * (1:ncol(volcano))
image(x, y, volcano, col = terrain.colors(100), main = "Volcan", sub = "Hawai, EEUU")
```




```r
contour(x, y, volcano, col = 2:3)
```




```
persp(x, y, volcano, theta = 30, phi = 40)
```

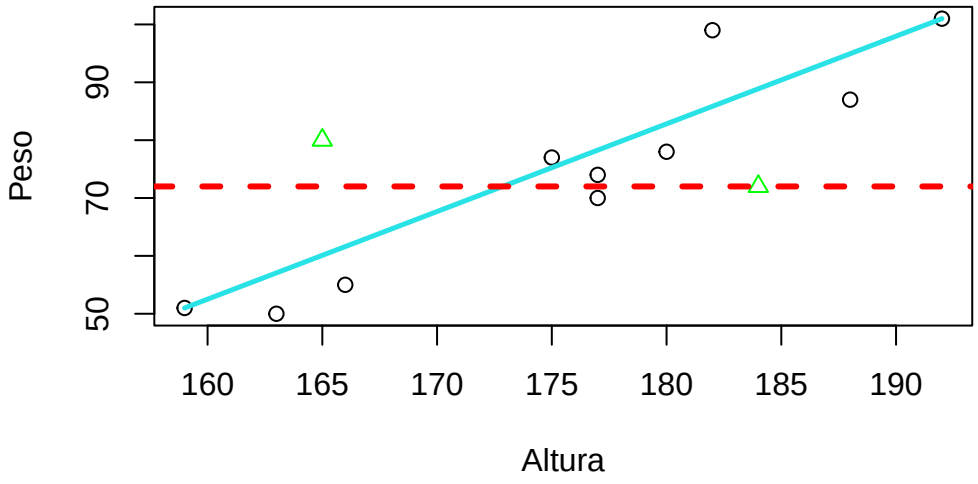


No uso que aquí damos da función `image()` podemos atopar un novo exemplo de parámetro que se pode aplicar sobre gráficos de alto nivel: `main` serve para dar un título ao gráfico, mentres que de forma análoga un subtítulo pode aplicarse a través de `sub`; `theta` e `phi` son parámetros de `persp` que nos permiten cambiar o punto de vista desde o que observamos a gráfica.

Gráficos de baixo nivel

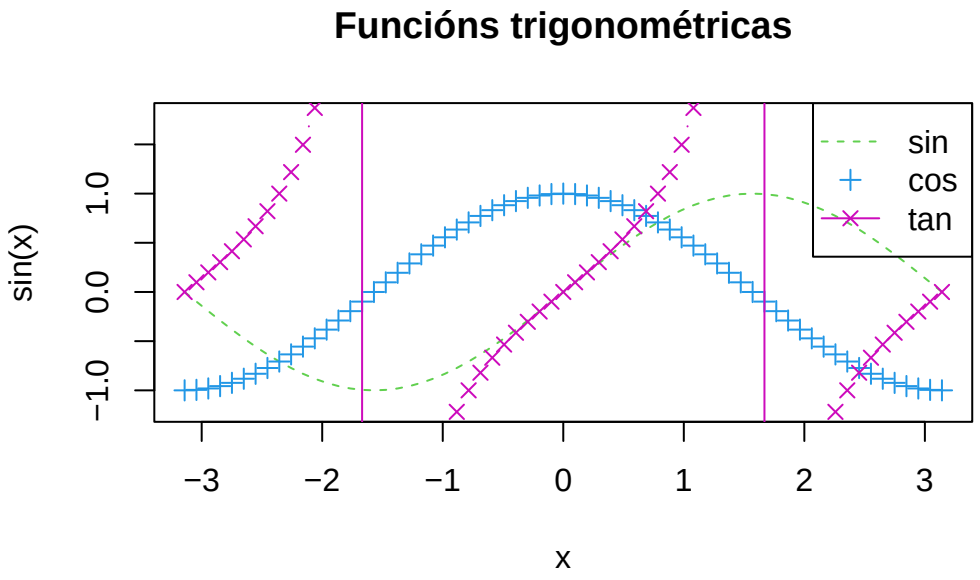
Algunhas veces as funcións que crean gráficos de alto nivel non producen exactamente a clase de gráfico que o usuario desexa. En tal caso, os comandos gráficos de baixo nivel poden ser usados para engadir información extra ao gráfico actual. No seguinte exemplo mostramos algúns dos gráficos de baixo nivel máis comúns: a función `points()` permite engadir a un gráfico un ou máis puntos especificando as súas coordenadas, o parámetro `pch` serve para que o usuario indique como quere sinalar tales puntos (círculo sólido ou oco, diamante, triángulo, ...); `lines()` permite engadir segmentos lineais a un gráfico preexistente, indicando as coordenadas dos puntos a unir; o comando `abline()` permite incluír rectas dentro dun gráfico, indicando o seu vector director e o termo independente.

```r
plot(Altura, Peso)
points(c(165, 184), c(80, 72), col = "green", pch = 2)
lines(c(159, 192), c(51, 101), lwd = 2.5, col = 29)
abline(a = 72, b = 0, col = "red", lty = 2, lwd = 3)
```



Ademais de `points()`, `lines()` e `abline()`, existen multitude de gráficos de baixo nivel en R. Aquí unicamente faremos referencia a dous máis: `legend()`, que permite incluír unha lenda nun gráfico preexistente, e `title()`, que permite engadir títulos e subtítulos a un gráfico xa creado, a condición de que non se fixo antes.

```r
x <- seq(-pi, pi, length = 65)
plot(x, sin(x), type = "l", ylim = c(-1.2, 1.8), col = 3, lty = 2)
points(x, cos(x), pch = 3, col = 4)
lines(x, tan(x), type = "b", lty = 1, pch = 4, col = 6)
legend("topright", c("sin", "cos", "tan"), col = c(3, 4, 6), lty = c(2, -1, 1), pch = c(-1, 3
title(main = "Funcións trigonométricas")
```



Funcións

Unha das vantaxes da linguaxe de programación R , e á vez una das claves para o seu crecemento e o desenvolvemento de paquetes e librarías, é que permite ao usuario crear obxectos do tipo función. As funcións son secuencias de comandos e ordes que devolven unha ou máis saídas, e que poden ser usadas en expresións posteriores.

A modo de exemplo, mostramos unha función cuxos parámetros de entrada son dúas vectores `y1` e `y2`, e que devolve como saída a media e a desviación típica da résta de ditos vectores.

```
calculo <- function(y1, y2) {
  Ry1y2 <- y1 - y2
  return(list("media" = mean(Ry1y2), "desv" = sd(Ry1y2)))
}
```

Pode observarse que un dos comandos fundamentais no interior da función é `return()`, que indica os parámetros de saída da función. Para chamar á función e obter a saída, o procedemento sería o seguinte.

```
# Xeramos x e z como vectores lonxitude 50 xerados dunha normal
x <- rnorm(50)
z <- rnorm(50)
# Chamamos á función e obtemos a saída
calculo(x, z)
```

Na chamada á función é importante saber que non é necesario que os parámetros de entrada tomen o mesmo nome que tiñan cando se creou a función. Unha función pode ter tantos parámetros de entrada ou saída como se queira, e poden ser de distintos tipos e clases.

Ademais, no interior das funcións é habitual empregar **estruturas de control** como condicións lóxicas (`if`, `else`) e bucles (`for`, `while`). Estas estruturas permiten que o comportamento da función dependa dos valores de entrada ou que se repitan determinadas operacións de forma sistemática, por exemplo, ao percorrer os elementos dun vector ou ao realizar cálculos iterativos.

Cando un usuario de R crea funcións é altamente probable que se cometan erros, tanto de programación como nos posibles cálculos/ordenes que levan acabo no seu interior, e máis especialmente canto máis longa sexa a función. En tales casos, e para depurar o programa, faise moi útil o uso da función `print()` para escribir saídas en pantalla.

Librarías

Como xa comentamos previamente, unha das vantaxes máis importantes de R é que cada usuario pode construír as súas propias funcións, almacenalas en librarías ou paquetes (**packages**) e facelas públicas de modo que estean dispoñibles para o resto de usuarios. Para coñecer que paquetes están xa instalados na nosa contorna R, executamos `library()`. Para que un determinado paquete funcione, non só ha de estar instalado, tamén ha de cargarse na consola de R. Para saber cales dos paquetes instalados están cargados na nosa contorna, executamos a orde `search()`.

En moitas ocasións, atopámonos con que o paquete que nos interesa non se atopa instalado en R. Para instalar un paquete, executamos `install.packages("nome_paquete")` . Unha vez instalado, cárgase en pantalla mediante a orde `library(nome_paquete)` e todas as súas funcións pasarán a estar dispoñibles.

Para obter información acerca dun paquete cargado na nosa consola, podemos facer un `help(nome_paquete)`, ou ben descargar a axuda en pdf do paquete dispoñible en CRAN.

Importación de datos

Ata agora vimos unha forma moi sinxela de introducir datos en R: directamente mediante o teclado. Na práctica, porén, o habitual é dispoñer dun conxunto de datos almacenado nun ficheiro, nunha base de datos ou noutra fonte externa. Polo tanto, é necesario saber como importar datos en R.

Existen diferentes funcións para levar a cabo esta tarefa, dependendo do formato de entrada dos datos. Centrarémonos na importación de ficheiros de texto (por exemplo, con extensións

.csv ou .txt), que será o formato empregado ao longo do curso. Aínda que R dispón de funcións específicas para importar datos desde outros formatos (por exemplo, ficheiros de Excel con extensión .xls ou .xlsx), na práctica o máis sinxelo nestes casos é converter previamente os ficheiros ao formato .csv.

Lectura de datos dun ficheiro de texto

A función `read.table()` é a que se usa normalmente para ler datos de arquivos externos tipo texto (con extensión .txt). O seu formato é moi sinxelo e o número de parámetros da función moi grande, tal e como se pode apreciar na axuda da mesma (tecleando `?read.table`).

Un exemplo de aplicación de dita orde podémolo ver descargándonos en formato texto calquera base de datos de interese que atopemos pola rede. Podemos descargar, a modo de exemplo, a información sobre o cociente de computadores persoais por cada 100 habitantes dispoñible no Campus Virtual. A orde correcta para importar os devanditos datos a R sería:

```
datosPC <- read.table(file = "datos/personal_computers_per_100_people.csv", header = TRUE, s
```

En `file` indicámoslle a R a ruta a seguir (no propio computador) para chegar ao arquivo desexado; `header` é un parámetro lóxico para indicar a R se a primeira fila do arquivo corresponde (`TRUE`) ou non (`FALSE`) a unha cabeceira indicando os nomes das variables; con `sep` indicámoslle a R cal é a separación entre variables nun arquivo (espazo, separador, coma, ...). Opcionalmente, poderíanse usar calquera dos outros parámetros que aparecen na axuda, como `dec`, co cal indicamos a R o separador decimal utilizado no arquivo de texto de partida.

Importante: Cada vez que abrimos unha sesión de R, o programa sitúase no directorio de traballo por defecto (working directory). Este directorio de traballo é o sitio no que por defecto R le e escribe ficheiros. Para saber cal é o teu directorio de traballo actual, escribe:

```
getwd()
```

Cada vez que empezas unha sesión de R, o primeiro que debes facer é cambiar o directorio de traballo ao cartafol onde gardas os teus scripts e ficheiros de datos. Para cambiar o directorio de traballo, selecciona **Arquivo Cambiar dir...** no menú superior e elixe o cartafol onde gardaches os ficheiros de datos. Tamén se pode cambiar o directorio de traballo co comando `setwd()`.

A orde `read.csv()` é o equivalente a `read.table()` para importar a R datos procedentes de arquivos Excel con extensión .csv (nalgúns casos será necesario usar `read.csv2()`). Na axuda de R podemos atopar o seu modo de uso:

```
help(read.csv)
```

Un exemplo de aplicación podémolo ver tamén a partir dos datos que xa usamos antes:

```
datosPC <- read.csv(file = "datos/personal_computers_per_100_people.csv")
```

Acceso ás bases de datos de R

Ao redor dun centenar de bases de datos son fornecidas por R , ben a través do paquete `datasets` (que se instala por defecto con R , ben a través doutros paquetes. Podemos ver a lista de bases de datos ás que temos acceso desde R usando o comando `data()`. O acceso ás bases de datos que se atopan nos paquetes basee de R realízase a través do seu nome. Obviamente, antes de usar ditas bases, estaremos interesados en coñecer as súas características, para o que faremos un `help()` sobre o nome das devanditas bases:

```
CO2
help(CO2)
```

Pode ocorrer que en determinadas ocasións esteamos interesados en dispor das bases de datos presentes en paquetes de R que non veñen instalados por defecto co programa. Se por exemplo desexamos coñecer as bases de datos no paquete `rpart`:

```
data(package = "rpart")
```

Se, doutra banda, desexamos obter acceso á base de datos `imports85` dentro do paquete `randomForest`, escribiremos:

```
data(imports85, package = "randomForest")
```

Lembra que para que o anterior comando non de erro, debes instalar a libraría mediante a orde `install.packages("randomForest")`

EXERCICIOS

> Boletín de exercicios
>
> Descargar (PDF)

Arquivos de datos

personal_computers_per_100_people.csv
spotify.csv
redes.csv
vjuegos.csv

1Estatística descritiva

Como comentamos nas sesións expositivas, unha análise estatística normalmente comeza cun conxunto de datos que usamos como mostra representativa dunha poboación máis ampla. O obxectivo da estatística descritiva é organizar, resumir e presentar a información contida nesta mostra, empregando principalmente táboas de frecuencias, medidas numéricas e representacións gráficas.

Esta fase, relacionada coa análise exploratoria de datos (EDA, polas súas siglas en inglés), tamén inclúe o preprocesado dos datos como paso fundamental. Nesta etapa realízanse tarefas como eliminar ou imputar valores ausentes, corrixir formatos incorrectos e detectar e tratar valores atípicos.

1.1. Táboas de frecuencias

Unha táboa de frecuencias está composta por varias columnas: as frecuencias absolutas, as frecuencias relativas e as súas correspondentes acumuladas, cando teñan sentido. Comezaremos por obter as frecuencias absolutas, para o que se utiliza a función `table()`. No caso das frecuencias relativas é tan sinxelo como dividir as frecuencias absolutas previamente calculadas entre o tamaño muestral, que se pode obter facilmente coa función `length()`.

Consideremos unha mostra de 20 observacións da variable X = Satisfacción co servizo de atención ao cliente, que introducimos en R do seguinte modo:

```
satisfaccion <- c("boa", "boa", "regular", "moi boa", "mala", "boa", "boa", "regular", "moi r
```

Esta variable así definida é almacenada en R como unha variable de tipo carácter. Para facela de tipo factor (que é o tipo de variables que R emprega para as variables cualitativas) e que permite comprender que existen unhas determinadas categorías que se van repetindo na nosa mostra faremos:

```
satisfaccion <- as.factor(satisfaccion)
levels(satisfaccion)
```

Por defecto, e tal e como se observa na saída, R ordena as categorías alfabeticamente. Se queremos que aparezan nunha orde "lóxica" debemos de permutarlas segundo ese criterio:

```
satisfaccion <- factor(satisfaccion, levels = levels(satisfaccion)[c(4, 2, 5, 1, 3)])
levels(satisfaccion)
```

Nun primeiro lugar obteremos as frecuencias absolutas e relativas:

```
Fabs <- table(satisfaccion)
Fabs
```

```
Frel <- table(satisfaccion) / length(satisfaccion)
Frel
```

Para obter as frecuencias acumuladas (tanto absolutas como relativas) que neste caso ten sentido calcular por tratarse dunha variable cualitativa ordinal, utilízase a función `cumsum()`:

```
FabsAcum <- cumsum(Fabs)
FabsAcum
```

```
FrelAcum <- cumsum(Frel)
FrelAcum
```

Para ver a táboa de frecuencias completa, podemos "pegar por columnas" os vectores calculados previamente coa función `cbind()`:

```
Taboa <- cbind(Fabs, Frel, FabsAcum, FrelAcum)
Taboa
```

Vexamos agora un exemplo con datos reais. No conxunto de datos `quine` da libraría `MASS` recóllese información sobre o absentismo escolar de 146 escolares en Nova Gales (Australia). A función `head()` amosa as primeiras filas do conxunto de datos. Entre as variables recollidas están o sexo (`Sex`) e a idade (`Age`) medida en grupos. Para construír as correspondentes táboas de frecuencias empregaremos o seguinte código (hai que ter en conta que a variable `Age` está categorizada e é cualitativa ordinal, mentres que `Sex` é cualitativa nominal).

```
library(MASS)
head(quine)
ni <- table(quine$Age)
fi <- ni / length(quine$Age)
Nin <- cumsum(ni)
Fi <- cumsum(fi)
tabAge <- cbind(ni, fi, Nin, Fi)
tabAge
```

Podemos estar interesados en redondear a un número de cifras decimais concreto, para o que podemos utilizar a función `round()`.

```r
tabAge <- round(tabAge, 2)
tabAge
```

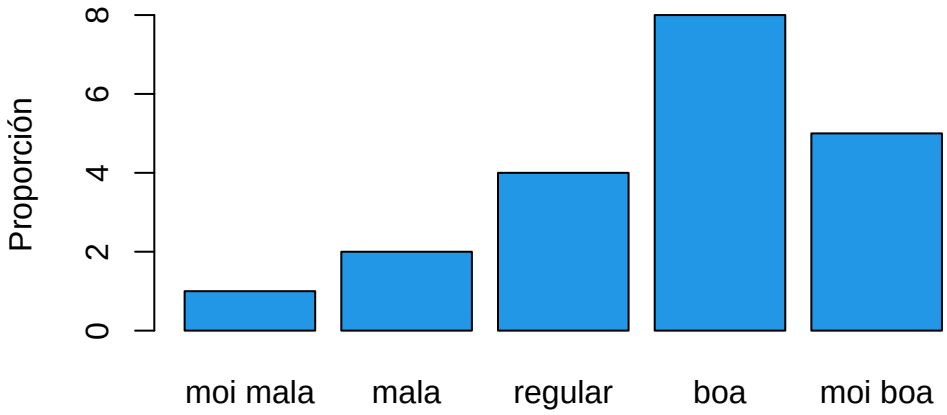
Para a variable `Sex` non calculamos frecuencias acumulados, pois non ten sentido.

```r
ni <- table(quine$Sex)
fi <- ni / length(quine$Sex)
tabSex <- cbind(ni, fi)
tabSex
```

1.2. Representacións gráficas

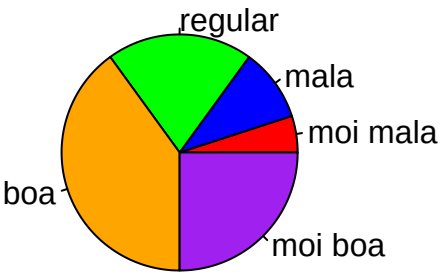
As representacións gráficas máis simples para variables cualitativas ou cuantitativas discretas son o **diagrama de barras** e o **diagrama de sectores**, como vimos no Tema 1. Sobre os datos do exemplo inicial sobre a variable X = Satisfacción co servizo de atención ao cliente, poderiamos construír un diagrama de barras executando:

```r
barplot(table(satisfaccion), ylab = "Proporción", col = 4)
```



e para debuxar o diagrama de sectores fariamos:

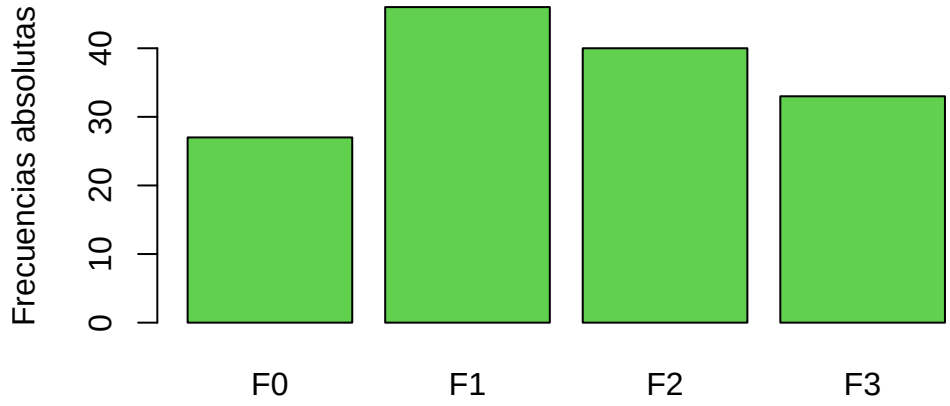
```r
pie(table(satisfaccion), main = "Satisfacción", col = c("red", "blue", "green", "orange", "pu
```



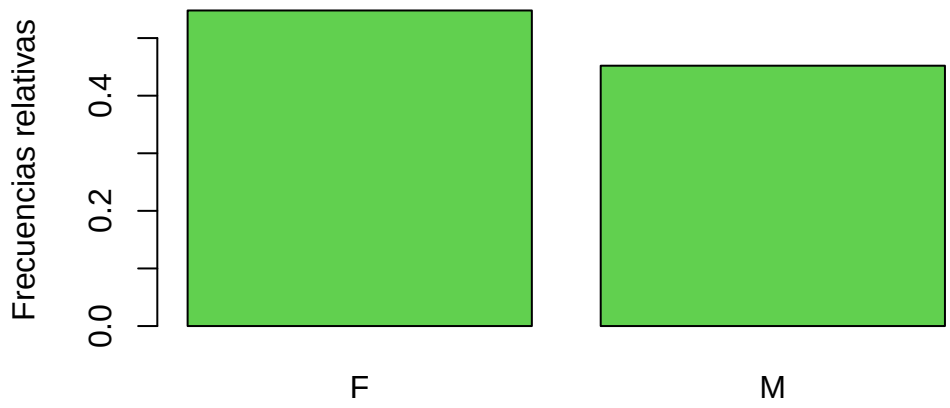
Nótese que calquera das representacións anteriores poderíanse obter para unha táboa de frecuencias relativas simplemente dividindo entre o tamaño muestral, é dicir, `length(satisfaccion)`.

No caso do exemplo con datos reais, representaremos tanto `Age` como `Sex` mediante diagramas de barras, na primeira farémolo coas frecuencias absolutas e na segunda coas relativas:

```
barplot(table(quine$Age), ylab = "Frecuencias absolutas", col = 3)
```



```r
barplot(table(quine$Sex) / length(quine$Sex), ylab = "Frecuencias relativas", col = 3)
```



Para variables cuantitativas continuas consideremos o exemplo dos datos do geyser Old Faithful, dispoñible dentro da libraría `MASS` no datase `geyser`:

```r
library(MASS)
names(geyser)
duration <- geyser$duration
```

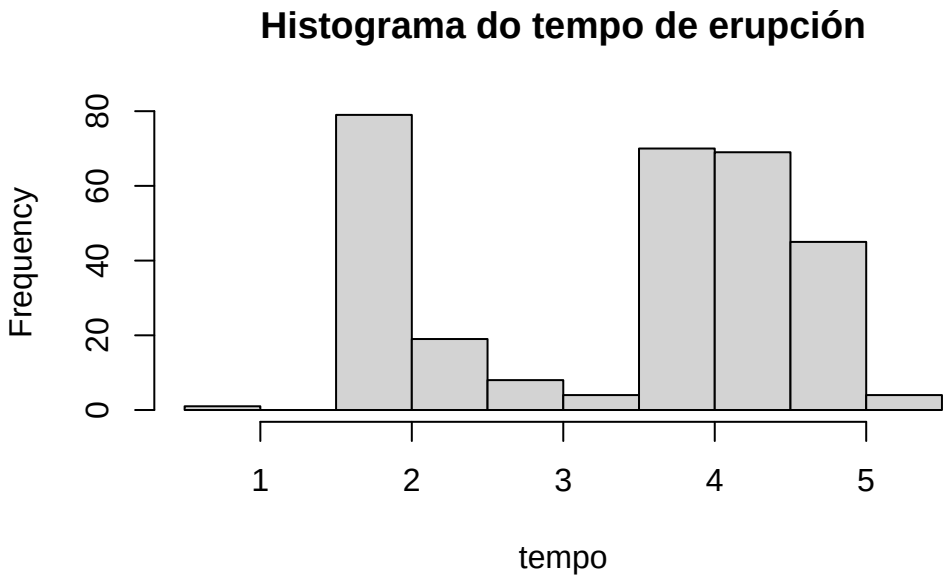
Se debuxamos un diagrama de barras ou un diagrama de sectores para os datos de duración de erupciones:

```r
barplot(duration)
pie(duration)
```

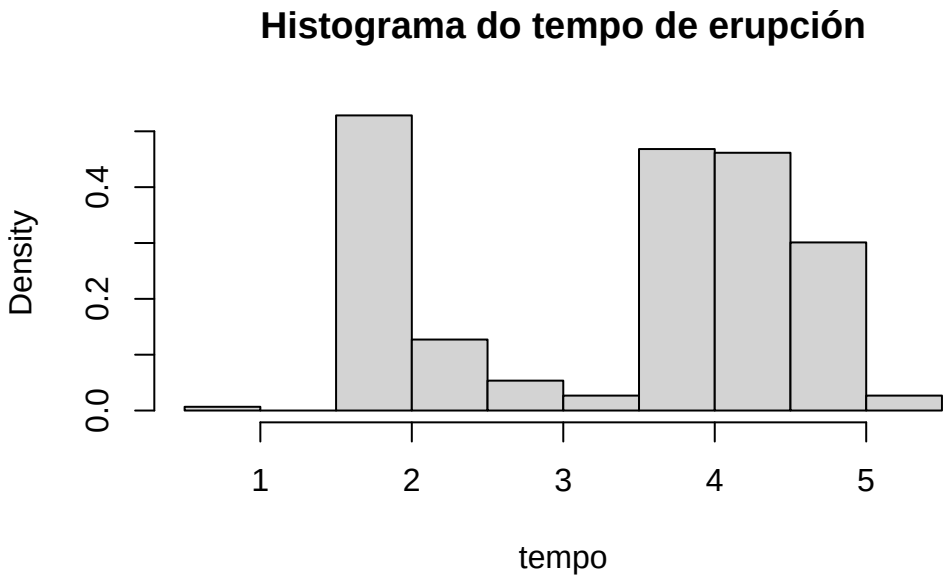
ponse de manifesto que ningún deles resulta informativo cando se aplican sobre variables cuantitativas continuas (como xa sabiamos pola teoría estudada). Unha forma de poder aplicalos é categorizando, ou facendo un resumo dos valores da variable por intervalos, aínda que iso leva certa perdida de información.

Un dos gráficos adecuados para este tipo de datos é o histograma, que podemos debuxar en frecuencias absolutas ou relativas empregando a función `hist()` co argumento `freq=FALSE` para a densidade de frecuencia.

```r
hist(duration, main = "Histograma do tempo de erupción", xlab = "tempo")
```

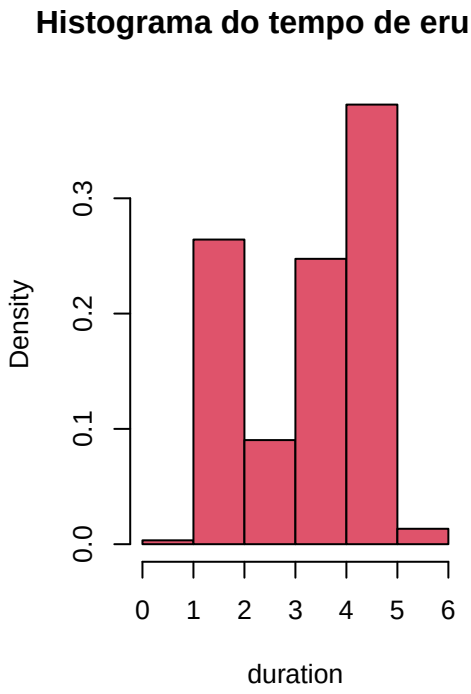


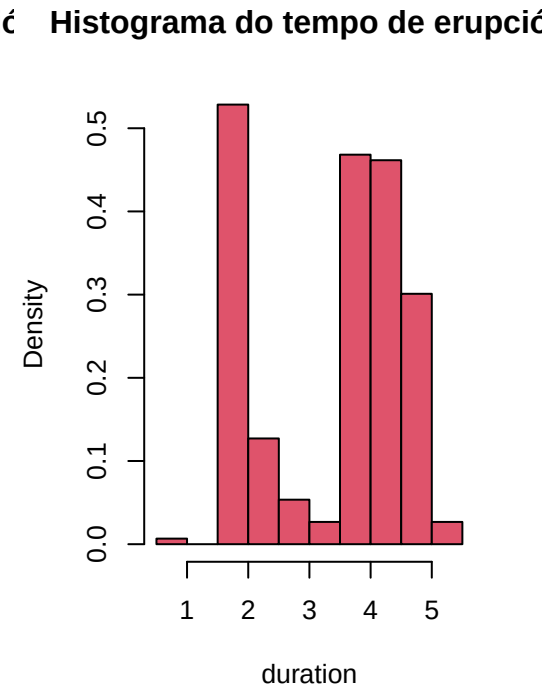

```
hist(duration, main = "Histograma do tempo de erupción", xlab = "tempo", freq = F)
```

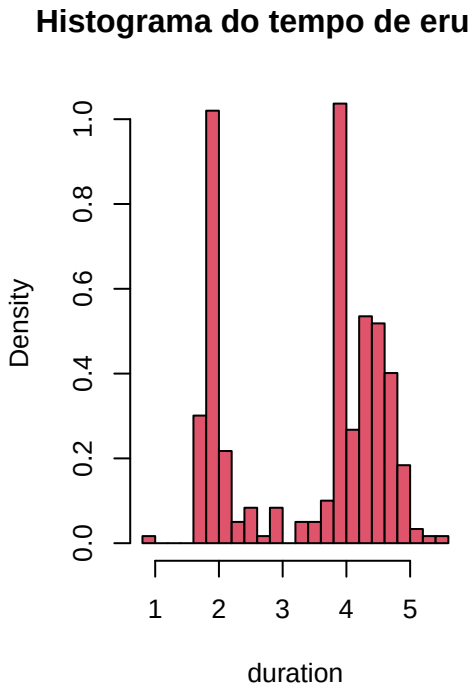


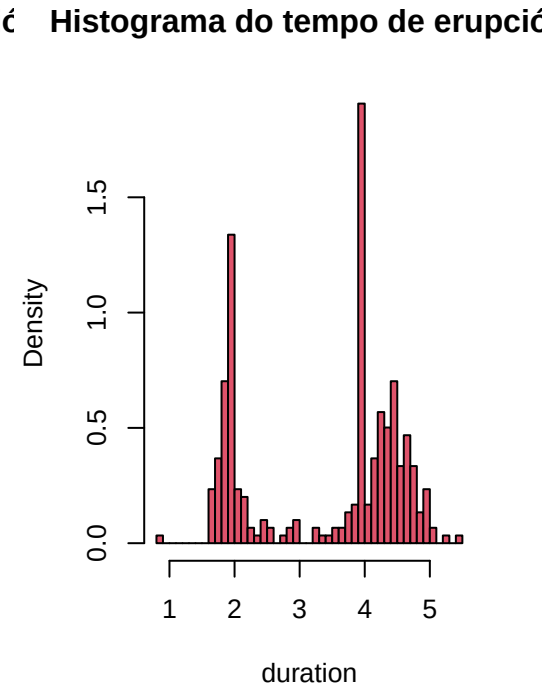
O histograma é sensible ao número de intervalos que se consideran. A continuación mostramos histogramas para distintos números de intervalos. En R, isto contrólase mediante o argumento `breaks`, que define como se dividen os datos. Se se lle pasa un valor numérico, R tentará crear aproximadamente ese número de intervalos. Tamén se poden especificar os puntos de corte exactos dos intervalos, garantindo así o número e a posición exacta das barras do histograma.

```r
par(mfrow = c(2, 2))
hist(duration, freq = F, breaks = 4, col = 2, main = "Histograma do tempo de erupción")
hist(duration, freq = F, breaks = 9, col = 2, main = "Histograma do tempo de erupción")
hist(duration, freq = F, breaks = 24, col = 2, main = "Histograma do tempo de erupción")
hist(duration, freq = F, breaks = 49, col = 2, main = "Histograma do tempo de erupción")
```


Histograma do tempo de erupció
Density
0.0
0.1
0.2
0.3
0
1
2
3
4
5
6
duration


Histograma do tempo de erupció
Density
0.0
0.1
0.2
0.3
0.4
0.5
1
2
3
4
5
duration


Histograma do tempo de erupció
Density
0.0
0.2
0.4
0.6
0.8
1.0
1
2
3
4
5
duration


Histograma do tempo de erupció
Density
0.0
0.5
1.0
1.5
1
2
3
4
5
duration

1.3. Medidas características

A función `summary()` proporciónanos información xeral sobre calquera variable. No caso de variables cuantitativas, o que devolve é o valor do mínimo, o máximo, a media e os cuartiles. Para variables cualitativas é máis simple e devolve unicamente un reconto por categorías, é dicir, as frecuencias absolutas (e por tanto o mesmo que a función `table` que vimos anteriomente).

Para calcular a **media muestral** utilizamos a función `mean()` coa que tamén se poden obter medias truncadas mediante o argumento `trim`, que nos permite introducir unha fracción (de 0 a 0.5) que indica a porcentaxe de datos máis altos e máis baixos que se eliminan. Deste xeito, obtense unha media robustificada no sentido de que xa non é tan sensible á presenza de datos atípicos.

Non existe unha función en R que calcule a **varianza muestral**. A función `var()` devolve a **case-varianza** muestral, polo que debemos transformar o resultado para obter a varianza:

$$s_c^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2 \Longrightarrow s^2 = \frac{n-1}{n}s_c^2$$

Do mesmo xeito que ocorre coa función `var()`, a función `sd()` proporciona a **case-desviación típica muestral**, se queremos calcular a **desviación típica muestral** podemos obtela empregando a súa definición como raíz cadrada da varianza.

Outras funcións para obter medidas características que se viron no Tema 1 son:

- `median()` para calcular a **mediana** muestral.
- `min()` para calcular o **mínimo** da mostra.
- `max()` para calcular o **máximo** da mostra
- `diff(range())` para calcular o **rango muestral**.
- `quantile()` para calcular diferentes **cuantiles muestrales** adaptando o argumento `probs`.
- `IQR()` para calcular o **rango intercuartílico** da mostra.

Retomemos o exemplo dos datos de `geyser` da libraría `MASS` e a variable `duration` para a que calcularemos algunhas destas medidas:

```
mean(duration) # media
median(duration) # mediana
quantile(duration, probs = 0.7) # cuantil 0.7
quantile(duration, probs = c(0.25, 0.5, 0.75)) # cuartiles
var(duration) # case-varianza
diff(range(duration)) # rango
```

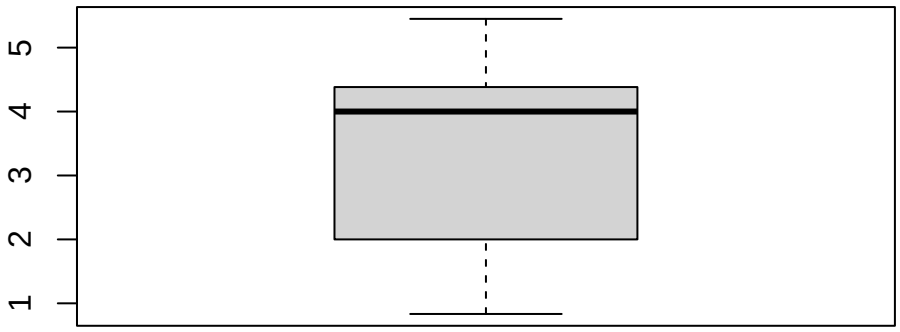
Pódennos interesar tamén a simetría e a forma da variable. Para obter os coeficientes de asimetría e curtosis, debemos cargar a libraría `moments`. As funcións a utilizar son:

- `skewness()` para calcular o **coeficiente de asimetría**.
- `kurtosis()` para calcular o **coeficiente de curtosis**.

```
library(moments)
skewness(duration)
kurtosis(duration)
```

Como tamén vimos, unha forma de resumir a información das medidas características é representando un **diagrama de caixa** ou Box-Plot. Obteriamos esta representación en R da seguinte forma:

```
boxplot(duration)
```

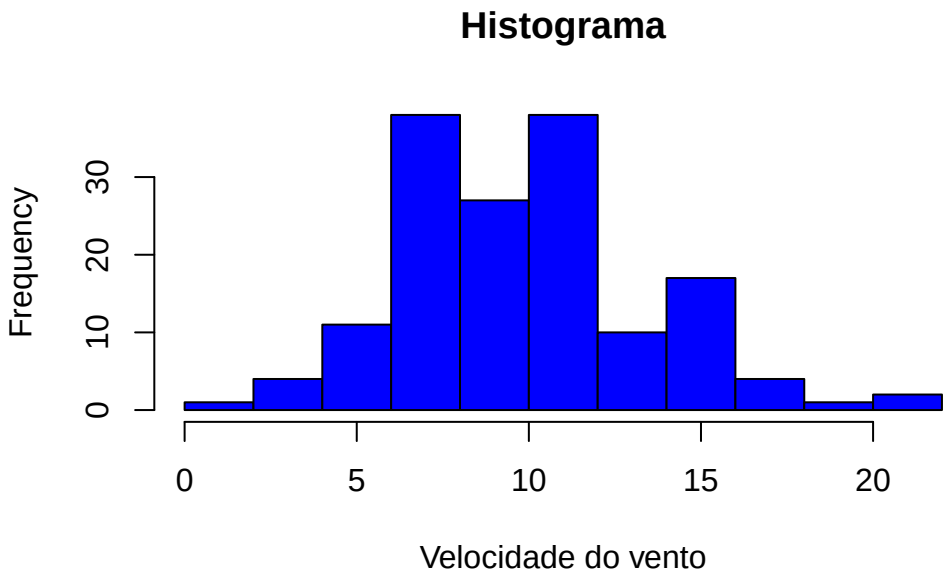


O diagrama de caixa ou Box-Plot resume nun único gráfico algunhas das medidas de centralización e dispersión máis usadas en estatística. A barra negra central mostra onde se sitúa a mediana, mentres que os extremos da caixa son o primeiro cuartil, Q_1 (inferior) e o terceiro cuartil, Q_3 (superior). Deste xeito, a altura da caixa é o rango intercuantílico $Q_3 - Q_1$. No caso de que a variable sexa simétrica, a mediana ha de situarse nun punto equidistante de ambos os cuartiles e coincidirá coa media. Os extremos das barras do Box-Plot (bigotes) representan os límites superior e inferior. As observacións que caian fose destes límites son datos atípicos.

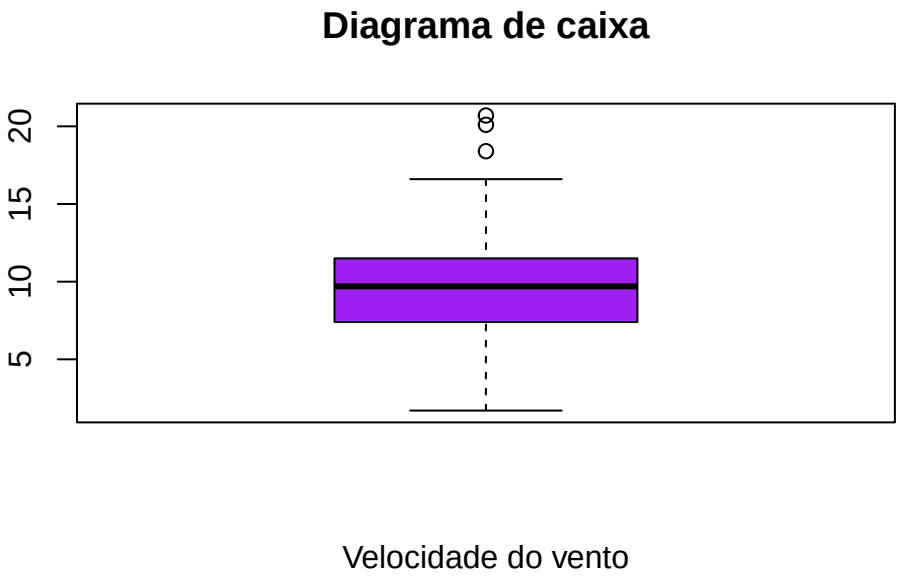
Vexamos agora un novo exemplo con datos de calidade de aire. O conxunto de datos `airquality` contén información sobre calidade do aire. Entre outras variables, recóllese a velocidade de vento rexistrada (`Wind`), en millas por hora, que se trata dunha variable cuantitativa continua. Para representar gráficamente esta variable necesitamos:

```
names(airquality)
```

```r
hist(airquality$Wind, col = "blue", main = "Histograma", xlab = "Velocidade do vento")
```




```r
boxplot(airquality$Wind, col = "purple", main = "Diagrama de caixa", xlab = "Velocidade do ve
```



1.4. Tipificación e transformación de datos en R

En moitas análises estatísticas é útil **tipificar** os datos (converter a escala para que teñan media 0 e desviación estándar 1) ou aplicar transformacións como o **logaritmo** para normalizar distribucións sesgadas.

1.4.1. Tipificar datos

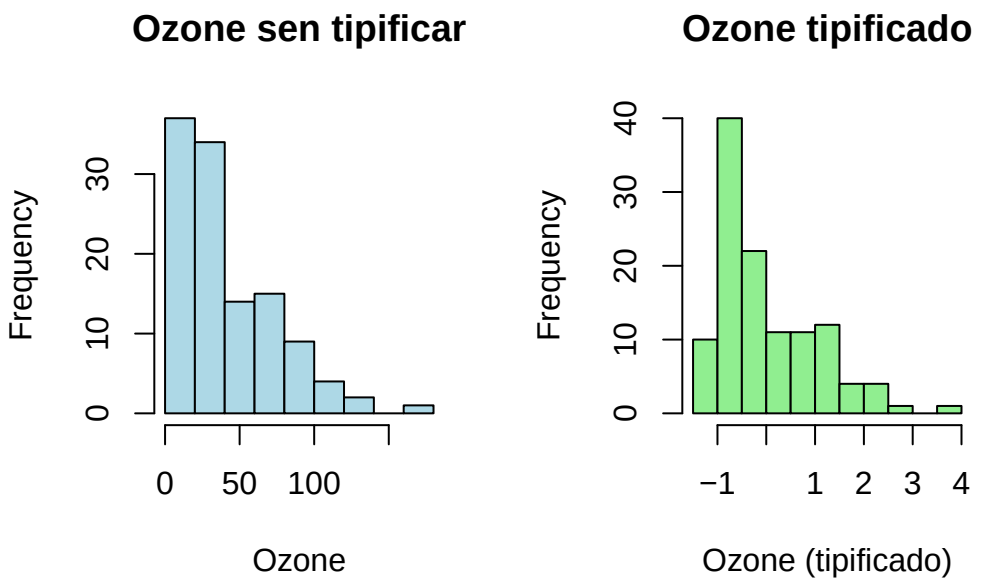
Para tipificar un conxunto de observacións en R podemos usar a función `scale()`. Usaremos como exemplo os datos da columna `Ozone` en `airquality`.

```
ozone <- airquality$Ozone
mean(ozone, na.rm = TRUE)
sd(ozone, na.rm = TRUE)
ozone_tipificado <- scale(ozone)
mean(ozone_tipificado, na.rm = TRUE)
sd(ozone_tipificado, na.rm = TRUE)
```

Observa que `airquality$Ozone` contén algúns valores `NA`. Se se executa directamente `mean(airquality$Ozone)` o resultado tamén será `NA`. Por iso, para calcular a media ignorando os valores ausentes, usamos o argumento `na.rm = TRUE`.

Representamos a continuación os histogramas dos datos orixinais e os datos estandarizados.

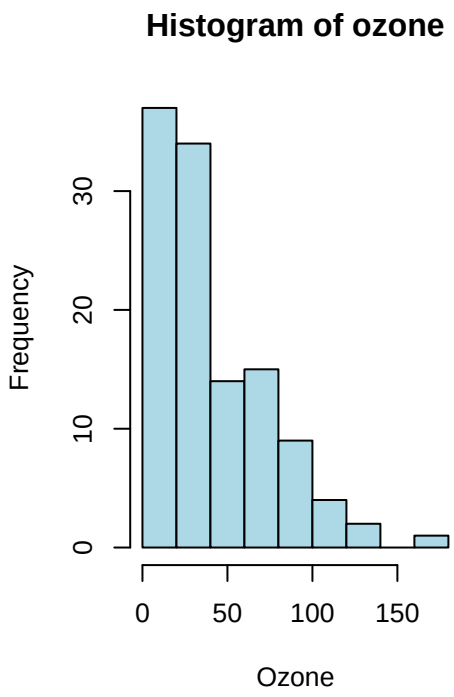
```
par(mfrow = c(1, 2))
hist(ozone, main = "Ozone sen tipificar", col = "lightblue", xlab = "Ozone")
hist(ozone_tipificado, main = "Ozone tipificado", col = "lightgreen", xlab = "Ozone (tipifica
```

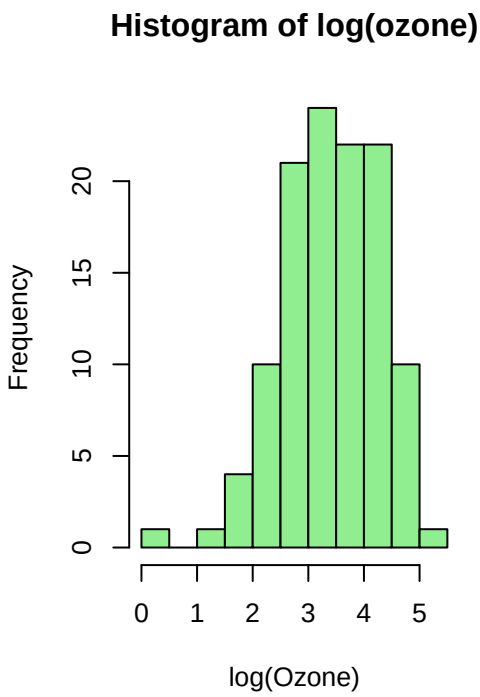


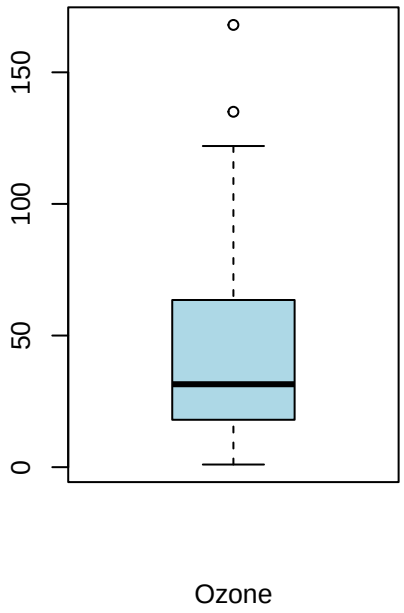
1.4.2. Transformación de datos

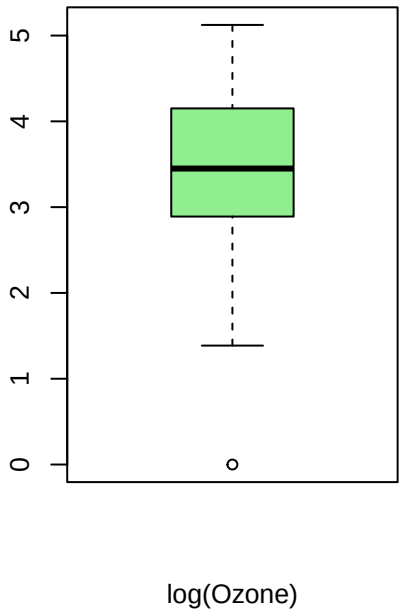
En moitos caso, como ocorre coa variable `Ozone`, os datos presentan unha distribución sesgada con algúns valores moi altos, o que pode dificultar a análise e a visualización. Aplicar neses casos unha **transformación logarítmica** axuda a reducir esta asimetría, aproximando a distribución a unha forma máis normal.

```
par(mfrow = c(2, 2))
hist(ozone, col = "lightblue", xlab = "Ozone")
hist(log(ozone), col = "lightgreen", xlab = "log(Ozone)")
boxplot(ozone, col = "lightblue", xlab = "Ozone")
boxplot(log(ozone), col = "lightgreen", xlab = "log(Ozone)")
```


Histogram of ozone
Frequency
0
10
20
30
0
50
100
150
Ozone


Histogram of log(ozone)
Frequency
0
5
10
15
20
0
1
2
3
4
5
log(Ozone)


0
50
100
150
Ozone


0
1
2
3
4
5
log(Ozone)

1.5. Estatística descritiva bidimensional

O obxectivo fundamental deste apartado é introducir os principais procedementos que R pon á nosa disposición para o estudo da estatística descritiva con dúas variables.

1.5.1. Táboas de continxencia

A función `table()` tamén permite construír **táboas de continxencia** nas cales se representan datos de dúas (ou máis) variables categorizadas. Para o exemplo dos datos de absentismo escolar cos que xa traballamos con anterioridade, podemos representar, de maneira conxunta, aos individuos segundo sexo e idade, así como obter as distribucións marxinais coa función `margin.table()` e completar a táboa coa función `addmargins()` da seguinte forma:

```
Taboa <- table(quine$Age, quine$Sex)
Taboa
margin.table(Taboa, 1)
margin.table(Taboa, 2)
addmargins(Taboa)
```

Nota. O cálculo das distribucións marxinais tamén pode facerse usando as funcións `colSums()`, `cbind()`, `rowSums()` e `rbind()`.

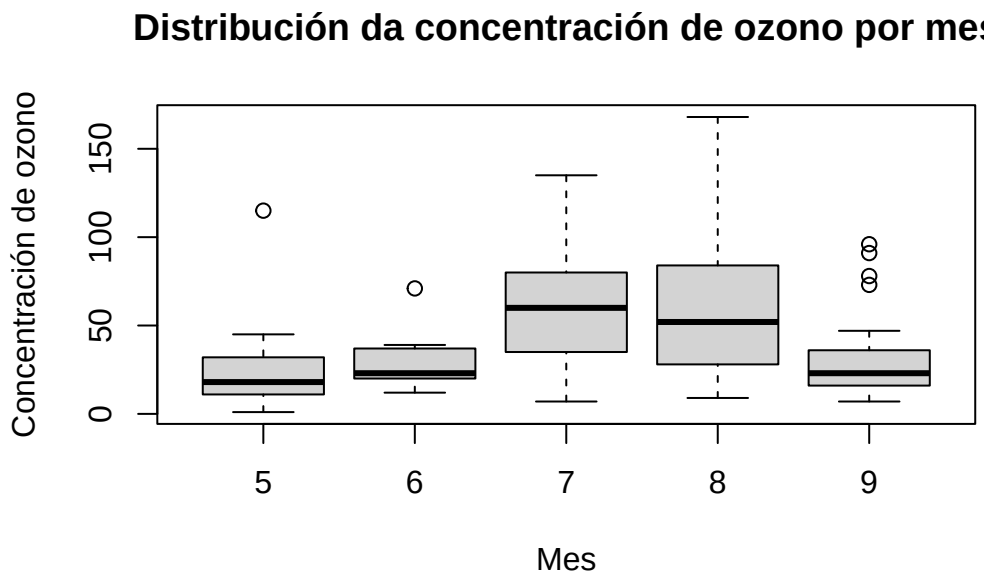
1.5.2. Boxplot para unha variable continua segundo unha categórica

Xa vimos que o diagrama de caixa (boxplot) é unha ferramenta gráfica moi útil para analizar a distribución dunha variable cuantitativa continua, xa que permite visualizar de maneira sintética medidas de posición (mediana e cuartís), dispersión e posibles valores atípicos.

Cando se combina unha variable continua cunha categórica, o boxplot resulta especialmente informativo: constrúese un diagrama de caixa para cada categoría, o que facilita a comparación das distribucións da variable continua entre os distintos grupos. Deste xeito pódense detectar diferenzas na tendencia central, na variabilidade ou na presenza de valores extremos segundo a categoría considerada, sendo unha representación moi habitual na análise exploratoria de datos.

Por exemplo, cos datos de `airquality` podemos representar mediante diagramas de caixa os valores da temperatura (`Temp`) en función do mes (`Month`). Cada caixa corresponde a un mes distinto, que no conxunto de datos `airquality` vai de maio (5) a setembro (9).

```r
boxplot(
  airquality$Ozone ~ airquality$Month,
  main = "Distribución da concentración de ozono por mes",
  xlab = "Mes",
  ylab = "Concentración de ozono"
)
```

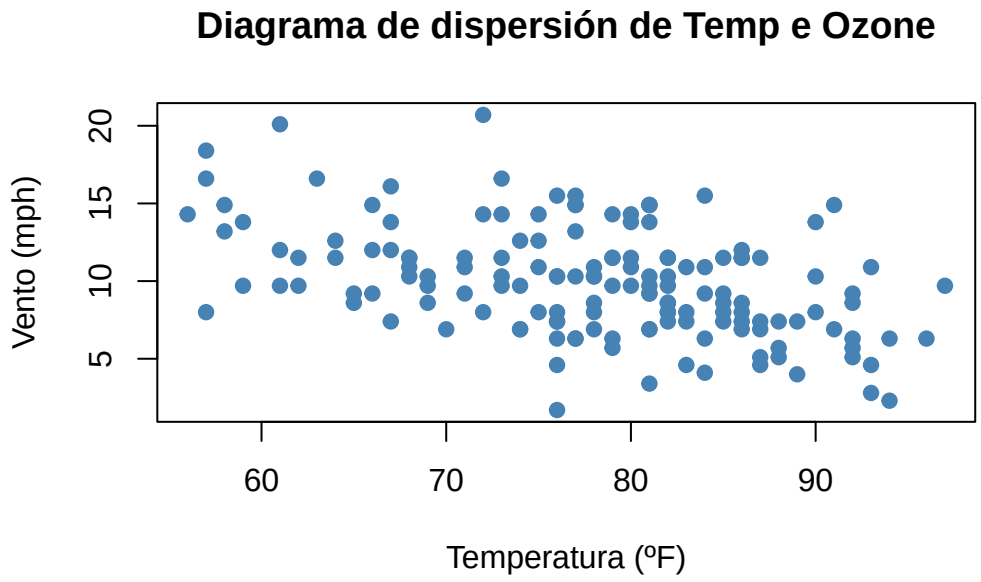


1.5.3. Análise de dúas variables cuantitativas

Agora imos analizar de forma conxunta dúas variables continuas do conxunto de datos `airquality`.

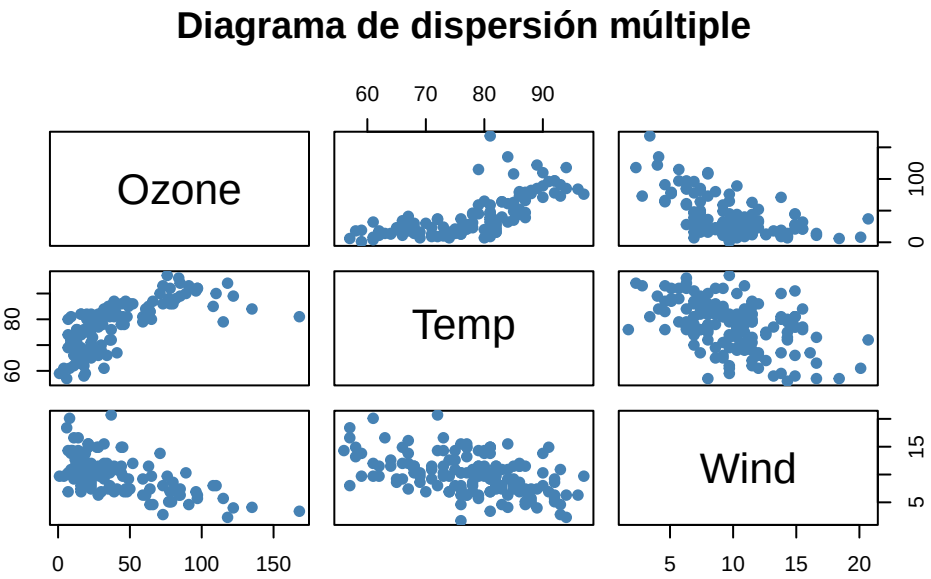
En primeiro lugar consideramos a temperatura (`Temp`) e a velocidade do vento (`Wind`). Gardamos cada unha nunha variable e representamos o diagrama de dispersión correspondente coa función `plot()`.

```r
x <- airquality$Temp
y <- airquality$Wind
plot(x, y,
  main = "Diagrama de dispersión de Temp e Ozone",
  xlab = "Temperatura (ºF)",
  ylab = "Vento (mph)",
  pch = 19, col = "steelblue"
)
```



Se queremos analizar varias variables á vez, podemos usar `pairs()` para facer un diagrama de dispersión matricial.

```
vars <- airquality[, c("Ozone", "Temp", "Wind")]
pairs(vars, main = "Diagrama de dispersión múltiple", pch = 19, col = "steelblue")
```



A **covarianza** mide a variabilidade conxunta de dúas variables. En R calcúlase a covarianza corrixida (con denomidar $n-1$) coa función `cov()`. A **correlación** estandariza a covarianza para que teña valores entre -1 e 1, permitindo comparar a forza da relación entre variables con

distintas unidades. En R calcúlase coa función `cor()`. Calculamos a continuación a covarianza corrixida e a correlación entre a temperatura e a valocidade do vento.

```
cov(x, y)
cor(x, y)
```

Se agora queremos analizar `Temp` e `Ozone`, debemos ter en conta que `airquality$Ozone` contén algúns valores `NA`. Se tentamos calcular directamente covarianza ou correlación, R devolverá `NA`. Para solucionar isto, usamos o argumento `use = "complete.obs"`, que indica a R que só considere os pares de valores completos (sen `NA`):

```
z <- airquality$Ozone
cov(x, z, use = "complete.obs")
cor(x, z, use = "complete.obs")
```

1.6. Táboa resumo de procedementos e funcións

A seguinte táboa resume os principais cálculos e visualizacións estatísticas comentados no tema, xunto coas funcións correspondentes en R.

Procedementos	**Funcións**
Tamaño muestral	`length()`
Táboas	`table()`
Sumas acumuladas	`cumsum()`
Diagrama de barras	`barplot()`
Diagrama de sectores	`pie()`
Histograma	`hist()`
Diagrama de caixa	`boxplot()`
Resumo de medidas	`summary()`
Media	`mean()`
Cuasivarianza	`var()`
Mediana	`median()`
Cuantil	`quantile()`
Rango intercuartílico	`IQR()`
Rango	`diff()` e `range()`
Asimetría	`skewness()`
Curtose	`kurtosis()`
Marxinais	`addmargins()`
Diagrama de dispersión	`plot()`
Matriz de dispersión	`pairs()`

Procedementos	Funcións
Covarianza (corrixida)	`cov()`
Correlación	`cor()`

A táboa seguinte recolle outras funcións básicas auxiliares.

Outros procedementos	Función
Instalar paquete	`install.packages()`
Cargar paquete	`library()`
Primeira parte dun obxecto	`head()`
Nomes de obxecto	`names()`

1.7. EXERCICIOS

Boletín de exercicios

Descargar (PDF)

Arquivos de datos

personal_computers_per_100_people.csv
spotify.csv
redes.csv
vjuegos.csv

2Fundamentos de probabilidade

Vimos que un experimento aleatorio a aquel que cumpre os seguintes requisitos:

- Todos os seus posibles resultados son coñecidos de antemán.
- O resultado particular de cada realización do experimento é imprevisible.
- O experimento pódese repetir indefinidamente en condicións idénticas.

Por exemplo, un experimento aleatorio básico sería lanzar un dado. En R, podemos simular experimentos aleatorios de forma moi sinxela usando as funcións de xeración de números aleatorios.

2.1. A función `sample()`

Unha das funcións máis empregadas para simular realizacións de experimentos aleatorios é a función `sample()`, que permite xerar mostras aleatorias (con ou se remplazamento) e permutacións a partir de conxunto de valores.

Unha permutación é unha ordenación aleatoria dos elementos dun vector. En `sample()` isto é o que se devolve por defecto, cando non se especifica ningún outro argumento da función.

```
sample(1:6) # permutación dos números 1 a 6
```

Nós imos empregar a función `sample()` para simular un lanzamento de dado. O que teríamos que facer é:

```
sample(1:6, size = 1)
```

O segundo argumento, `size`, é o número de elementos a xerar. Por defecto, a mostra xérase sen reposición. Se queremos que o proceso se realice con reposición, necesitaremos empregar o terceiro argumento `replace = TRUE`.

```
sample(1:6, size = 4) # 4 números entre {1,2,...,6} (sen reposición)
sample(1:6, size = 4, replace = TRUE) # 4 números entre {1,2,...,6} (con reposición)
```

Observa que sempre que executas a primeira orde, os catro número elixidos son diferentes. Coa segunda opción, poden aparecer valores repetidos.

Fíxate no seguinte exemplo, no que simulamos 10, 100, 1000 e 10000 lanzamentos dun dado e calculamos a frecuencia de aparición de cada resultado.

```r
x <- sample(1:6, size = 10, replace = TRUE)
table(x) / length(x)

x <- sample(1:6, size = 100, replace = TRUE)
table(x) / length(x)

x <- sample(1:6, size = 1000, replace = TRUE)
table(x) / length(x)

x <- sample(1:6, size = 10000, replace = TRUE)
table(x) / length(x)
```

Aínda que non estamos a lanzar dados reais, os resultados son equivalentes e permiten explorar conceptos como o de probabilidade a través da súa definición frecuentista, observando como se comportan as frecuencias relativas cando aumentamos o número de lanzamentos.

A función `sample()` tamén permite xenerar números aleatorios dun conxunto onde non todos os valores son equiprobables usando o argumento `prob`. Por exemplo, supoñamos que queremos simular 10 lanzamentos dunha moeda na que a probabilidade de cara é 0.7. Poderiamos facer:

```r
sample(c("cara", "cruz"), size = 10, replace = TRUE, prob = c(0.7, 0.3))
```

O problema do cabaleiro de Meré

No século XVII, os xogos de azar eran a principal diversión da alta sociedade francesa. Antoine Gombard, Cabaleiro de Meré, preguntoulle a Blaise Pascal (1623–1662) durante unha viaxe se era vantaxoso apostar polo resultado de obter polo menos un seis nunha serie de catro lanzamentos dun dado. Pascal e Pierre de Fermat (1601–1665) mantiveron abundante correspondencia sobre este e outros problemas relacionados co cálculo de probabilidades. No seguinte tema veremos como resolver o problema de forma exacta empregando a distribución binomial.
Se non fósemos capaces de obter a solución exacta, poderiamos abordar o problema mediante simulación. A idea sería repetir moitas veces a situación descrita, é dicir, simular un gran número de series de catro tiradas dun dado. A continuación, calcularíamos a proporción de veces nas que aparece polo menos un seis. Esa frecuencia relativa proporcionaría unha aproximación da probabilidade buscada, que se iría achegando á solución analítica exacta canto maior fose o número de simulacións realizadas.
Para facer isto en R, precisaremos empregar estruturas de control como **bucles**, que nos permiten repetir un mesmo experimento moitas veces, e tamén **sentenzas condicionais**, que serven para comprobar se nunha simulación se cumpre ou non o evento de interese (por exemplo, se apareceu algún seis nas catro tiradas).
En R, un bucle `for` permite repetir un mesmo bloque de instrucións un número determinado

de veces. A súa sintaxe é:

```
for (i in obxecto) {
  comandos
}
```

Isto significa que a variable `i` toma sucesivamente os valores de `obxecto`, e en cada iteración execútanse os `comandos` que están dentro das chaves. Existen tamén bucles `while`

```
while (condición) {
  comandos
}
```

que executan as instrucións dadas en `comandos` mentres a `condición` establecida entre parénteses sexa verdadeira (`TRUE`). Pola súa parte, unha sentenza condicional `if` permite executar un código só cando se cumpre unha condición:

```
if (condición) {
  comandosA
} else {
  comandosB
}
```

Se `condición` é verdadeira (`TRUE`), execútanse os `comandosA`. Se é falsa (`FALSE`), execútanse os `comandosB`.
No seguinte código simulamos `B = 1000` series de catro tiradas dun dado. En cada simulación contamos cantos seises aparecen. Se aparece polo menos un, a condición do `if` cúmprese e sumamos 1 ao contador de victorias `win`. Finalmente, calculamos a frecuencia relativa de éxito como aproximación da probabilidade buscada.

```
win <- 0 # Número de veces que gaña Mere
B <- 1000 # Número de apostas
for (i in 1:B) {
  dice <- sample(1:6, 4, replace = TRUE) # catro lanzamentos dun dado
  n6 <- sum(dice == 6) # Número de 6's nos catro lanzamentos dun dado
  if (n6 >= 1) {
    win <- win + 1
  }
}
fwin <- win / B # Frecuencia de polo menos un seis (probabilidade aproximada)
fwin
```

2.2. Reproducibilidade

En moitas ocasións necesitamos que os resultados sexan reproducibles: que outra persoa (ou ti mesmo noutro momento) poida obter exactamente os mesmos resultados ao executar o mesmo código.

En R, moitas funcións aleatorias (como `sample()`) xeran números baseándose nun xerador de números pseudoaleatorios. Isto significa que os números non son verdadeiramente aleatorios, senón que se obtén unha secuencia determinista a partir dun estado inicial (semente). A función `set.seed()` establece ese estado inicial. Ao fixar a semente, garantimos que a secuencia de números aleatorios será sempre a mesma. O recomendable é establecer un número enteiro, como no seguinte exemplo.

```
set.seed(123)
sample(1:6)
sample(1:6)

set.seed(123)
sample(1:6)
sample(1:6)
```

2.3. EXERCICIOS

Boletín de exercicios

Descargar (PDF)

3Variables aleatorias

Neste laboratorio imos explorar de forma práctica as ideas fundamentais vistas no tema de variables aleatorias. Estudaremos como representar e visualizar con R as distribucións máis importantes. Veremos como simular con R datos destas distribucións e analizaremos as mostras obtidas mediante representacións gráficas e comparacións coas distribucións teóricas. Tamén verificaremos o TCL de forma experimental.

3.1. Distribucións en R

R dispón de catro funcións básicas para traballar con distribucións de probabilidade. Cada unha destas funcións fórmase cun prefixo, que indica a acción que realiza a función, e un sufixo, que indica o modelo de distribución co que se está a traballar.

Prefixo en R	Descrición
`d`	función de masa (distribucións discretas) ou función de densidade (distribucións continuas)
`p`	probabilidade acumulada ou función de distribución
`q`	cuantís da distribución
`r`	xeración de números aleatorios da distribución

A continuación amósanse algúns dos sufixos que se empregan en R para modelos de variables aleatorias discretas:

Distribución	Sufixo en R
Binomial	`binom`
Xeométrica	`geom`
Poisson	`pois`
Binomial negativa	`nbinom`
Hiperxeométrica	`hyper`

Na seguinte táboa figuran os sufixos que se empregan en R para os principais modelos de variables aleatorias continuas.

Distribución	Sufixo en R
Uniforme continua	`unif`
Normal	`norm`

Distribución	Sufixo en R
Exponencial	`exp`
Gamma	`gamma`
Beta	`beta`

Por exemplo, a función `rbinom()` utilízase para xerar números aleatorios dunha variable binomial, mentres que a función `dpois()` serve para calcular a función de masa dunha variable de Poisson e a función `dnorm()` para avaliar a función de densidade dunha variable normal. Estas funcións son moi útiles para comprender mellor como se comportan as variables aleatorias, xa que permiten visualizar as súas funcións de masa ou densidade, estudar a súa función de distribución e analizar o seu comportamento mediante a xeración de datos simulados.

3.2. Principais modelos de variables aleatorias discretas

3.2.1. Distribucións de Bernoulli e Binomial

Unha variable aleatoria binomial X conta o **número de éxitos en n experimentos de Bernoulli independentes** con probabilidade de éxito en cada intento p. Lembra que unha variable de Bernoulli só toma dous valores: 1 (éxito) e 0 (fracaso) e a distribución caracterízase pola probabilidade de éxito p.

Se $X \sim \text{Binom}(n, p)$, entón:

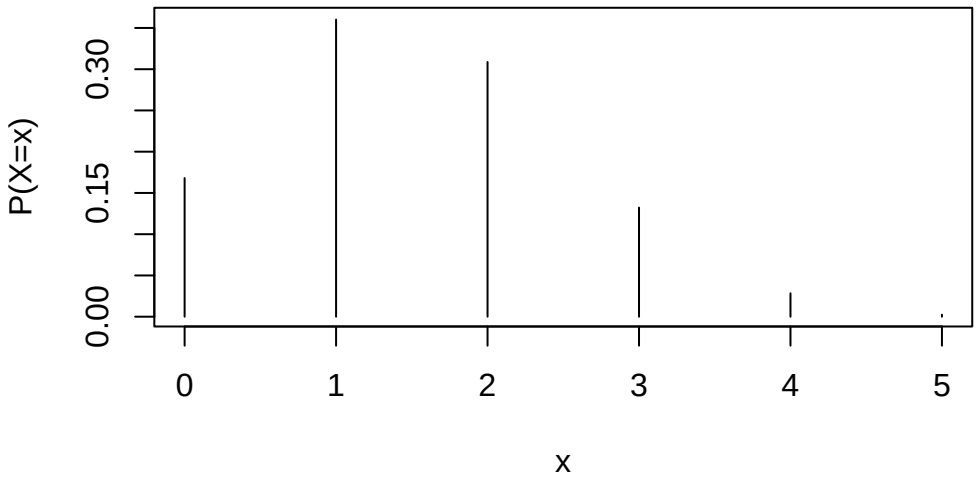
$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, \dots, n$$

Podemos calcular a función de masa dunha binomial en R coa función `dbinom()`. Por exemplo, se $X \sim \text{Binom}(n = 5, p = 03)$ e queremos calcular $P(X = 2)$, escribimos:

```
dbinom(2, size = 5, prob = 0.3)
```

Representamos a continuación a función de masa dunha variable $X \sim \text{Binom}(5, 03)$.

```
n <- 5 # Número de intentos
p <- 0.3 # Probabilidade de éxito de cada intento
x <- 0:n # Valores que toma a variable
px <- dbinom(x, n, p) # Función de masa dunha Binom(n,p) avaliada en x
plot(x, px, type = "h", xlab = "x", ylab = "P(X=x)") # Representación da función de masa
```



O problema do cabaleiro de Meré

No século XVII, os xogos de azar eran a principal diversión da alta sociedade francesa. Antoine Gombard, Cabaleiro de Meré, preguntoulle a Blaise Pascal (1623–1662) durante unha viaxe se era vantaxoso apostar polo resultado de obter polo menos un seis nunha serie de catro lanzamentos dun dado. No tema anterior obtivemos unha solución aproximada a este problema mediante simulación, pero pódese resolver de forma exacta empregando a distribución binomial como se amosa a continuación. (*Podes comparar o resultado coa solución aproximada obtida no tema anterior*).

```
1 - dbinom(0, size = 4, prob = 1 / 6) # Probabilidade de obter polo menos un seis en 4 dado
```

Lembra que a función de distribución $F(x)$ dunha variable aleatoria X vén dada por

$$F(x) = P(X \leq x)$$

A función de distribución $F(x)$ describe a probabilidade de obter un valor igual ou inferior a x. Se queremos avaliar a función de distribución dunha variable binomial en R utilizaremos a función `pbinom()`. Por exemplo, sexa $X \sim \text{Bin}(5, 03)$. Para calcular $F(2) = P(X \leq 2)$, escribimos:

```
pbinom(2, size = 5, prob = 0.3)
```

Observa que é equivalente a

```
sum(dbinom(0:2, size = 5, prob = 0.3))
```

A función cuantil é a inversa da función de distribución. Representa o primeiro valor da variable que alcanza a probabilidade pedida. A función para calcular os cuantís dunha binomial en R é `qbinom()`. Por exemplo, a mediana de $X \sim \text{Binom}(5, 03)$ é:

```r
qbinom(0.5, size = 5, prob = 0.3)
```

Os cuartís de $X \sim \text{Binom}(5, 03)$, calcúlanse como:

```r
qbinom(c(0.25, 0.5, 0.75), size = 5, prob = 0.3)
```

A función para xerar en R números aleatorios dunha binomial é `rbinom()`. O primeiro argumento da función é o número de elementos que se van xerar. Por exemplo, se queremos xerar un único número dunha variable $X \sim \text{Binom}(5, 03)$ escribimos:

```r
rbinom(1, size = 5, prob = 0.3)
```

O problema do cabaleiro de Meré (outra forma de simular resultados do experimento)

Agora que sabemos que o número de seises obtidos en catro lanzamentos dun dado segue unha distribución binomial, podemos simular os resultados de moitas apostas de forma máis sinxela empregando a función `rbinom()`, sen necesidade de simular cada lanzamento de dado individualmente. Así, no seguinte código `n6` contén o número de seises obtidos en 4 lanzamentos de dados en cada unha das `B=100000` apostas. Finalmente, calculamos a frecuencia coa que se obtivo polo menos un seis, que proporciona unha aproximación á probabilidade exacta calculada anteriormente.

```r
B <- 100000
n6 <- rbinom(B, 4, 1 / 6)
sum(n6 >= 1) / B
```

3.2.2. Distribucións xeométrica e binomial negativa

Consideramos agora o experimento que consiste en repetir un experimento de Bernoulli ata obter o primeiro éxito, e definimos a variable aleatoria X como o **número de fracasos antes de obter o primeiro éxito**. Así definida, X segue unha distribución xeométrica de parámetro p, sendo p probabilidade de éxito en cada intento. En R, `dgeom()` calcula a función de masa, `pgeom()` a función de distribución, `qgeom()` os cuantís, e `rgeom()` xera valores aleatorios dunha distribución xeométrica.

Por exemplo, supoñamos que estamos xogando ao parchís e necesitamos sacar un cinco para poder saír da casa. Lanzamos repetidamente un dado. Cal é a probabilidade de obter exactamente 10 resultados distintos de cinco antes de conseguir o noso primeiro cinco? Observa que se consideramos a variable aleatoria X= número de resultados distintos de 5 antes do primeiro cinco, entón $X \sim \text{Xe}(p = 1/6)$. Queremos calcular $P(X = 10)$, é dicir:

```r
dgeom(10, prob = 1 / 6)
```

Sexa agora X a variable aleatoria que representa o **número de fracasos antes de acadar o k-ésimo éxito**. Así definida, X segue unha distribución binomial negativa e denótase como $X \sim \text{BN}(k, p)$ (a distribución xeométrica é un caso particular con $k = 1$). En R, `dnbinom()` calcula a función de masa, `pnbinom()` a función de distribución, `qnbinom()` os cuantís, e `rnbinom()` xera valores aleatorios dunha distribución binomial negativa.

Por exemplo, supoñamos que no experimento que consiste en repetir o lanzamento dun dado, queremos calcular a probabilidade de obter exactamente 10 resultados distintos de cinco antes de conseguir o noso terceiro cinco. Observa que se consideramos a variable aleatoria X= número de resultados distintos de 5 antes do terceiro cinco, entón $X \sim \text{BN}(k = 3, p = 1/6)$. Queremos calcular $P(X = 10)$, é dicir:

```r
dnbinom(10, size = 3, prob = 1 / 6)
```

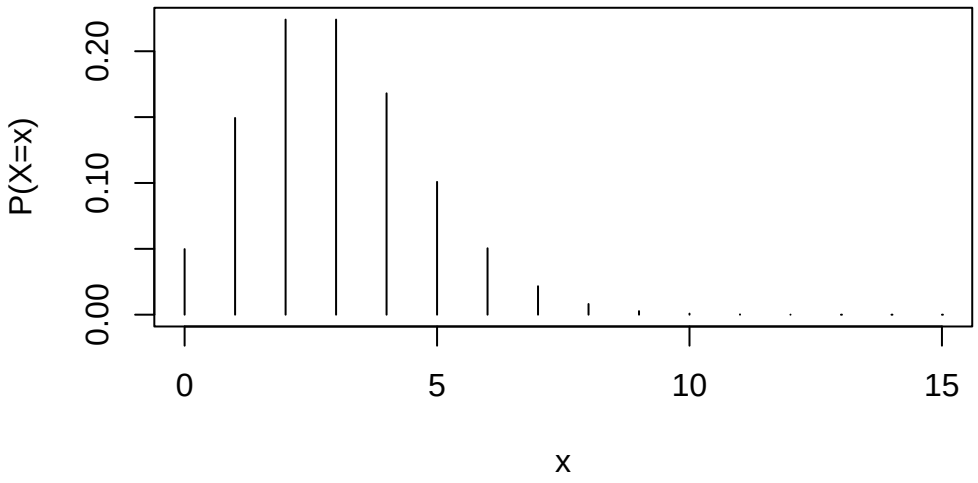
3.2.3. Distribución de Poisson

Un proceso de Poisson xeneraliza en certa medida o proceso de Bernoulli. Consiste en observar o número de veces que se presenta un suceso (número de éxitos) nun determinado intervalo (xeralmente de tempo). Nestes procesos supónse que hai estabilidade, no sentido de que o número medio de sucesos por unidade de tempo (λ) permanece constante. Sexa $\lambda > 0$ o número medio de sucesos no intervalo e X o **número de éxitos por intervalo**. Así definida X ten distribución de Poisson de parámetro λ e denotamos $X \sim \text{Pois}(\lambda)$.

En R, `dpois()` calcula a función de masa, `ppois()` a función de distribución, `qpois()` os cuantís, e `rpois()` xera valores aleatorios dunha distribución de Poisson.

Por exemplo, representamos a continuación a función de masa dunha variable $X \sim \text{Pois}(\lambda = 3)$.

```r
lambda <- 3
x <- 0:15
px <- dpois(x, lambda)
plot(x, px, type = "h", xlab = "x", ylab = "P(X=x)")
```



3.2.4. Distribución hiperxeométrica

Consideremos unha poboación finita de N elementos, dos cales k pertencen á unha certa clase D e $N-k$ ao resto. Se agora tomamos unha mostra **sen reposición** de tamaño n e estudamos a variable aleatoria X que representa o **número de elementos da clase D na mostra de tamaño** n, esta segue unha distribución hiperxeométrica $X \sim \mathrm{H}(N, n, k)$.

En R, `dhyper()` calcula a función de masa, `phyper()` a función de distribución, `qhyper()` os cuantís, e `rhyper()` xera valores aleatorios dunha distribución hiperxeométrica. Para especificar a distribución hiperxeométrica coa que queremos traballar, debemos especificar tres argumentos (`m`, `n`, `k`). Temos que ter coidado con estes argumentos, pois non coinciden exactamente cos valores N, n e k. Así:

Argumento	Interpretación
`m`	número de elementos da clase D na poboación (k)
`n`	número de elementos da outra clase ($N-k$)
`k`	tamaño da mostra (n)

Supoñamos o seguinte exemplo: 15 cidadáns estadounidenses e 20 non cidadáns estadounidenses pasan por unha liña de seguridade nun aeroporto. Selecciónanse aleatoriamente dez persoas para un control adicional. ¿Cal é a probabilidade de que dous ou menos dos pasaxeiros seleccionados sexan cidadáns estadounidenses? Sexa X o número de cidadáns estadounidenses na mostra de tamaño $n = 10$. Así definida $X \sim \mathrm{H}(N = 35, n = 10, k = 15)$. Queremos calcular $P(X \leq 2)$. En R teríamos que executar:

```
phyper(2, m = 15, n = 20, k = 10)
```

3.3. Principais modelos de variables aleatorias continuas

3.3.1. Distribución uniforme

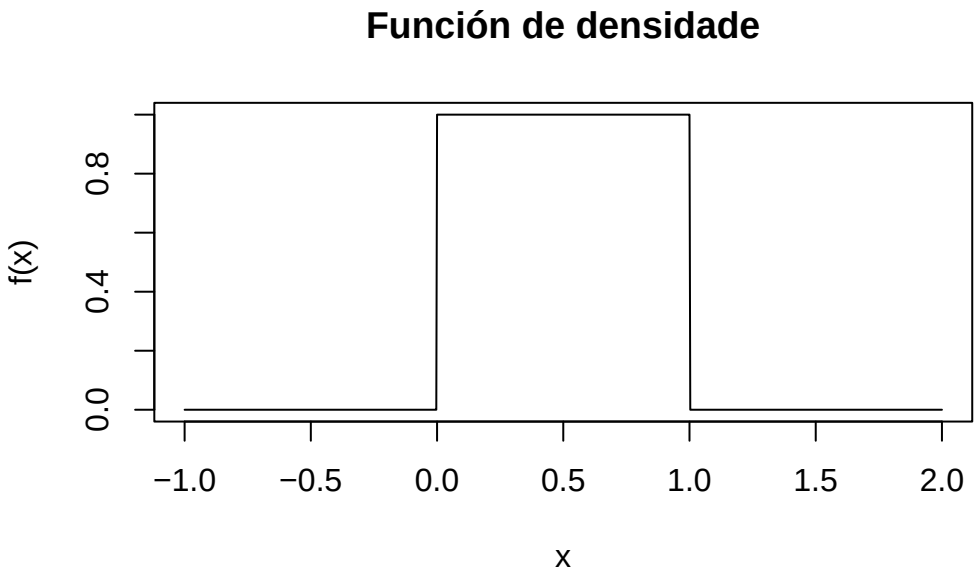
Unha variable aleatoria X segue unha distribución uniforme nun intervalo $[a, b]$ se a función de densidade é constante no intervalo, é dicir,

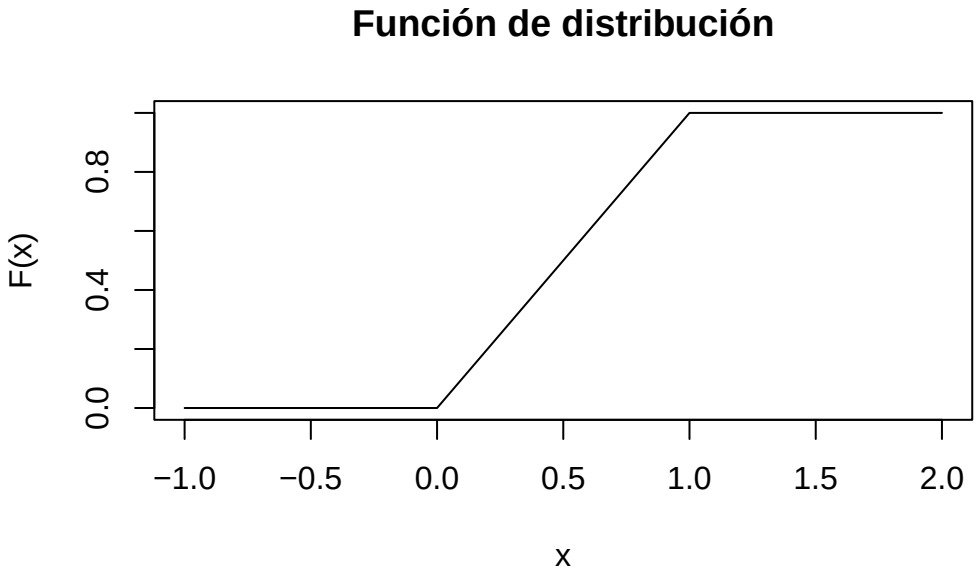
$$f(x) = \begin{cases} \frac{1}{b-a}, & \text{se } x \in (a, b), \\ 0, & \text{en caso contrario} \end{cases}$$

En R, `dunif()` avalía a función de densidade, `punif()` a función de distribución, `qunif()` os cuantís, e `runif()` xera valores aleatorios dunha distribución uniforme continua.

Representamos a función de densidade e a función de distribución dunha variable $X \sim \text{Unif}([0, 1])$.

```
a <- 0
b <- 1
x <- seq(a - 1, b + 1, length = 1000) # Valores onde avaliar f
fx <- dunif(x, min = a, max = b) # Función de densidade
Fx <- punif(x, min = a, max = b) # Función de distribución
plot(x, fx, xlab = "x", ylab = "f(x)", type = "l", main = "Función de densidade")
plot(x, Fx, xlab = "x", ylab = "F(x)", type = "l", main = "Función de distribución")
```





Calculamos como exemplo os cuartís dunha variable $X \sim \text{Unif}([1, 2])$.

```
qunif(c(0.25, 0.5, 0.75), min = 1, max = 2)
```

A continuación xeramos 1000 números aleatorios seguindo unha distribución uniforme continua no intervalos $[0, 1]$ e calculamos a súa media.

```
xu <- runif(1000, min = 0, max = 1)
mean(xu)
```

3.3.2. Distribución normal

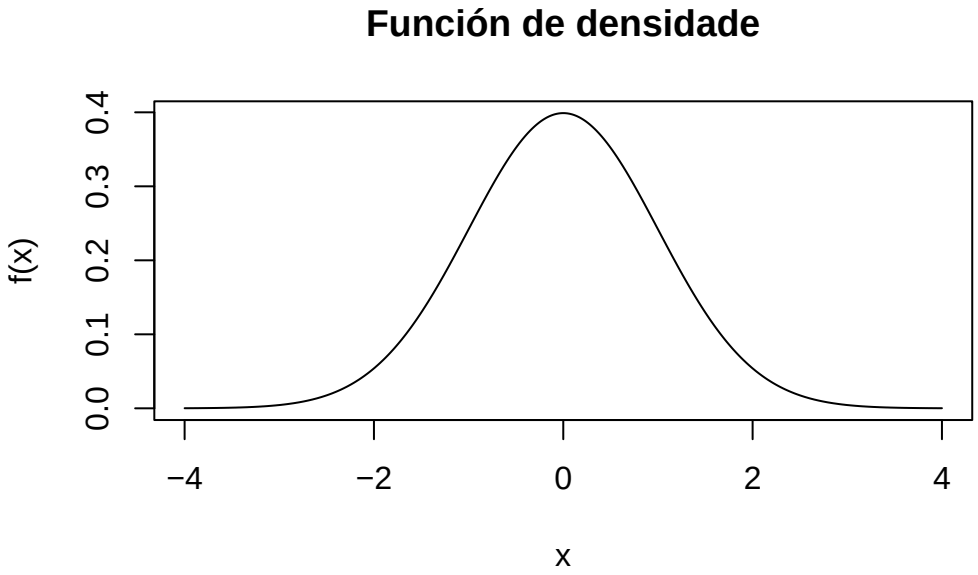
Sexa X unha variable aleatoria con distribución normal de media μ e varianza σ^2. Denotámola $X \sim \text{N}(\mu, \sigma)$. Lembramos que a súa función de densidade vén dada por:

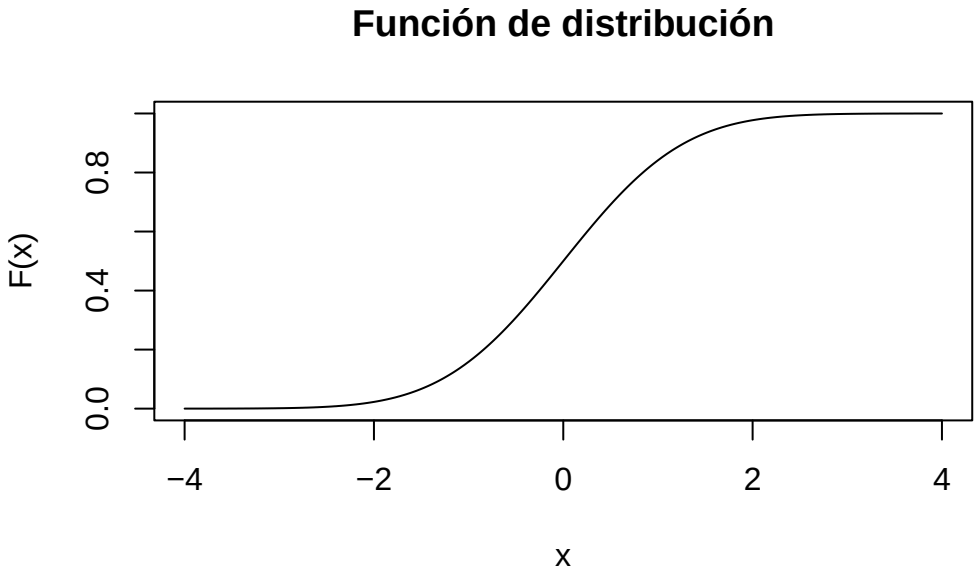
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \quad -\infty < x < \infty$$

En R, `dnorm()` avalía a función de densidade, `pnorm()` a función de distribución, `qnorm()` os cuantís, e `rnorm()` xera valores aleatorios dunha distribución normal.

Representamos a función de densidade e a función de distribución dunha variable $X \sim \text{N}(0, 1)$ (normal estándar).

```r
x <- seq(-4, 4, length = 1000) # Valores onde avaliar f
fx <- dnorm(x, mean = 0, sd = 1) # Función de densidade
Fx <- pnorm(x, mean = 0, sd = 1) # Función de distribución
plot(x, fx, xlab = "x", ylab = "f(x)", type = "l", main = "Función de densidade")
plot(x, Fx, xlab = "x", ylab = "F(x)", type = "l", main = "Función de distribución")
```





Moitas veces resulta útil transformar unha variable normal $X \sim \mathrm{N}(\mu, \sigma)$ nunha variable normal estándar (de media 0 e varianza 1). Isto faise mediante a estandarización:

$$Z = \frac{X - \mu}{\sigma} \sim \mathrm{N}(0, 1)$$

Por exemplo, se $X \sim \mathrm{N}(\mu = 4, \sigma = 2)$, entón $P(X \leq 6) = P\left(\frac{X-4}{2} \leq \frac{6-4}{2}\right) = P(Z \leq 1)$. Comprobámolo con R:

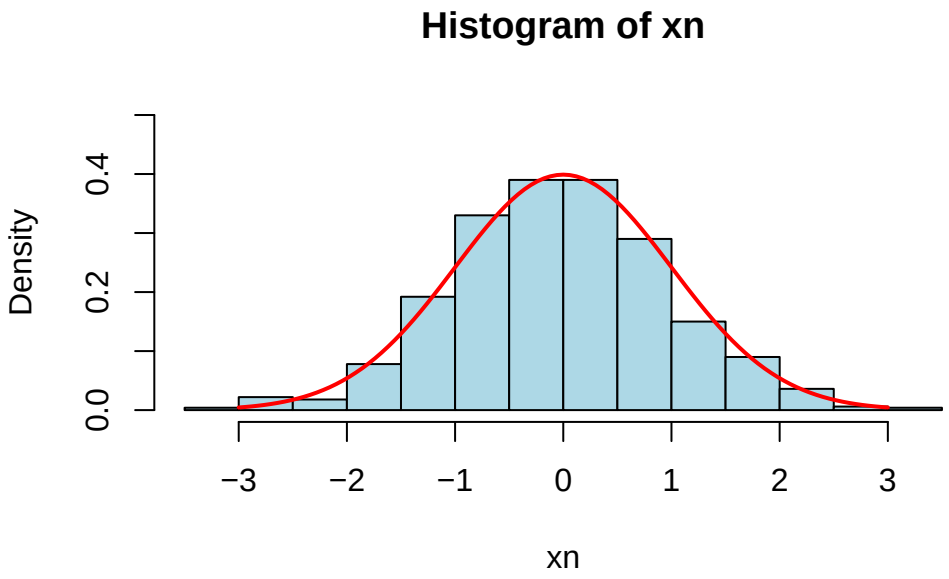
```
pnorm(6, mean = 4, sd = 2) # P(X<6) con X~N(6,2)
pnorm(1) # P(Z<1) con Z~N(0,1)
```

Calculamos a continuación os cuartís dunha variable $X \sim \mathrm{N}(0, 1)$. Observa que non é necesario especificar os argumentos `mean = 0` e `sd = 1`, xa que eses son os seus valores por defecto.

```
qnorm(c(0.25, 0.5, 0.75))
```

Por último, xeramos 1000 números aleatorios seguindo unha distribución normal estándar. Representamos o histograma destes datos e engadimos a curva da función de densidade correspondente (neste caso o facemos coa función `curve()`, que permite debuxar unha función matemática sobre un gráfico). Deste xeito, podemos comprobar visualmente que os datos xerados se distribúen segundo o esperado, é dicir, se parecen á distribución normal teórica.

```
xn <- rnorm(1000, mean = 0, sd = 1)
hist(xn, freq = FALSE, col = "lightblue")
curve(dnorm(x), from = -3, to = 3, add = TRUE, col = "red", lwd = 2)
```



3.3.3. Distribución exponencial

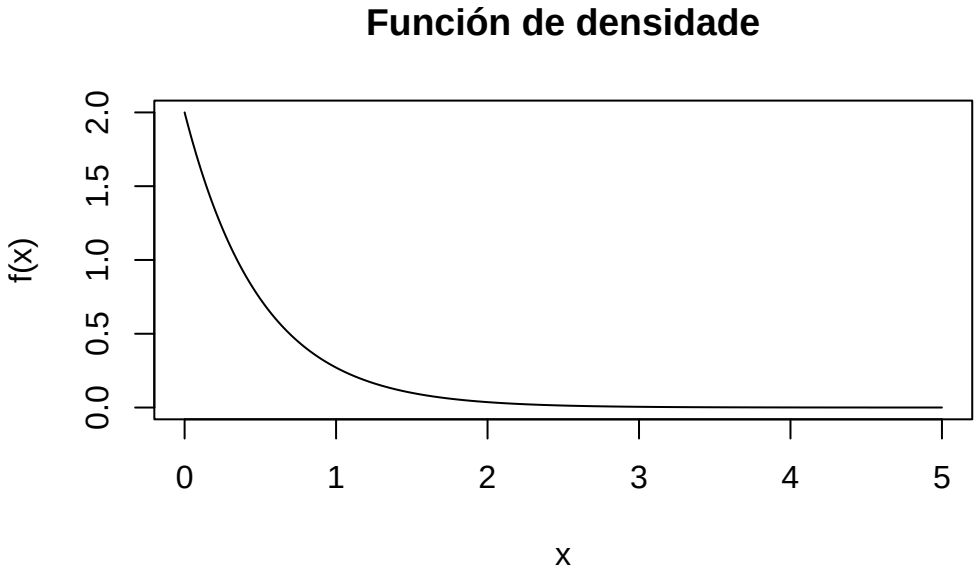
Sexa X unha variable aleatoria con distribución exponencial de parámetro λ. Denotámola $X \sim \text{Exp}(\lambda)$. Lembramos que a súa función de densidade vén dada por:

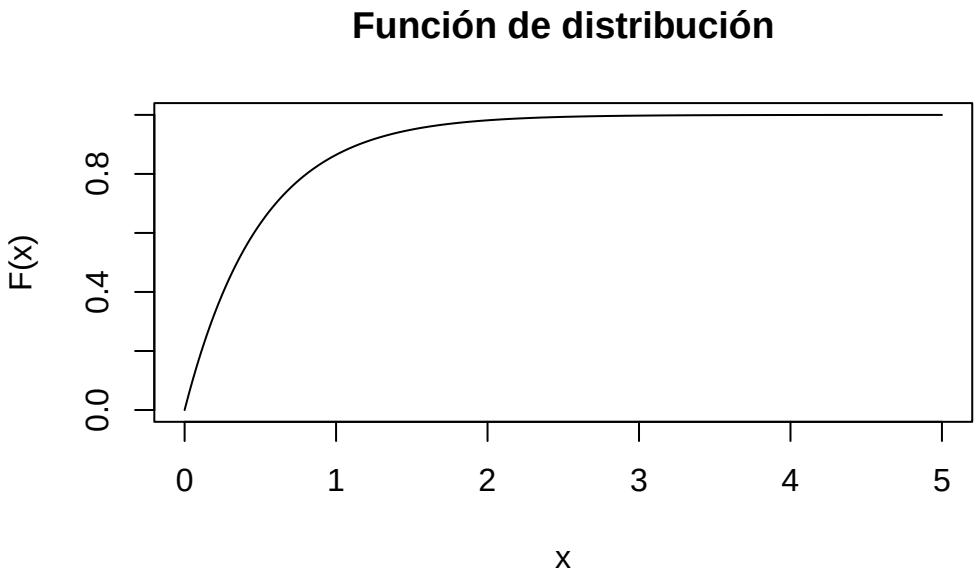
$$f(x) = \begin{cases} \lambda e^{-\lambda x} & \text{se } x > 0, \\ 0 & \text{noutro caso} \end{cases}$$

En R, `dexp()` avalía a función de densidade, `pexp()` a función de distribución, `qexp()` os cuantís, e `rexp()` xera valores aleatorios dunha distribución exponencial.

Como exemplo, representamos a función de densidade e a función de distribución dunha variable $X \sim \text{Exp}(\lambda = 2)$.

```
lambda <- 2
x <- seq(0, 5, length = 1000) # Valores onde avaliar f
fx <- dexp(x, rate = lambda) # Función de densidade
Fx <- pexp(x, rate = lambda) # Función de distribución
plot(x, fx, xlab = "x", ylab = "f(x)", type = "l", main = "Función de densidade")
plot(x, Fx, xlab = "x", ylab = "F(x)", type = "l", main = "Función de distribución")
```





3.3.4. Distribución gamma

Unha variable aleatoria X que mide o tempo de espera ata o suceso número k nun proceso de Poisson segue unha distribución gamma de parámetros λ e k. Denótase por $X \sim \text{Gamma}(\lambda, k)$, onde o primeiro parámetro representa o número medio de sucesos por unidade de tempo e k é o número de sucesos que queremos que ocorran (é unha xeneralización da distribución exponencial). A función de densidade é

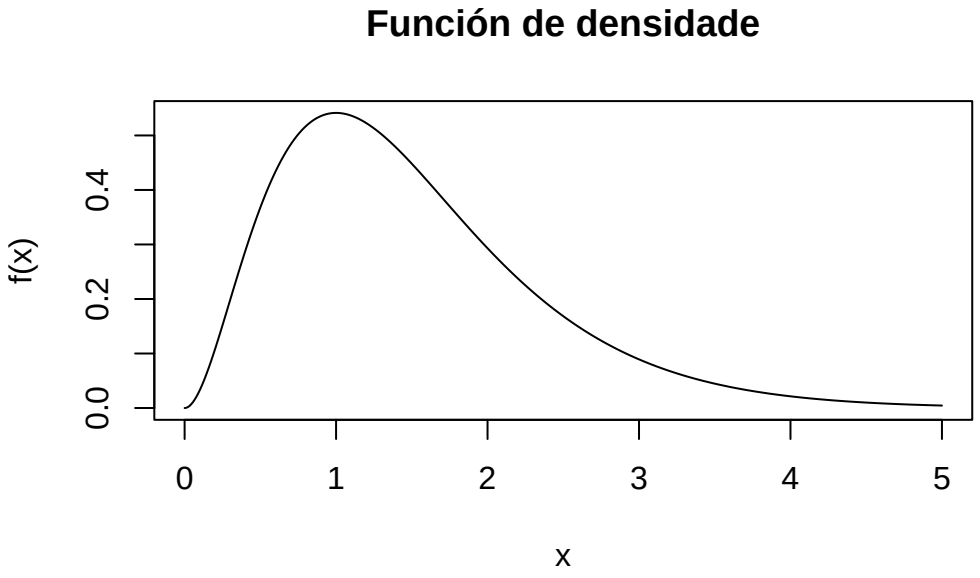
$$f(x) = \begin{cases} \frac{\lambda^k}{\Gamma(k)} e^{-\lambda x} x^{k-1} & \text{se } x > 0, \\ 0 & \text{noutro caso} \end{cases}$$

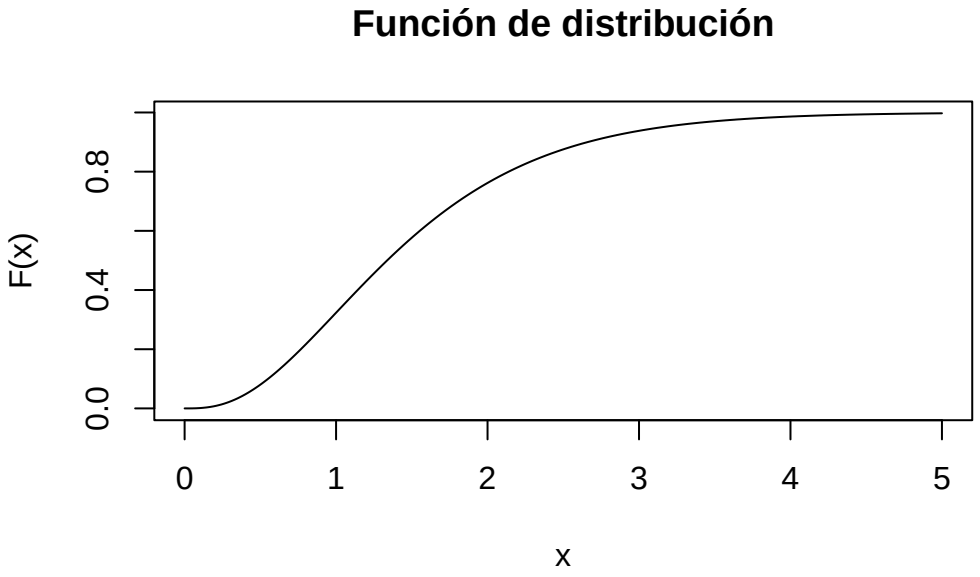
onde para valores de k naturais distintos de cero, $\Gamma(k) = (k-1)! = (k-1)(k-2)\cdots 2 \cdot 1$.

En R, `dgamma()` avalía a función de densidade, `pgamma()` a función de distribución, `qgamma()` os cuantís, e `rgamma()` xera valores aleatorios dunha distribución gamma.

Como exemplo, representamos a función de densidade e a función de distribución dunha variable $X \sim \Gamma(\lambda = 2, k = 3)$.

```
k <- 3
lambda <- 2
x <- seq(0, 5, length = 1000) # Valores onde avaliar f
fx <- dgamma(x, shape = k, rate = lambda) # Función de densidade
Fx <- pgamma(x, shape = k, rate = lambda) # Función de densidade
plot(x, fx, xlab = "x", ylab = "f(x)", type = "l", main = "Función de densidade")
plot(x, Fx, xlab = "x", ylab = "F(x)", type = "l", main = "Función de distribución")
```





3.3.5. Distribución beta

Diremos que unha variable aleatoria X segue unha distribución beta de parámetros p e q, e denotamos $X \sim \text{Beta}(p, q)$ con p e q positivos se a súa función de densidade vén dada como se indica a continuación.

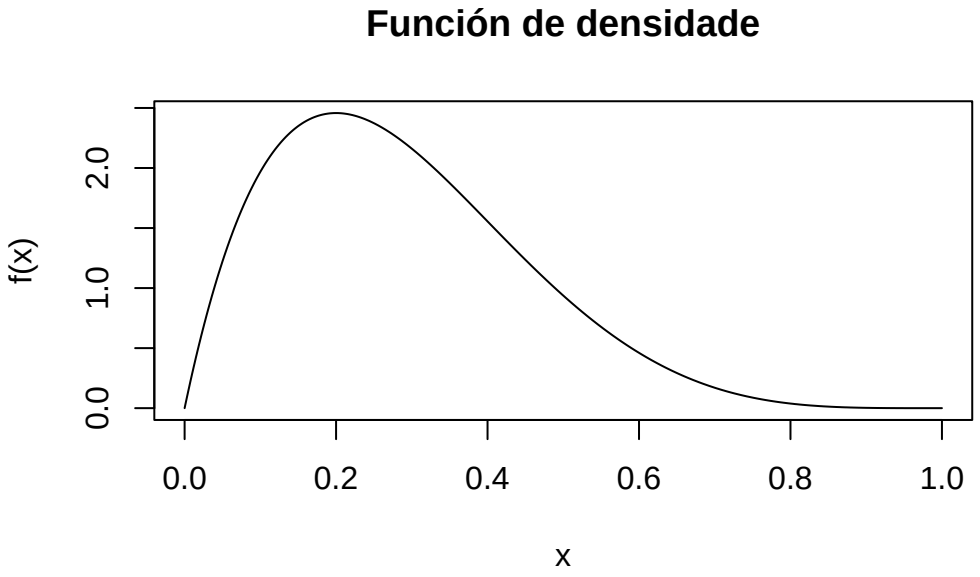
$$f(x) = \begin{cases} \frac{x^{p-1}(1-x)^{q-1}}{\beta(p,q)} & \text{se } 0 < x < 1, \\ 0 & \text{noutro caso} \end{cases}$$

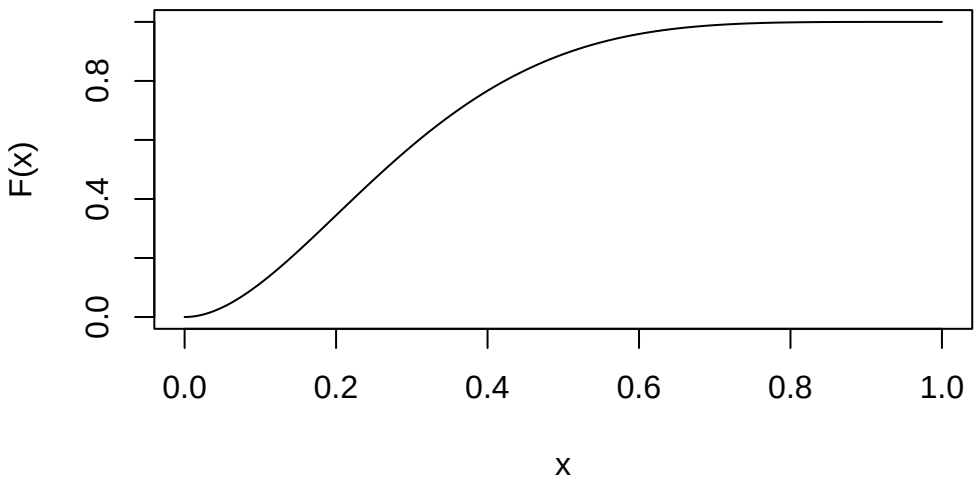
onde $\beta(p,q) = \frac{\Gamma(p)\cdot\Gamma(q)}{\Gamma(p+q)}$

En R, `dbeta()` avalía a función de densidade, `pbeta()` a función de distribución, `qbeta()` os cuantís, e `rbeta()` xera valores aleatorios dunha distribución beta.

Como exemplo, representamos a función de densidade e a función de distribución dunha variable $X \sim \text{Beta}(p = 2, q = 5)$.

```
p <- 2
q <- 5
x <- seq(0, 1, length = 1000) # Valores onde avaliar f
fx <- dbeta(x, shape1 = p, shape2 = q) # Función de densidade
Fx <- pbeta(x, shape1 = p, shape2 = q) # Función de densidade
plot(x, fx, xlab = "x", ylab = "f(x)", type = "l", main = "Función de densidade")
plot(x, Fx, xlab = "x", ylab = "F(x)", type = "l", main = "Función de distribución")
```





3.4. Teorema Central do Límite

O Teorema Central do Límite (TCL) establece que, se temos unha sucesión de variables aleatorias independentes e idénticamente distribuídas (i.i.d.), cunha media finita μ e unha varianza finita σ^2, entón a distribución da súa suma (ou, de maneira equivalente, da súa media) tende a unha distribución normal cando o número de variables é suficientemente grande.

Formalmente, se $X_1, X_2, \dots, X_n$ son variables aleatorias i.i.d. con $\mathbb{E}(X_i) = \mu$ e $\text{Var}(X_i) = \sigma^2$, entón

$$\frac{\sum_{i=1}^{n} X_i - n\mu}{\sigma\sqrt{n}} \underset{n\to\infty}{\longrightarrow} N(0,1)$$

De forma equivalente, a media muestral $\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i$ verifica

$$\frac{\bar{X}_n - \mu}{\sigma/\sqrt{n}} \underset{n\to\infty}{\longrightarrow} N(0,1)$$

Podemos observar e xustificar de maneira intuitiva este resultado mediante simulación. En esencia, o TCL afirma que, se tomamos moitas mostras de tamaño n dunha variable aleatoria con media μ e varianza σ^2, entón a distribución das sumas (ou das medias) destas mostras tende a aproximarse a unha distribución normal cando n é suficientemente grande. Na primeira formulación, isto significa que, se para cada mostra calculamos a suma dos seus valores e representamos un histograma con todas esas sumas, a forma do histograma debería ser aproximadamente normal. De maneira equivalente, na segunda formulación, podemos calcular a media de cada unha das mostras xeradas e representar a distribución desas medias mediante un histograma. De novo, observaremos que esa distribución se asemella cada vez máis a unha

normal a medida que aumenta n. Finalmente, se estandarizamos estas sumas ou medias como se indica en cada caso, os histogramas obtidos deberían aproximarse á distribución normal estándar $\mathrm{N}(0,1)$.

Para ilustrar o TCL, imos partir dunha distribución que non é normal, por exemplo unha distribución exponencial. Recorda que se $X \sim \mathrm{Exp}(\lambda)$, entón a súa media é $\mu = 1/\lambda$ e a súa varianza é $\sigma^2 = 1/\lambda^2$.

```r
set.seed(123)

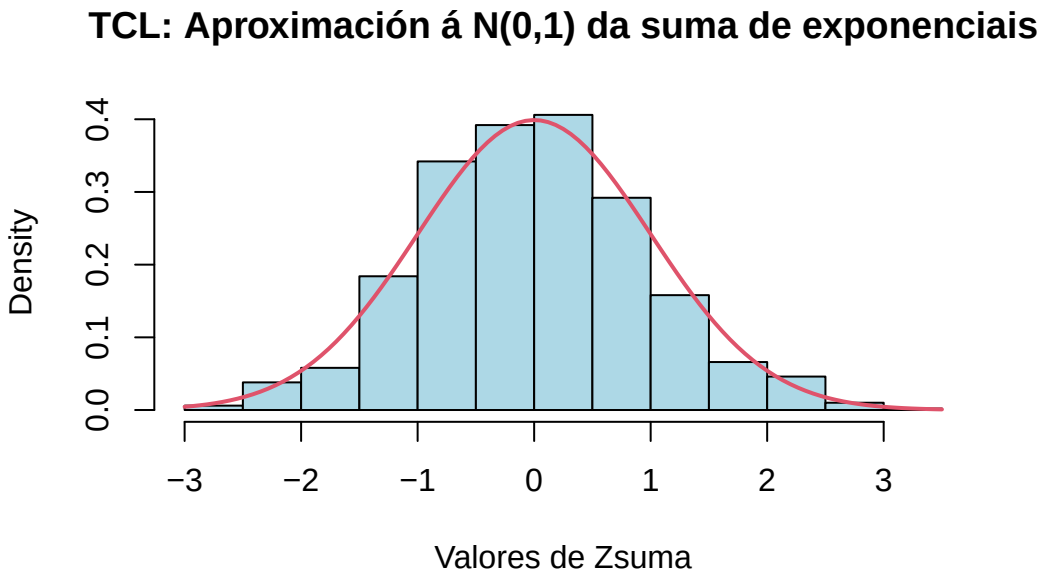
B <- 1000 # número de mostras
n <- 500 # tamaño de cada mostra

# Distribución orixinal: Exponencial(lambda)
lambda <- 1
mu <- 1 / lambda # media teórica
sigma <- 1 / lambda # desviación típica teórica

# Simulamos B mostras de tamaño n e, para cada mostra, calculamos suma e media
medias <- sumas <- numeric()
for (i in 1:B) {
  mostra <- rexp(n)
  sumas[i] <- sum(mostra)
  medias[i] <- mean(mostra)
}
# Estandarizamos as sumas
Zsuma <- (sumas - n * mu) / (sigma * sqrt(n))
# Estandarizamos as medias
Zmedia <- (medias - mu) / (sigma / sqrt(n))

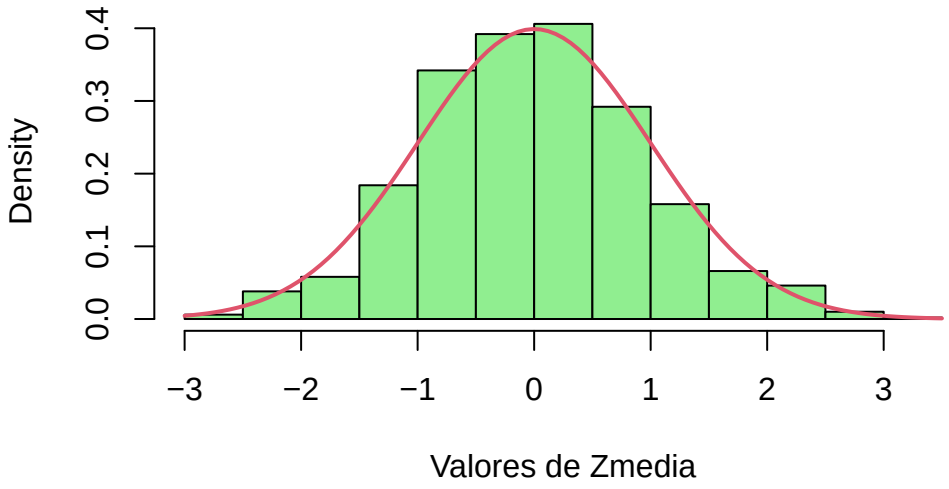
# Histograma das sumas estandarizadas
hist(Zsuma,
  freq = FALSE,
  col = "lightblue",
  main = "TCL: Aproximación á N(0,1) da suma de exponenciais",
  xlab = "Valores de Zsuma"
)

# Engadimos a densidade dunha N(0,1)
curve(dnorm(x), add = TRUE, lwd = 2, col = 2)
```




```
# Histograma das medias estandarizadas
hist(Zmedia,
  freq = FALSE,
  col = "lightgreen",
  main = "TCL: Aproximación á N(0,1) da media de exponenciais",
  xlab = "Valores de Zmedia"
)

# Engadimos a densidade dunha N(0,1)
curve(dnorm(x), add = TRUE, lwd = 2, col = 2)
```



O Teorema TCL xustifica, entre outras cousas, por que unha distribución binomial pode aproximarse por unha distribución normal cando n é grande. Para entendelo, basta lembrar que unha variable binomial $X \sim \text{Binom}(n, p)$ conta o número de éxitos en n repeticións independentes dun experimento con probabilidade de éito en cada intento p. Por exemplo, lanzar unha moeda n veces e contar cantas caras saen. Agora ben, cada repetición individual pode representarse cunha variable de Bernoulli:

$$X_i = \begin{cases} 1, & \text{se hai éxito,} \\ 0, & \text{se non hai éxito} \end{cases}$$

A variable binomial non é máis que a suma destes n resultados:

$$X = X_1 + X_2 + \cdots + X_n$$

É dicir, unha binomial é simplemente a suma de variables de Bernoulli independentes X_i (cada unha delas con $\mathbb{E}(X_i) = p$ e $\text{Var}(X_i) = p(1-p)$). E aquí é onde entra o TCL: este teorema di que a suma dun gran número de variables aleatorias independentes tende a seguir unha distribución normal, independentemente da distribución orixinal. Por iso, cando n é suficientemente grande, a distribución binomial $\text{Binom}(n, p)$ pode aproximarse por unha normal de media np e de desviación típica $\sqrt{np(1-p)}$.

Na seguinte gráfica amosamos a función de masa dunha variable aleatoria $X \sim \text{Binom}(n, p)$ para distintos valores de n. En cada caso, superpoñemos tamén a densidade da distribución normal$\text{N}(\mu = np, \sigma = \sqrt{np(1-p)})$, que proporciona unha aproximación cada vez mellor á binomial a medida que n aumenta, tal e como xustifica o TCL.

```r
p <- 0.3 # Parámetro p
n_values <- c(5, 10, 50) # Valores de n que imos comparar

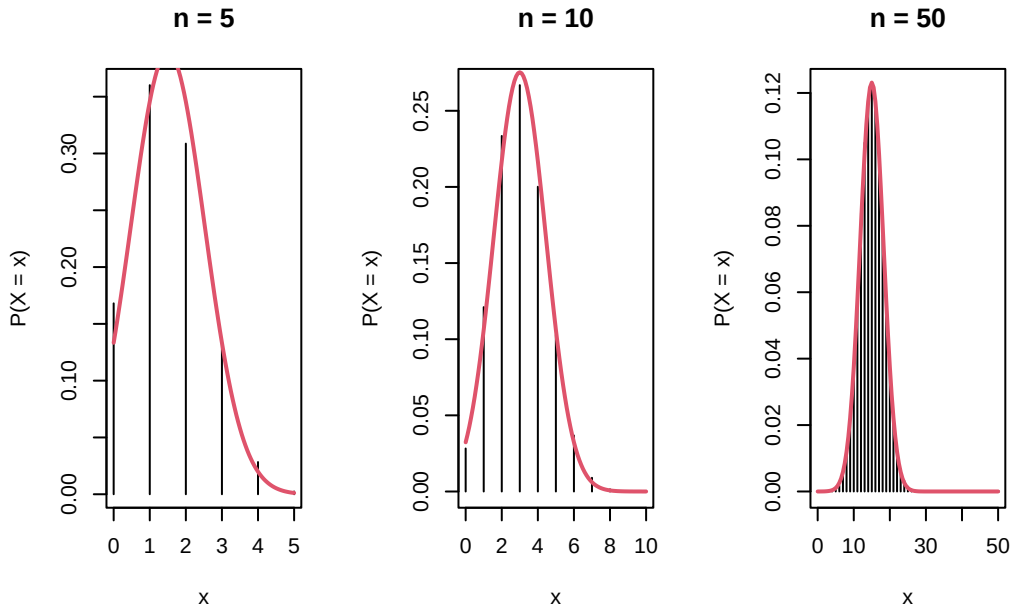
par(mfrow = c(1, 3)) # Debuxar 3 gráficos en fila

for (n in n_values) {
  x <- 0:n # Valores posibles da binomial
  fx <- dbinom(x, size = n, prob = p) # Función de masa binomial

  # Parámetros da aproximación normal
  mu <- n * p
  sigma <- sqrt(n * p * (1 - p))

  # Representar función de masa
  plot(x, fx,
    type = "h",
    main = paste("n =", n),
    xlab = "x", ylab = "P(X = x)"
  )

  # Engadir densidade normal aproximada
  curve(dnorm(x, mean = mu, sd = sigma),
    add = TRUE, lwd = 2, col = 2
  )
}
```



Cómpre lembrar que a distribución binomial é **discreta** e só pode tomar valores enteiros $(0, 1, 2, \ldots, n)$, mentres que a distribución normal é **continua**. Por iso, cando aproximamos unha binomial $X \sim \text{Binom}(n, p)$ mediante unha normal $Y \sim \text{N}(np, \sqrt{np(1-p)})$, adoita empregarse a chamada **corrección por continuidade**:

$$P(X = k) \approx P(k - 05 \leq Y \leq k + 05),$$

Por exemplo:

```
n <- 50
p <- 0.2
k <- 5

mu <- n * p
sigma <- sqrt(n * p * (1 - p))

dbinom(5, n, p) # P(X=k)
pnorm(5.5, mu, sigma) - pnorm(4.5, mu, sigma) # P(k-0.5<Y<k+0.5)=F(5.5)-F(4.5)
```

3.5. EXERCICIOS

Boletín de exercicios

Descargar (PDF)

Arquivos de datos

laliga_matches_2024_2025.csv Este ficheiro foi obtido a partir de Masryo, T. (2025). Football Matches Results 2024–25 [Dataset]. GitHub. https://github.com/tarekmasryo/football-matches-2025-dataset. Licencia CC BY 4.0.

4Introdución á inferencia estatística e á estimación de parámetros

Nesta práctica aplicaremos os **conceptos de inferencia estatística** estudados na teoría utilizando o entorno de programación R. O obxectivo é que poidamos **estimar parámetros** e construír **intervalos de confianza** a partir de mostras, así como interpretar os resultados de forma práctica.

Traballaremos con datos simulados ou recollidos de exemplos reais. Ao longo destas prácticas, aprenderemos a:

- Calcular estadísticos de mostra (media, varianza, proporcións).
- Construír intervalos de confianza para diferentes tipos de parámetros (medias, diferenzas de medias, proporcións, varianzas).
- Interpretar os resultados no contexto da poboación de interese, conectando así a teoría coa práctica.

4.1. Estimación puntual

Sexa $X_1, X_2, \dots, X_n$ unha m.a.s dunha poboación. Sexa θ un parámetro de interese desa poboación. Un **estimador puntual (estimador)** de θ é un estatístico $\widehat{\theta} \equiv T(X_1, X_2, \dots, X_n)$ cuxa función é aproximar o valor de θ. Lembramos que un estatístico é unha variable aleatoria e, por iso, terá unha certa distribución.

4.1.1. Estimación puntual da media e a varianza

Consideremos unha mostra aleatoria simple $X_1, \dots, X_n$ de tamaño n dunha poboación con media μ e desviación típica σ. O problema consiste en estimar os parámetros μ (media poboacional) e σ (desviación típica poboacional).

Estimación puntual da media

Xa comentamos que como estimador natural para a media poboacional, μ empregaremos **media muestral**:

$$\bar{X} = \frac{X_1 + \cdots + X_n}{n}$$

Sabemos que $\mathbb{E}(\bar{X}) = \mu$, polo que é un estimador insesgado da media.

Distribución do estatístico

Se $X_1, \ldots, X_n$ son variables aleatorias independentes e distribuídas segundo $N(\mu, \sigma)$, entón a media muestral é exactamente normal $\bar{X} \sim N\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$ e podemos facer inferencias exactas mesmo con mostras pequenas. Se $X_1, \ldots, X_n$ non son necesariamente normais, polo Teorema Central do Límite , para n grande, a media muestral aproxímase a unha normal $\bar{X} \xrightarrow[n\to\infty]{} N\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$.

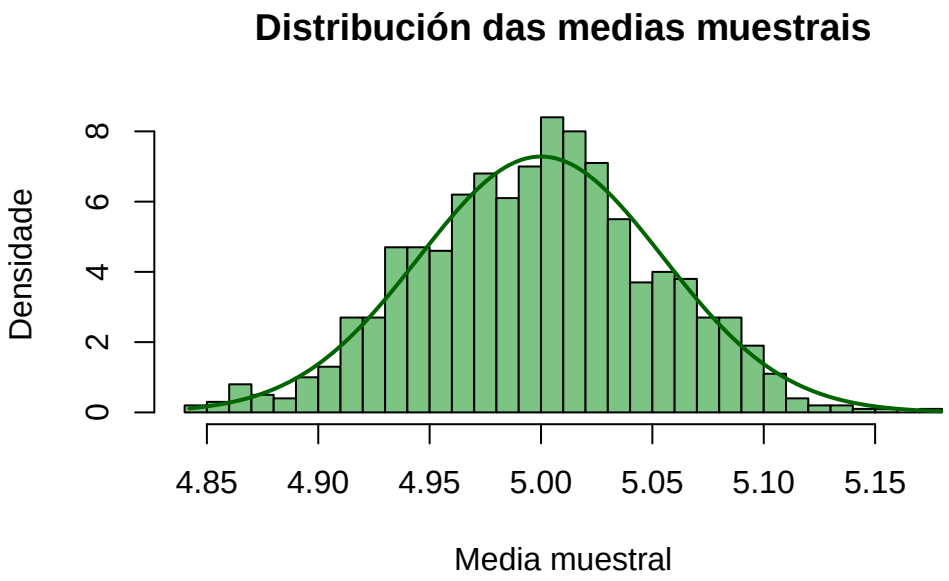
Imos facer unha simulación para entender a distribución da media muestral co seguinte exemplo. Unha plataforma de comercio eletrónico quere estudar o tempo medio de resposta (en ms) do seu sistema de recomendación. Supoñamos que o tempo de resposta segue unha distribución normal con media $\mu = 5$ ms e desviación típica $\sigma = 03$ ms. O obxectivo é entender como varía a media muestral cando tomamos distintas mostras do mesmo tamaño. A continuación:

- Simulamos 1000 mostras de tamaño $n = 30$ cada unha, extraídas dunha poboación normal con media $\mu = 5$ e desviación típica $\sigma = 03$.
- Calculamos a media muestral de cada mostra.
- Facemos un histograma das 1000 medias muestrais.
- Engadimos a curva teórica da distribución da media muestral $\bar{X} \sim N\left(\mu, \frac{\sigma}{\sqrt{n}}\right)$

Observamos como se concentra a distribución de $\bar{X}$ arredor de μ e comparamos o histograma dos datos simulados coa curva teórica. Observa, cambiando o tamaño da mostra n, como se ve afectada a dispersión da media muestral $\bar{X}$.

```r
# Parámetros da poboación
mu <- 5
sigma <- 0.3
n <- 30 # tamaño da mostra
B <- 1000 # número de mostras
# Simulación das medias muestrais
medias <- numeric(B)
set.seed(123) # para reproducibilidade
for (i in 1:B) {
  mostra <- rnorm(n, mean = mu, sd = sigma)
  medias[i] <- mean(mostra)
}
# Histograma das medias muestrais
hist(medias,
  freq = FALSE, col = "#7DC383", border = "black",
```

```
  main = "Distribución das medias muestrais",
  xlab = "Media muestral", ylab = "Densidade", breaks = 30
)
# Engadir curva teórica da media muestral
x_vals <- seq(min(medias), max(medias), length = 200)
y_vals <- dnorm(x_vals, mean = mu, sd = sigma / sqrt(n))
lines(x_vals, y_vals, col = "darkgreen", lwd = 2)
```



Empregando a distribución do estatístico podemos resolver cuestións como a seguinte: Sexa X unha variable aleatoria normal con media $\mu = 50$ e desviación típica $\sigma = 10$. Calcula a probabilidade de que a media $\bar{X}$ dunha m.a.s de tamaño $n = 25$ difira da media poboacional μ menos de 2 unidades.

```
# Parámetros
mu <- 50
sigma <- 10
n <- 25
delta <- 2 # Diferencia da media
se <- sigma / sqrt(n) # Erro estándar da media
# Probabilidade de que |Xbar - mu| < delta
pnorm(mu + delta, mean = mu, sd = se) - pnorm(mu - delta, mean = mu, sd = se)
## [1] 0.6826895
```

Estimación puntual da varianza

Como estimador para a varianza poblacional σ^2 empregaremos:

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}\left(X_i - \bar{X}\right)^2,$$

tamén chamada **cuasivarianza muestral**. Definida desta maneira, S^2 cumpre que $\mathbb{E}(S^2) = \sigma^2$, polo que é un estimador insesgado de σ^2.

> Distribución do estatístico
>
> Se $X_1, \ldots, X_n$ son variables aleatorias independentes e distribuídas segundo $N(\mu, \sigma)$, entón
>
> $$\frac{(n-1)S^2}{\sigma^2} \sim \chi^2_{n-1}$$
>
> É dicir, a cuasivarianza muestral (debidamente escalada) segue unha distribución chi cadrado con $n-1$ graos de liberdade. Se a poboación non é normal, esta distribución xa non é válida exactamente, polo que as inferencias sobre a varianza baséanse nesta hipótese.

Imos facer unha simulación para entender a distribución da cuasivarianza muestral co mesmo exemplo que no caso da media. Unha plataforma de comercio eletrónico quere estudar a varianza do tempo de resposta (en ms) do seu sistema de recomendación. Supoñamos que o tempo de resposta segue unha distribución normal con media $\mu = 5$ ms e desviación típica $\sigma = 03$ ms. O obxectivo é entender como varía a cuasivarianza muestral cando tomamos distintas mostras do mesmo tamaño. A continuación:

- Simulamos 1000 mostras de tamaño $n = 30$ cada unha, extraídas dunha poboación normal con media $\mu = 5$ e desviación típica $\sigma = 03$.
- Calculamos a cuasivarianza muestral S^2 de cada mostra.
- Facemos un histograma dos 1000 valores de $\frac{(n-1)S^2}{\sigma^2} \sim \chi^2_{n-1}$
- Engadimos a curva teórica da distribución de $\frac{(n-1)S^2}{\sigma^2} \sim \chi^2_{n-1}$

```
# Parámetros da poboación
mu <- 5
sigma <- 0.3
n <- 30 # tamaño da mostra
B <- 1000 # número de mostras

# Simulación das varianzas muestrais
```

```r
varianzas <- numeric(B)
set.seed(123)

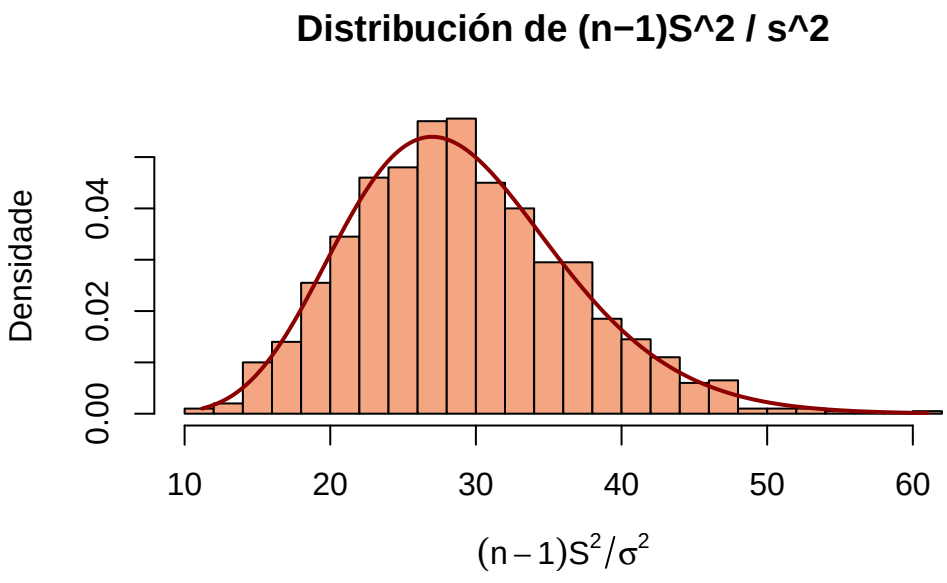
for (i in 1:B) {
  mostra <- rnorm(n, mean = mu, sd = sigma)
  varianzas[i] <- var(mostra) # xa usa n-1
}

# Transformación ao estatístico chi cadrado
chi_vals <- (n - 1) * varianzas / sigma^2

# Histograma
hist(chi_vals,
  freq = FALSE, col = "#f4a582", border = "black",
  main = "Distribución de (n-1)S^2 /  ^2",
  xlab = expression((n - 1) * S^2 / sigma^2),
  ylab = "Densidade", breaks = 30
)

# Curva teórica chi cadrado
x_vals <- seq(min(chi_vals), max(chi_vals), length = 200)
y_vals <- dchisq(x_vals, df = n - 1)

lines(x_vals, y_vals, col = "darkred", lwd = 2)
```



4.1.2. Estimación puntual dunha proporción

Consideremos agora o problema de estimar a proporción de individuos dunha poboación que presentan unha certa característica, a partir dunha mostra aleatoria simple. Para iso, definimos a variable aleatoria

$$X = \begin{cases} 1, & \text{se o individuo presenta a característica de interese} \\ 0, & \text{se o individuo non presenta a característica de interese.} \end{cases}$$

Observa que así definida, X ten distribución Bernoulli(p), onde o parámetro p é precisamente a proporción (descoñecida) de individuos coa característica na poboación.

A mostra está formada por n variables $X_1, \dots, X_n$ independentes e coa mesma distribución que X. O estimador razoable para p é a **proporción muestral**:

$$\hat{p} = \frac{\text{número de individuos coa característica na mostra}}{n} = \frac{X_1 + \cdots + X_n}{n}$$

Observamos que $\mathbb{E}(\hat{p}) = p$, polo que $\hat{p}$ é insesgado para p.

Distribución do estatístico

Polo TCL, para n grande, $\hat{p}$ é aproximadamente normal con media p e varianza $p(1-p)/n$. É dicir,

$$\hat{p} \xrightarrow[n\to\infty]{} N\left(p, \sqrt{\frac{p(1-p)}{n}}\right)$$

Imos facer unha simulación para entender a distribución da proporción muestral cun exemplo sinxelo. Supoñamos que lanzamos unha moeda equilibrada, onde a probabilidade de obter cara é $p = 05$. O obxectivo é entender como varía a proporción muestral cando tomamos distintas mostras do mesmo tamaño. A continuación:

- Simulamos 1000 mostras de tamaño $n = 100$ cada unha, onde cada observación é unha variable Bernoulli con probabilidade $p = 05$.
- Calculamos a proporción muestral $\hat{p}$ en cada mostra.
- Facemos un histograma das 1000 proporcións mostrais.
- Engadimos a aproximación teórica normal $\hat{p} \sim N\left(p, \sqrt{\frac{p(1-p)}{n}}\right)$.

Observamos como se concentra a distribución de $\hat{p}$ arredor de p e comparamos o histograma dos datos simulados coa curva teórica. Observa, cambiando o tamaño da mostra n, como se reduce a variabilidade da proporción muestral.

```r
# Parámetros
p <- 0.5
n <- 100
B <- 1000

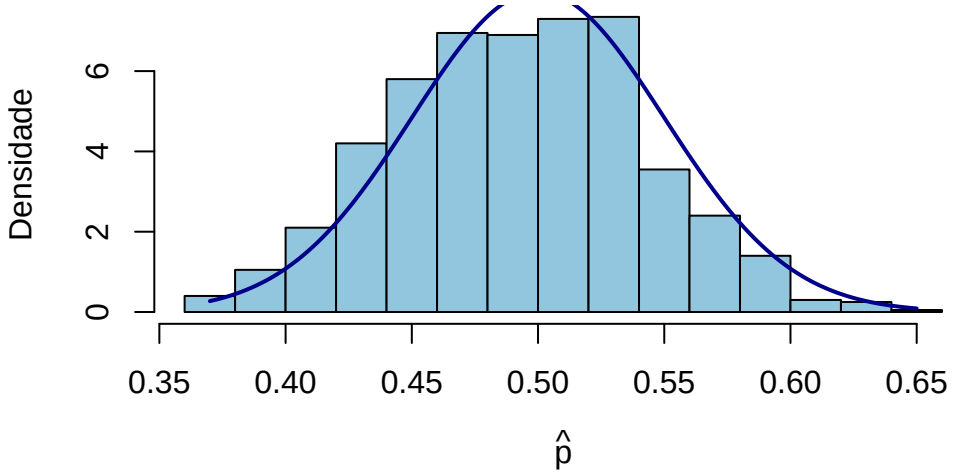
# Simulación das proporcións mostrais
prop <- numeric(B)
set.seed(123)

for (i in 1:B) {
  mostra <- rbinom(n, size = 1, prob = p)
  prop[i] <- mean(mostra)
}

# Histograma
hist(prop,
  freq = FALSE, col = "#92c5de", border = "black",
  main = "Distribución da proporción muestral",
  xlab = expression(hat(p)), ylab = "Densidade", breaks = 20
)

# Curva normal teórica (aproximación)
x_vals <- seq(min(prop), max(prop), length = 200)
y_vals <- dnorm(x_vals, mean = p, sd = sqrt(p * (1 - p) / n))

lines(x_vals, y_vals, col = "darkblue", lwd = 2)
```



4.2. Intervalos de confianza

Un **intervalo de confianza** é un intervalo construído a partir da mostra e, por tanto, **aleatorio**, que contén ao parámetro cunha certa probabilidade, chamada **nivel de confianza**. Formalmente, un intervalo de confianza para un parámetro descoñecido θ con nivel de confianza $1-\alpha$, $\alpha \in [0,1]$ é un intervalo de extremos aleatorios $L_1 \equiv L_1(X_1, \dots, X_n)$ e $L_2 \equiv L_2(X_1, \dots, X_n)$ que, con probabilidade $1-\alpha$ conté ao parámetro, é dicir,

$$P(L_1 \leq \theta \leq L_2) = 1 - \alpha$$

4.2.1. Intervalo de confianza para a media dunha poboación normal

Consideremos unha mostra aleatoria simple $X_1, \dots, X_n$ de tamaño n procedente dunha poboación normal $N(\mu, \sigma)$. O obxectivo é construír un intervalo de confianza para o parámetro μ, é dicir, para a media da poboación.

IC de nivel $1-\alpha$ para μ con σ^2 coñecida

$$\left(\bar{X} - z_{\alpha/2}\frac{\sigma}{\sqrt{n}},\ \ \bar{X} + z_{\alpha/2}\frac{\sigma}{\sqrt{n}}\right)$$

Como exemplo, supoñamos unha plataforma de comercio eletrónico que quere estimar o tempo medio de resposta (en ms) do seu sistema de recomendación. Para iso mídese o tempo de resposta en 10 usuarios:

545, 444, 495, 557, 466, 558, 504, 490, 545, 461

Suponse que a variable é normal con desviación típica $\sigma = 03$. Calculamos un intervalo de confianza do 95 % para o tempo medio de resposta.

```
x <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
sigma <- 0.3
n <- length(x)
alpha <- 0.05
xbar <- mean(x)
z_alpha2 <- qnorm(1 - alpha/2)
LI <- xbar - z_alpha2 * sigma/sqrt(n)
LS <- xbar + z_alpha2 * sigma/sqrt(n)
cat("O intervalo de confianza do 95% para a media é: [", LI, ",", LS, "]\n")
## O intervalo de confianza do 95% para a media é: [ 4.879061 , 5.250939 ]
```

Un nivel de confianza do 95 % por exemplo significa que, se repetimos o procedemento de construción do intervalo moitas veces, aproximadamente o 95 % dos intervalos conterán o verdadeiro valor do parámetro. Para entender o significado do nivel de confianza, realizamos a seguinte simulación:

- Xeramos $B = 1000$ mostras independentes dunha poboación normal con $\mu = 5$ e $\sigma = 03$
- Para cada mostra, construímos un intervalo de confianza do 95 % para μ (supoñendo σ coñecida)
- Contamos cantos destes intervalos conteñen o verdadeiro valor μ.

O resultado que se obtén é unha proporción próxima a 0.95. En resumo, o 95 % refírese á fiabilidade do método, non a un intervalo particular.

```
# Parámetros da poboación
mu <- 5
sigma <- 0.3
# Outros parámetros
n <- 30
alpha <- 0.05
z_alpha2 <- qnorm(1 - alpha/2)
B <- 1000

set.seed(1234)
conta <- 0
for (i in 1:B) {
```

```
    mostra <- rnorm(n, mean = mu, sd = sigma)
    xbar <- mean(mostra)
    LI <- xbar - z_alpha2 * sigma/sqrt(n)
    LS <- xbar + z_alpha2 * sigma/sqrt(n)
    if (LI <= mu & mu <= LS) {
        conta <- conta + 1
    }
}
cat("Proporción de intervalos que conteñen :", conta/B, "\n")
## Proporción de intervalos que conteñen : 0.955
```

IC de nivel $1-\alpha$ para μ con σ^2 descoñecida

$$\left(\bar{X} - t_{n-1,\alpha/2}\frac{S}{\sqrt{n}},\ \ \bar{X} + t_{n-1\alpha/2}\frac{S}{\sqrt{n}}\right)$$

Como exemplo, volvamos ao exemplo dunha plataforma de comercio eletrónico na que quere estimar o tempo medio de resposta (en ms) do seu sistema de recomendación. Para iso mídese o tempo de resposta en 10 usuarios:

$$545,\ 444,\ 495,\ 557,\ 466,\ 558,\ 504,\ 490,\ 545,\ 461$$

Suponse que a variable é normal, pero supoñemos agora que non coñecemos a desviación típica σ. Calculamos un intervalo de confianza do 95 % para o tempo medio de resposta.

```
x <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
n <- length(x)
alpha <- 0.05
xbar <- mean(x)
s <- sd(x)
t_alpha2 <- qt(1 - alpha/2, df = n - 1)
LI <- xbar - t_alpha2 * s/sqrt(n)
LI
## [1] 4.761551
LS <- xbar + t_alpha2 * s/sqrt(n)
LS
## [1] 5.368449
cat("O intervalo de confianza do 95% para a media é: [", LI, ",", LS, "]\n")
## O intervalo de confianza do 95% para a media é: [ 4.761551 , 5.368449 ]

t.test(x, conf.level = 0.95)$conf.int
```

```
## [1] 4.761551 5.368449
## attr(,"conf.level")
## [1] 0.95
```

Observa que en R, o cálculo de intervalos de confianza para a media con varianza descoñecida pódese facer directamente coa función `t.test()`, que implementa automaticamente o procedemento baseado na distribución t de Student.

4.2.2. Intervalo de confianza para a diferenza de medias de poboacións normais

Veremos agora como construír intervalos de confianza para **comparar dúas medias**, xa sexa de dúas poboacións diferentes ou de dúas medicións sobre os mesmos individuos dunha poboación. Segundo o tipo de datos dispoñibles, a comparación realízase con **mostras independentes** ou **mostras emparelladas**, nas que se obteñen dúas observacións relacionadas para cada individuo.

Mostras independentes, varianzas coñecidas

Supoñamos dúas poboacións nas que se estudan dúas características distribuídas normalmente con medias descoñecidas μ_1 e μ_2, respectivamente. Estamos interesados en construír un intervalo de confianza para a diferenza de medias $\mu_1 - \mu_2$ a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

As mostras considéranse **independentes**, é dicir, os individuos da poboación 1 son distintos dos individuos da poboación 2. Supoñemos ademais que as **varianzas σ_1^2 e σ_2^2 son coñecidas**.

IC de nivel $1-\alpha$ para $\mu_1 - \mu_2$ con σ_1^2 e σ_2^2 coñecidas

$$\left((\bar{X}_1 - \bar{X}_2) - z_{\alpha/2}\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}},\ \ (\bar{X}_1 - \bar{X}_2) + z_{\alpha/2}\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}} \right)$$

Supoñamos como exemplo unha plataforma de comercio electrónico que quere comparar o tempo medio de resposta do seu sistema en dous servidores distintos. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $545, 444, 495, 557, 466, 558, 504, 490, 545, 461$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Supoñendo que os tempos en cada servidos son normais con $\sigma_1 = 03$ e $\sigma_2 = 025$, calculamos un intervalo de confianza do 95 % para a diferenza de tempos medios entre servidores.

```r
x1 <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)
n1 <- length(x1)
n2 <- length(x2)
sigma1 <- 0.3
sigma2 <- 0.25
xbar1 <- mean(x1)
xbar2 <- mean(x2)
alpha <- 0.05
z_alpha2 <- qnorm(1 - alpha/2)
se_diff <- sqrt(sigma1^2/n1 + sigma2^2/n2)
LI <- (xbar1 - xbar2) - z_alpha2 * se_diff
LS <- (xbar1 - xbar2) + z_alpha2 * se_diff
cat("O intervalo de confianza do 95% para  1- 2 é: [", LI, ",", LS, "]\n")
## O intervalo de confianza do 95% para  1- 2 é: [ -0.1420377 , 0.3420377 ]
```

Mostras independentes, varianzas descoñecidas e iguais

En moitas aplicacións, os valores de σ_1^2 e σ_2^2 son descoñecidos, pero podemos supoñer que son iguais. Consideremos dúas poboacións con características distribuídas normalmente con medias descoñecidas μ_1 e μ_2. Queremos construír un intervalo de confianza para $\mu_1 - \mu_2$ a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

As mostras considéranse **independentes** e supoñemos que as **varianzas son descoñecidas pero iguais**.

IC de nivel $1-\alpha$ para $\mu_1 - \mu_2$ con σ_1^2 e σ_2^2 descoñecidas ($\sigma_1^2 = \sigma_2^2$)

$$\left((\bar{X}_1 - \bar{X}_2) - t_{n_1+n_2-2,\alpha/2}\sqrt{\frac{S_p^2}{n_1} + \frac{S_p^2}{n_2}},\ \ (\bar{X}_1 - \bar{X}_2) + t_{n_1+n_2-2,\alpha/2}\sqrt{\frac{S_p^2}{n_1} + \frac{S_p^2}{n_2}} \right)$$

Recorda que, no intervalo anterior,

$$S_p^2 = \frac{(n_1-1)S_1^2 + (n_2-1)S_2^2}{n_1 + n_2 - 2}$$

Como exemplo, supoñamos de novo unha plataforma de comercio electrónico que compara o tempo medio de resposta do seu sistema en dous servidores distintos. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $545, 444, 495, 557, 466, 558, 504, 490, 545, 461$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Supoñendo que os tempos son normais e as varianzas descoñecidas pero iguais, constrúe un intervalo de confianza do 95 % para a diferenza de tempos medios entre servidores.

```r
x1 <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)
n1 <- length(x1)
n2 <- length(x2)
xbar1 <- mean(x1)
xbar2 <- mean(x2)
s1 <- sd(x1)
s2 <- sd(x2)
sp2 <- ((n1 - 1) * s1^2 + (n2 - 1) * s2^2)/(n1 + n2 - 2)
alpha <- 0.05
t_alpha2 <- qt(1 - alpha/2, df = n1 + n2 - 2)
se_diff <- sqrt(sp2/n1 + sp2/n2)
LI <- (xbar1 - xbar2) - t_alpha2 * se_diff
LS <- (xbar1 - xbar2) + t_alpha2 * se_diff
cat("O intervalo de confianza do 95% para  1- 2 é: [", LI, ",", LS, "]\n")
## O intervalo de confianza do 95% para  1- 2 é: [ -0.1934196 , 0.3934196 ]

t.test(x1, x2, var.equal = TRUE, conf.level = 0.95)$conf.int
## [1] -0.1934196  0.3934196
## attr(,"conf.level")
## [1] 0.95
```

Como no caso do cálculo do intervalo de confianza para a media dunha poboación con varianza descoñecida, podemos empregar en R a función `t.test()`, que implementa automaticamente o procedemento baseado na distribución t de Student. Neste caso temos que dar como entrada os vectores que conteñen os datos das dúas mostras que queremos comparar e indicar con `var.equal = TRUE` que asumimos que as dúas poboacións teñen a mesma varianza.

Mostras independentes, varianzas descoñecidas e distintas

Consideremos agora dúas poboacións con características distribuídas normalmente con medias descoñecidas μ_1 e μ_2. Queremos construír un intervalo de confianza para $\mu_1 - \mu_2$ a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

As mostras considéranse **independentes** e supoñemos que as **varianzas son descoñecidas e diferentes**. Agora non podemos ter unha estimación global da varianza como no caso anterior.

IC de nivel $1-\alpha$ para $\mu_1 - \mu_2$ con σ_1^2 e σ_2^2 descoñecidas

O intervalo de confianza aproximado de nivel $1-\alpha$ para a diferenza de medias $\mu_1 - \mu_2$ para n grande será:

$$\left((\bar{X}_1 - \bar{X}_2) - z_{\alpha/2}\sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}},\ \ (\bar{X}_1 - \bar{X}_2) + z_{\alpha/2}\sqrt{\frac{S_1^2}{n_1} + \frac{S_2^2}{n_2}} \right)$$

O IC pode ser pouco preciso para tamaños de mostra pequenos. Cando n_1 ou $n_2 < 30$, empréganse aproximacións (aproximación de Welch, que substitúe $z_{\alpha/2}$ polo valor $t_{\alpha/2}$ dunha distribución t de Student adecuada).

Volvamos entón ao exemplo da plataforma de comercio electrónico na que queremos comparar o tempo medio de resposta do seu sistema en dous servidores distintos. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $545, 444, 495, 557, 466, 558, 504, 490, 545, 461$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Supoñendo que os tempos son normais e as varianzas descoñecidas, calculamos un intervalo de confianza do 95 % para a diferenza de tempos medios entre servidores (non asumimos nada en relación ás varianzas).

```
x1 <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)
n1 <- length(x1)
n2 <- length(x2)
xbar1 <- mean(x1)
xbar2 <- mean(x2)
s1 <- sd(x1)
s2 <- sd(x2)
alpha <- 0.05
z_alpha2 <- qnorm(1 - alpha/2)
se_diff <- sqrt(s1^2/n1 + s2^2/n2)
LI <- (xbar1 - xbar2) - z_alpha2 * se_diff
```

```r
LS <- (xbar1 - xbar2) + z_alpha2 * se_diff
cat("O intervalo de confianza do 95% para  1- 2 é: [", LI, ",", LS, "]\n")
## O intervalo de confianza do 95% para  1- 2 é: [ -0.1737331 , 0.3737331 ]

t.test(x1, x2, var.equal = FALSE, conf.level = 0.95)$conf.int  # Aproximación de Welch
## [1] -0.209184  0.409184
## attr(,"conf.level")
## [1] 0.95
```

Observa que se calculamos o intervalo de confianza para a diferenza de medias usando a aproximación normal, o resultado non coincide exactamente co que obtemos coa función `t.test()` de R (indicamos agora `var.equal = FALSE` para sinalar que estamos no caso de varianzas descoñecidas e diferentes). A diferencia entre os intervalos calculados débese a que `t.test()` aplica a aproximación de Welch, que toma en conta os graos de liberdade efectivos e a estimación de cada varianza muestral. Polo tanto, `t.test()` proporciona un intervalo máis preciso cando as varianzas das dúas mostras son descoñecidas e potencialmente diferentes, polo que é recomendable empregalo en lugar da aproximación normal directa.

Mostras emparelladas

Consideremos agora unha poboación na que medimos dúas características relacionadas X_1 e X_2 sobre os mesmos individuos. Supoñemos que $X_1 \sim N(\mu_1, \sigma_1)$ e $X_2 \sim N(\mu_2, \sigma_2)$, pero tendo en conta que agora X_1 e X_2 **non son independentes**. Queremos construír un intervalo de confianza para a diferenza de medias $\mu_1 - \mu_2$. Na práctica, dispoñemos de dúas **mostras emparelladas** de tamaño n observadas **nos mesmos individuos**:

- $X_{11}, \dots, X_{1n}$ coa mesma distribución $N(\mu_1, \sigma_1)$.
- $X_{21}, \dots, X_{2n}$ coa mesma distribución $N(\mu_2, \sigma_2)$.

IC de nivel $1-\alpha$ para $\mu_1 - \mu_2$. Mostras emparelladas

$$\left(\bar{D} - t_{n-1,\alpha/2}\frac{S_D}{\sqrt{n}},\ \ \bar{D} + t_{n-1\alpha/2}\frac{S_D}{\sqrt{n}}\right)$$

No intervalo anterior $\bar{D}$ e S_D refírese, respectivamente, á media e desviación tipica de $D_i = X_{1i} - X_{2i}, \quad i = 1, \dots, n$.

Como exemplo, un equipo de IA quere avaliar o impacto dunha nova técnica de preprocesamento sobre o tempo de adestramento dun modelo de clasificación de imaxes. Para iso, obtéñense os tempos de adestramento do modelo (en segundos) para 10 imaxes antes e despois de aplicar a técnica.

- Antes do preprocesamento: $82, 85, 78, 90, 88, 84, 87, 80, 86, 83$
- Despois do preprocesamento: $85, 88, 80, 92, 90, 86, 89, 82, 88, 85$

Calculamos un intervalo de confianza do 95 % para a diferenza dos tempos medios de adestramento.

```
antes <- c(82, 85, 78, 90, 88, 84, 87, 80, 86, 83)
despois <- c(85, 88, 80, 92, 90, 86, 89, 82, 88, 85)
d <- antes - despois
n <- length(d)
dbar <- mean(d)
sD <- sd(d)
alpha <- 0.05
t_alpha2 <- qt(1 - alpha/2, df = n - 1)
LI <- dbar - t_alpha2 * sD/sqrt(n)
LS <- dbar + t_alpha2 * sD/sqrt(n)
cat("O IC do 95% para _antes- _despois é: [", LI, ",", LS, "]\n")
## O IC do 95% para _antes- _despois é: [ -2.501621 , -1.898379 ]

t.test(antes, despois, paired = TRUE, conf.level = 0.95)$conf.int
## [1] -2.501621 -1.898379
## attr(,"conf.level")
## [1] 0.95
```

Observa que ao empregar `t.test()` con `paired = TRUE`, indicámoslle a R que se trata de observacións emparelladas, polo que o test considera a variabilidade das diferenzas individuais en lugar de tratar as mostras como independentes.

4.2.3. Intervalo de confianza para a varianza dunha poboación normal

Consideremos de novo unha mostra aleatoria simple $X_1, \ldots, X_n$ de tamaño n procedente dunha poboación normal $N(\mu, \sigma)$. Agora o obxectivo é construír un intervalo de confianza para o parámetro σ^2, é dicir, para a varianza da poboación.

IC de nivel $1-\alpha$ para σ^2

$$\left(\frac{(n-1)S^2}{\chi^2_{n-1,\,\alpha/2}}, \frac{(n-1)S^2}{\chi^2_{n-1,\,1-\alpha/2}} \right)$$

Supoñamos que unha plataforma de comercio eletrónico mide o tempo de resposta (en ms) do seu sistema de recomendación en 10 usuarios:

545, 444, 495, 557, 466, 558, 504, 490, 545, 461

Asumimos que a variable é normal. Calculamos un intervalo de confianza do 95 % para a varianza do tempo de resposta.

```r
x <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
n <- length(x)
alpha <- 0.05
xbar <- mean(x)
s2 <- var(x)
chi_lower <- qchisq(alpha/2, df = n - 1)
chi_upper <- qchisq(1 - alpha/2, df = n - 1)
LI <- (n - 1) * s2/chi_upper
LS <- (n - 1) * s2/chi_lower
cat("O IC do 95% para a varianza é: [", LI, ",", LS, "]\n")
## O IC do 95% para a varianza é: [ 0.0851322 , 0.5997098 ]
```

4.2.4. Intervalo de confianza para o cociente de varianzas de poboacións normais

Supoñamos dúas poboacións nas que se estudan dúas características distribuídas normalmente con varianzas descoñecidas σ_1^2 e σ_2^2, respectivamente. O obxectivo deste apartado é construír un intervalo de confianza para **comparar as dúas varianzas**. Estamos interesados en construír un intervalo de confianza para o cociente de varianzas σ_1^2/σ_2^2 a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

IC de nivel $1-\alpha$ para σ_1^2/σ_2^2

$$\left(\frac{S_1^2/S_2^2}{F_{n_1-1,n_2-1,\,\alpha/2}},\ \frac{S_1^2/S_2^2}{F_{n_1-1,n_2-1,\,1-\alpha/2}}\right) = \left(\frac{S_1^2}{S_2^2}F_{n_2-1,n_1-1,\,1-\alpha/2},\ \frac{S_1^2}{S_2^2}F_{n_2-1,n_1-1,\,\alpha/2}\right)$$

Como exemplo, unha plataforma de comercio electrónico quere analizar a variabilidade do tempo de resposta do seu sistema de recomendación en dous servidores distintos. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: 545, 444, 495, 557, 466, 558, 504, 490, 545, 461
- Servidor 2: 510, 480, 485, 520, 490, 500, 495, 488, 505, 492

Supoñendo que os tempos son normais, calculamos un intervalo de confianza do 95 % para a o cociente das varianzas dos tempos nos dous servidores. Neste caso, empregaremos directamente a función `var.test()`

```
x1 <- c(5.45, 4.44, 4.95, 5.57, 4.66, 5.58, 5.04, 4.9, 5.45, 4.61)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)
var_test <- var.test(x1, x2, conf.level = 0.95)
var_test$conf.int
## [1]  2.956622 47.922795
## attr(,"conf.level")
## [1] 0.95
```

4.2.5. Intervalo de confianza para unha proporción

Consideremos unha mostra aleatoria simple de tamaño n dunha poboación na que queremos estimar a proporción p dunha característica de interese.

IC aproximado de nivel $1-\alpha$ para p

$$\left(\hat{p} - z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}},\ \ \hat{p} + z_{\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n}}\right)$$

Supoñamos un modelo de clasificación de imaxes distingue entre gatos e cans. Avaliáronse 50 imaxes, das cales 42 foron clasificadas correctamente polo modelo. Calculamos un intervalo de confianza do 95 % para a proporción de clasificación correcta do modelo.

```
n <- 50  # Número total de imaxes
x <- 42  # Número de clasificacións correctas
phat <- x/n  # Proporción da mostra
alpha <- 0.05
z_alpha2 <- qnorm(1 - alpha/2)
se <- sqrt(phat * (1 - phat)/n)
LI <- phat - z_alpha2 * se
LS <- phat + z_alpha2 * se
cat("O IC do 95% para p é: [", LI, ",", LS, "]\n")
## O IC do 95% para p é: [ 0.7383839 , 0.9416161 ]
```

4.2.6. Intervalo de confianza para a diferenza de proporcións

Nalgúns casos estamos interesados en estimar a diferenza de proporcións $p_1 - p_2$ de dúas poboacións. Para iso dispoñemos de dúas mostras:

- Unha mostra formada por n_1 variables independentes procedentes da poboación 1.
- Unha mostra formada por n_2 variables independentes procedentes da poboación 2.

Supoñemos que as mostras son independentes, é dicir, os individuos nos que se realizan as medicións da poboación 1 son distintos dos individuos da poboación 2.

IC aproximado de nivel $1-\alpha$ para $p_1 - p_2$

$$\left((\hat{p}_1 - \hat{p}_2) - z_{\alpha/2}\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}},\ (\hat{p}_1 - \hat{p}_2) + z_{\alpha/2}\sqrt{\frac{\hat{p}_1(1-\hat{p}_1)}{n_1} + \frac{\hat{p}_2(1-\hat{p}_2)}{n_2}} \right)$$

Como exemplo, un equipo compara dous modelos de clasificación de imaxes: Modelo A e Modelo B. Avaliáronse 50 imaxes con cada modelo:

- Modelo A: 42 clasificacións correctas
- Modelo B: 45 clasificacións correctas

Calculamos un intervalo de confianza do 95 % para a diferenza de proporcións de clasificación correcta.

```
n1 <- 50
x1 <- 42  # Modelo A
n2 <- 50
x2 <- 45  # Modelo B
p1_hat <- x1/n1
p2_hat <- x2/n2
alpha <- 0.05
z_alpha2 <- qnorm(1 - alpha/2)
se_diff <- sqrt(p1_hat * (1 - p1_hat)/n1 + p2_hat * (1 - p2_hat)/n2)
LI <- (p1_hat - p2_hat) - z_alpha2 * se_diff
LS <- (p1_hat - p2_hat) + z_alpha2 * se_diff
cat("O IC do 95% para p1-p2 é: [", LI, ",", LS, "]\n")
## O IC do 95% para p1-p2 é: [ -0.191303 , 0.07130296 ]
```

4.3. EXERCICIOS

Boletín de exercicios

Descargar (PDF)

Arquivos de datos

StudentPerformanceFactors.csv Este ficheiro foi obtido de Kaggle. https://www.kaggle.com/datasets/sadiajavedd/students-academic-performance-dataset.
heart_rate_data.csv Este ficheiro foi obtido de Kaggle. https://www.kaggle.com/datasets/abdullahmalmutairi/pre-and-post-exercise-heart-rate-analysis.

5Contrastes de hipóteses

Nesta práctica aplicaremos os conceptos de contrastes de hipóteses estudados na teoría utilizando o entorno de programación R. O obxectivo é que aprendamos a formular hipóteses estatísticas, calcular estatísticos de contraste e interpretar os p-valores para tomar decisións sobre parámetros dunha poboación a partir de mostras.

5.1. Conceptos básicos

Un **contraste de hipóteses** é un procedemento estatístico que permite tomar unha decisión sobre un parámetro descoñecido dunha poboación a partir dunha mostra. Para iso formúlanse dúas hipóteses:

- **Hipótese nula** H_0: representa a afirmación inicial sobre o parámetro. É a hipótese que asumimos certa antes de observar os datos. Goza de *presunción de inocencia*, de modo que só se rexeitará se hai evidencia suficiente en contra dela.
- **Hipótese alternativa** H_1: representa a afirmación que se acepta se a evidencia da mostra é suficientemente forte contra H_0.

A decisión baséase nun **estatístico de contraste**, cuxa distribución baixo H_0 é coñecida. Ao fixar un nivel de significación α, obtense implícitamente unha división en dúas rexións do conxunto de posibles valores do estatístico de contraste:

- A **rexión de rexeitamento** ou rexión crítica, que ten probabilidade α (baixo H_0).
- A **rexión de aceptación**, que ten probabilidade $1-\alpha$ (baixo H_0).

Se o valor do estatístico cae na rexión de aceptación, non existen razóns suficientes para rexeitar a hipótese nula cun nivel de significación α, e o contraste considérase **estatísticamente non significativo**, é dicir, non existe evidencia a favor de H_1. Se o valor do estatístico cae na rexión de rexeitamento, os datos non son compatibles con H_0 e rexeitamos a hipótese. Entón dicimos que o contraste é **estatísticamente significativo**, é dicir, existe evidencia estatisticamente significativa a favor de H_1.

Tamén podemos tomar a decisión a partir do denominado **p-valor** do estatístico, que mide a evidencia da mostra contra a hipótese nula. Dado un **nivel de significación** $\alpha \in [0,1]$, a regra de decisión é:

- Se p-valor $< \alpha$ rexeitamos H_0 a nivel α.
- Se p-valor $> \alpha$ non rexeitamos H_0 a nivel α.

5.2. Contrastes para a media dunha poboación normal

Consideremos unha mostra aleatoria simple $X_1, \ldots, X_n$ de tamaño n procedente dunha poboación normal $N(\mu, \sigma)$. O obxectivo é formualr contrastes relativos á media poboacional μ.

5.2.1. Contrastes para a media con varianza coñecida

Supoñamos que a varianza da poboación σ^2 é coñecida. Queremos en primeiro lugar contrastar se a media da poboación é igual a un determinado valor μ_0. Formulamos o contraste

$$H_0 : \mu = \mu_0 \qquad \text{fronte a} \qquad H_1 : \mu \neq \mu_0$$

Se a varianza da poboación é coñecida, o **estatístico de contraste** é

$$Z = \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}},$$

que segue unha distribución normal estándar $N(0,1)$ cando H_0 é certa.

Contraste bilateral $H_0 : \mu = \mu_0$ fronte a $H_1 : \mu \neq \mu_0$ (σ^2 coñecida)

Rexeitamos $H_0 : \mu = \mu_0$ fronte a $H_1 : \mu \neq \mu_0$ para un nivel α se

$$\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} \leq -z_{\alpha/2} \quad \text{ou} \quad \frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} \geq z_{\alpha/2}$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é distinto de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica $\sigma = 03$. Para un nivel de significación $\alpha = 005$, ¿podemos asumir que o tempo medio de resposta é distinto de 5 ms?

```r
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
sigma <- 0.3
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
xbar
## [1] 5.24

# Estatístico de contraste
z <- (xbar - mu0)/(sigma/sqrt(n))
z
## [1] 2.529822

# Valor crítico
z_alpha2 <- qnorm(1 - alpha/2)
z_alpha2
## [1] 1.959964

# Decisión a partir da rexión crítica
if (z > z_alpha2 | z < (-z_alpha2)) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo medio
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
}
decision
## [1] "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo medio non é 5 m

# Decisión a partir do p-valor
p_valor <- 2 * (1 - pnorm(abs(z)))

cat("Estatístico de contraste Z =", z, "\n")
## Estatístico de contraste Z = 2.529822
cat("p-valor =", p_valor, "\n")
## p-valor = 0.01141204
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.01141204
```

Contraste unilateral $H_0 : \mu \leq \mu_0$ fronte a $H_1 : \mu > \mu_0$ (σ^2 coñecida)

Rexeitamos a hipótese nula $H_0 : \mu \leq \mu_0$ fronte a $H_1 : \mu > \mu_0$ se

$$\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} \geq z_\alpha$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é maior de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica $\sigma = 03$. Para un nivel de significación $\alpha = 005$, ¿hai evidencia de que o tempo medio de resposta é maior de 5 ms?

```r
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
sigma <- 0.3
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
xbar
## [1] 5.24

# Estatístico de contraste
z <- (xbar - mu0)/(sigma/sqrt(n))
z
## [1] 2.529822

# Valor crítico
z_alpha <- qnorm(1 - alpha)
z_alpha
## [1] 1.644854

# Decisión a partir da rexión crítica
if (z > z_alpha) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é mai
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
```

```
}
decision
## [1] "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é maior de 5 m

# Decisión a partir do p-valor
p_valor <- 1 - pnorm(z)

cat("Estatístico de contraste Z =", z, "\n")
## Estatístico de contraste Z = 2.529822
cat("p-valor =", p_valor, "\n")
## p-valor = 0.005706018
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.005706018
```

Contraste unilateral $H_0 : \mu \geq \mu_0$ fronte a $H_1 : \mu < \mu_0$ (σ^2 coñecida)

Rexeitamos a hipótese nula $H_0 : \mu \geq \mu_0$ fronte a $H_1 : \mu < \mu_0$ se

$$\frac{\bar{X} - \mu_0}{\sigma/\sqrt{n}} \leq -z_\alpha$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é menor de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica $\sigma = 03$. Para un nivel de significación $\alpha = 005$, ¿hai evidencia de que o tempo medio de resposta é menor de 5 ms?

```
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
sigma <- 0.3
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
xbar
## [1] 5.24
```

```r
# Estatístico de contraste
z <- (xbar - mu0)/(sigma/sqrt(n))
z
## [1] 2.529822

# Valor crítico
z_alpha <- qnorm(1 - alpha)
-z_alpha
## [1] -1.644854

# Decisión a partir da rexión crítica
if (z <= -z_alpha) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é meno
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
}
decision
## [1] "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun tempo maio

# Decisión a partir do p-valor
p_valor <- pnorm(z)

cat("Estatístico de contraste Z =", z, "\n")
## Estatístico de contraste Z = 2.529822
cat("p-valor =", p_valor, "\n")
## p-valor = 0.994294
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.994294
```

5.2.2. Contrastes para a media con varianza descoñecida

Supoñamos agora que a varianza da poboación σ^2 é coñecida. Queremos formular o contraste

$$H_0 : \mu = \mu_0 \qquad \text{fronte a} \qquad H_1 : \mu \neq \mu_0$$

Neste caso, o **estatístico de contraste** é

$$\frac{\bar{X} - \mu_0}{S/\sqrt{n}},$$

que segue unha distribución t de Student con $n-1$ grados de liberdade cando H_0 é certa.

Contraste bilateral $H_0 : \mu = \mu_0$ fronte a $H_1 : \mu \neq \mu_0$ (σ^2 descoñecida)

Rexeitamos $H_0 : \mu = \mu_0$ fronte a $H_1 : \mu \neq \mu_0$ para un nivel α se

$$\frac{\bar{X} - \mu_0}{S/\sqrt{n}} \leq -t_{n-1,\alpha/2} \quad \text{ou} \quad \frac{\bar{X} - \mu_0}{S/\sqrt{n}} \geq t_{n-1,\alpha/2}$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é distinto de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica descoñecida. Para un nivel de significación $\alpha = 005$, ¿podemos asumir que o tempo medio de resposta é distinto de 5 ms?

```r
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
S <- sd(x)

# Estatístico de contraste
T <- (xbar - mu0)/(S/sqrt(n))
T
## [1] 5.212467

# Valor crítico
t_alpha2 <- qt(1 - alpha/2, n - 1)
t_alpha2
## [1] 2.262157

# Decisión a partir da rexión crítica
if (T > t_alpha2 | T < (-t_alpha2)) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo medio
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
}
decision
```

```
## [1] "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo medio non é 5 m
```

```
# Decisión a partir do p-valor
p_valor <- 2 * (1 - pt(abs(T), n - 1))

cat("Estatístico de contraste T =", T, "\n")
## Estatístico de contraste T = 5.212467
cat("p-valor =", p_valor, "\n")
## p-valor = 0.0005547442
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.0005547442
```

Contraste unilateral $H_0 : \mu \leq \mu_0$ fronte a $H_1 : \mu > \mu_0$ (σ^2 descoñecida)

Rexeitamos a hipótese nula $H_0 : \mu \leq \mu_0$ fronte a $H_1 : \mu > \mu_0$ se

$$\frac{\bar{X} - \mu_0}{S/\sqrt{n}} \geq t_\alpha$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é maior de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica descoñecida. Para un nivel de significación $\alpha = 005$, ¿hai evidencia de que o tempo medio de resposta é maior de 5 ms?

```
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
sigma <- 0.3
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
S <- sd(x)

# Estatístico de contraste
T <- (xbar - mu0)/(S/sqrt(n))
T
```

```r
## [1] 5.212467

# Valor crítico
t_alpha <- qt(1 - alpha, n - 1)
t_alpha
## [1] 1.833113

# Decisión a partir da rexión crítica
if (T > t_alpha) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é mai
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
}
decision
## [1] "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é maior de 5 m

# Decisión a partir do p-valor
p_valor <- 1 - pt(T, n - 1)

cat("Estatístico de contraste T =", T, "\n")
## Estatístico de contraste T = 5.212467
cat("p-valor =", p_valor, "\n")
## p-valor = 0.0002773721
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.0002773721
```

Contraste unilateral $H_0 : \mu \geq \mu_0$ fronte a $H_1 : \mu < \mu_0$ (σ^2 descoñecida)

Rexeitamos a hipótese nula $H_0 : \mu \geq \mu_0$ fronte a $H_1 : \mu < \mu_0$ se

$$\frac{\bar{X} - \mu_0}{S/\sqrt{n}} \leq -t_{n-1,\alpha}$$

Isto é equivalente a comprobar se o **p-valor** é menor que α.

Como exemplo, supoñamos unha plataforma de comercio electrónico que quere comprobar se o tempo medio de resposta do seu sistema é menor de 5 ms. Para iso mídese o tempo de resposta en 10 usuarios:

$$531,\ 536,\ 496,\ 517,\ 529,\ 512,\ 512,\ 530,\ 531,\ 546$$

Suponse que a variable é normal con desviación típica descoñecida. Para un nivel de significación $\alpha = 005$, ¿hai evidencia de que o tempo medio de resposta é menor de 5 ms?

```r
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
sigma <- 0.3
mu0 <- 5
alpha <- 0.05

n <- length(x)
xbar <- mean(x)
S <- sd(x)

# Estatístico de contraste
T <- (xbar - mu0)/(S/sqrt(n))
T
## [1] 5.212467

# Valor crítico
t_alpha <- qt(1 - alpha, n - 1)
-t_alpha
## [1] -1.833113

# Decisión a partir da rexión crítica
if (T <= -t_alpha) {
    decision <- "Para un nivel alpha=0.05, rexeitamos H0: hai evidencia de que o tempo é meno
} else {
    decision <- "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun te
}
decision
## [1] "Para un nivel alpha=0.05, non rexeitamos H0: os datos son compatibles cun tempo maio

# Decisión a partir do p-valor
p_valor <- pt(T, n - 1)

cat("Estatístico de contraste T =", T, "\n")
## Estatístico de contraste T = 5.212467
cat("p-valor =", p_valor, "\n")
## p-valor = 0.9997226
cat("Rexeitamos H0 para valores alpha maiores que", p_valor, "\n")
## Rexeitamos H0 para valores alpha maiores que 0.9997226
```

En R, os contrastes para a media cunha varianza descoñecida pódense realizar directamente coa función `t.test()`. Esta función calcula automaticamente o estatístico de contraste e o p-valor. O argumento `alternative` indica o tipo de contraste que queremos realizar.

- `alternative = "two.sided"` realiza un contraste bilateral ($H_1 : \mu \neq \mu_0$)
- `alternative = "greater"` realiza un contraste unilateral á dereita ($H_1 : \mu > \mu_0$).
- `alternative = "less"` realiza un contraste unilateral á esquerda ($H_1 : \mu < \mu_0$).

Empregando os mesmos datos do exemplo anterior:

```
x <- c(5.31, 5.36, 4.96, 5.17, 5.29, 5.12, 5.12, 5.3, 5.31, 5.46)
mu0 <- 5

# Cotraste bilateral
t.test(x, mu = mu0, alternative = "two.sided")
##
##  One Sample t-test
##
## data:  x
## t = 5.2125, df = 9, p-value = 0.0005547
## alternative hypothesis: true mean is not equal to 5
## 95 percent confidence interval:
##  5.135842 5.344158
## sample estimates:
## mean of x
##      5.24

# Contraste unilateral á dereita (H1: mu > mu0)
t.test(x, mu = mu0, alternative = "greater")
##
##  One Sample t-test
##
## data:  x
## t = 5.2125, df = 9, p-value = 0.0002774
## alternative hypothesis: true mean is greater than 5
## 95 percent confidence interval:
##  5.155597      Inf
## sample estimates:
## mean of x
##      5.24

# Contraste unilateral á esquerda (H1: mu < mu0)
t.test(x, mu = mu0, alternative = "less")
##
##  One Sample t-test
##
```

```
## data:  x
## t = 5.2125, df = 9, p-value = 0.9997
## alternative hypothesis: true mean is less than 5
## 95 percent confidence interval:
##      -Inf 5.324403
## sample estimates:
## mean of x
##      5.24
```

5.3. Contrastes para a diferenza de medias

Moitas veces queremos **comparar as medias de dúas poboacións**, xa sexa de dúas poboacións diferentes ou de dúas medicións sobre os mesmos individuos dunha poboación. Segundo o tipo de datos dispoñibles, a comparación realízase con **mostras independentes** ou **mostras emparelladas**, nas que se obteñen dúas observacións relacionadas para cada individuo. Neste caso, traballaremos unicamente co caso de **varianzas descoñecidas**, que é o máis habitual en aplicacións reais. Todos os contrastes realizaranse de forma directa mediante a función `t.test()` de R, que implementa automaticamente os procedementos baseados na distribución t de Student para mostras independentes e emparelladas. Ademais, neste punto deixaremos de calcular explicitamente os estatísticos de contraste e as rexións críticas en cada caso, xa que estes detalles poden consultarse no material teórico. Na práctica centrámonos na interpretación do p-valor e na decisión estatística final.

5.3.1. Mostras independentes, varianzas descoñecidas e iguais

Supoñamos dúas poboacións nas que se estudan dúas características distribuídas normalmente con medias descoñecidas μ_1 e μ_2, respectivamente. Estamos interesados en realizar contrates sobre a diferencia de medias $\mu_1 - \mu_2$ a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

As mostras considéranse **independentes** e supoñemos que as **varianzas son descoñecidas pero iguais**.

Como exemplo, supoñamos de novo unha plataforma de comercio electrónico que compara o tempo medio de resposta do seu sistema en dous servidores distintos. Supoñemos que as varianzas dos tempos de resposta en cada servidor son descoñecidas pero iguais. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $560, 510, 535, 575, 520, 570, 545, 530, 565, 525$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Queremos contrastar se existen diferenzas entre os tempos medios dos dous servidores.

$$H_0 : \mu_1 = \mu_2 \qquad \text{fronte a} \qquad H_1 : \mu_1 \neq \mu_2$$

En R, podemos realizar directamente este contraste coa función `t.test()` indicando as dúas mostras e o argumento `var.equal = TRUE`.

```
x1 <- c(5.6, 5.1, 5.35, 5.75, 5.2, 5.7, 5.45, 5.3, 5.65, 5.25)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)

t.test(x1, x2, var.equal = TRUE, alternative = "two.sided")
##
##  Two Sample t-test
##
## data:  x1 and x2
## t = 5.7263, df = 18, p-value = 1.983e-05
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.2975622 0.6424378
## sample estimates:
## mean of x mean of y
##     5.435     4.965
```

Observamos que o p-valor é moi pequeno (inferior a 0.05), polo que rexeitamos a hipótese nula. Polo tanto, existe evidencia estatística suficiente para concluír que os tempos medios de resposta dos dous servidores son diferentes. Agora, se o que nos interesa é determinar se o tempo medio no servidor 1 é maior que no servidor 2, formulamos un contraste unilateral:

$$H_0 : \mu_1 \leq \mu_2 \qquad \text{fronte a} \qquad H_1 : \mu_1 > \mu_2$$

```
x1 <- c(5.6, 5.1, 5.35, 5.75, 5.2, 5.7, 5.45, 5.3, 5.65, 5.25)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)

t.test(x1, x2, var.equal = TRUE, alternative = "greater")
##
##  Two Sample t-test
##
## data:  x1 and x2
## t = 5.7263, df = 18, p-value = 9.917e-06
```

```
## alternative hypothesis: true difference in means is greater than 0
## 95 percent confidence interval:
##  0.3276729       Inf
## sample estimates:
## mean of x mean of y
##     5.435     4.965
```

De novo, o p-valor é moi pequeno, polo que, para calquera nivel de significación razoable, rexeitamos a hipótese nula. Existe evidencia estatística suficiente para concluír que o servidor 1 presenta un tempo medio de resposta maior que o servidor 2.

> **Nota**
>
> É importante ter en conta que en `t.test()` a orde na que se introducen as mostras é relevante, xa que determina a interpretación das hipóteses. En particular, a primeira mostra introducida considérase como referencia para o cálculo da diferenza de medias. Isto é especialmente importante nos contrastes unilaterais, xa que o sentido da hipótese alternativa depende directamente da orde na que se introducen os datos.

5.3.2. Mostras independentes, varianzas descoñecidas e distintas

Supoñamos dúas poboacións nas que se estudan dúas características distribuídas normalmente con medias descoñecidas μ_1 e μ_2, respectivamente. Estamos interesados en realizar contrates sobre a diferencia de medias $\mu_1 - \mu_2$ a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

As mostras considéranse **independentes** e supoñemos que as **varianzas son descoñecidas e distintas**.

Como exemplo, supoñamos de novo unha plataforma de comercio electrónico que compara o tempo medio de resposta do seu sistema en dous servidores distintos. Supoñemos agora que as varianzas dos tempos de resposta en cada servidor son descoñecidas e distintas. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $560, 510, 535, 575, 520, 570, 545, 530, 565, 525$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Queremos contrastar se existen diferenzas entre os tempos medios de resposta dos dous servidores.

$$H_0 : \mu_1 = \mu_2 \qquad \text{fronte a} \qquad H_1 : \mu_1 \neq \mu_2$$

En R, realizamos este contraste de novo coa función `t.test()` indicando agora o argumento `var.equal = FALSE`.

```
x1 <- c(5.6, 5.1, 5.35, 5.75, 5.2, 5.7, 5.45, 5.3, 5.65, 5.25)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)

t.test(x1, x2, var.equal = FALSE, alternative = "two.sided")
##
##  Welch Two Sample t-test
##
## data:  x1 and x2
## t = 5.7263, df = 13.805, p-value = 5.53e-05
## alternative hypothesis: true difference in means is not equal to 0
## 95 percent confidence interval:
##  0.2937289 0.6462711
## sample estimates:
## mean of x mean of y
##     5.435     4.965
```

Observamos que o p-valor é moi pequeno (inferior a 0.05), polo que rexeitamos a hipótese nula. Polo tanto, existe evidencia estatística suficiente para concluír que os tempos medios de resposta dos dous servidores son diferentes.

5.3.3. Mostras emparelladas

Consideremos agora unha poboación na que medimos dúas características relacionadas X_1 e X_2 sobre os mesmos individuos. Supoñemos que $X_1 \sim N(\mu_1, \sigma_1)$ e $X_2 \sim N(\mu_2, \sigma_2)$, pero tendo en conta que agora X_1 e X_2 **non son independentes**. Queremos realizar contrastes sobre a diferencia de medias $\mu_1 - \mu_2$. Na práctica, dispoñemos de dúas **mostras emparelladas** de tamaño n observadas **nos mesmos individuos**:

- $X_{11}, \ldots, X_{1n}$ coa mesma distribución $N(\mu_1, \sigma_1)$.
- $X_{21}, \ldots, X_{2n}$ coa mesma distribución $N(\mu_2, \sigma_2)$.

Como exemplo, un equipo de IA quere avaliar o impacto dunha nova técnica de preprocesamento sobre o tempo de adestramento dun modelo de clasificación de imaxes. Para iso, obtéñense os tempos de adestramento do modelo (en segundos) para 10 imaxes antes e despois de aplicar a técnica.

- Antes do preprocesamento: $82, 85, 78, 90, 88, 84, 87, 80, 86, 83$
- Despois do preprocesamento: $85, 88, 80, 92, 90, 86, 89, 82, 88, 85$

Queremos contrastar se existen diferenzas nos tempos medios de adestramento antes (μ_1) e despois (μ_2) da técnica de preprocesamento.

$$H_0 : \mu_1 = \mu_2 \qquad \text{fronte a} \qquad H_1 : \mu_1 \neq \mu_2$$

En R, realizamos este contraste coa función `t.test()` indicando agora o argumento `paired = TRUE`.

```
antes <- c(82, 85, 78, 90, 88, 84, 87, 80, 86, 83)
despois <- c(85, 88, 80, 92, 90, 86, 89, 82, 88, 85)

t.test(antes, despois, paired = TRUE, alternative = "two.sided")
##
##  Paired t-test
##
## data:  antes and despois
## t = -16.5, df = 9, p-value = 4.917e-08
## alternative hypothesis: true mean difference is not equal to 0
## 95 percent confidence interval:
##  -2.501621 -1.898379
## sample estimates:
## mean difference
##            -2.2
```

Á vista do p-valor, rexeitamos a hipótese nula. Polo tanto, existe evidencia estatística suficiente para concluír que os tempos medios de adestramento antes e despois da técnica de preprocesamento son diferentes. Se agora queremos comprobar se o tempo de adestramento é maior despois da técnica de preprocesamento, formulamos un contraste unilateral axeitado e repetimos o procedemento coa opción correspondente en `t.test()`.

$$H_0 : \mu_1 \geq \mu_2 \qquad \text{fronte a} \qquad H_1 : \mu_1 < \mu_2$$

```
antes <- c(82, 85, 78, 90, 88, 84, 87, 80, 86, 83)
despois <- c(85, 88, 80, 92, 90, 86, 89, 82, 88, 85)

t.test(antes, despois, paired = TRUE, alternative = "less")
##
##  Paired t-test
##
```

```
## data:  antes and despois
## t = -16.5, df = 9, p-value = 2.459e-08
## alternative hypothesis: true mean difference is less than 0
## 95 percent confidence interval:
##       -Inf -1.955585
## sample estimates:
## mean difference
##            -2.2
```

Observa que escribimos `alternative = "less"` porque estamos a probar se *antes* < *despois* (a hipótese alternativa é $H_1 : \mu_1 < \mu_2$). A partir do resultado obtido, rexeitamos a hipótese nula. Polo tanto, existe evidencia estatística suficiente para concluír que o tempo medio de adestramento é significativamente maior despois da técnica de preprocesamento.

5.4. Contrastes para a razón de varianzas de poboacións normais

Antes de estudar os contrastes para varianzas, é importante destacar que, do mesmo xeito que ocorría nos intervalos de confianza, en R non existe unha función básica específica para realizar contrastes dunha soa varianza. Este tipo de contraste pode abordarse mediante a distribución chi-cadrado, de forma manual ou empregando librerías adicionais. Porén, neste curso non imos tratar este procedemento de forma explícita. Porén, si dispoñemos dunha función directa para comparar dúas varianzas.

Supoñamos dúas poboacións nas que se estudan dúas características distribuídas normalmente con varianzas descoñecidas σ_1^2 e σ_2^2, respectivamente. O obxectivo é **comparar as dúas varianzas** a partir de dúas mostras:

- Unha mostra formada por n_1 variables independentes e coa mesma distribución $N(\mu_1, \sigma_1)$.
- Unha mostra formada por n_2 variables independentes e coa mesma distribución $N(\mu_2, \sigma_2)$.

Os contrastes que se consideran poden ser bilaterais ou unilaterais, dependendo do interese do estudo. É dicir, analizaremos contrastes do tipo

$$H_0 : \sigma_1^2 = \sigma_2^2 \qquad \text{fronte a} \qquad H_1 : \sigma_1^2 \neq \sigma_2^2,$$

ou ben os correspondentes contrastes unilaterais cando se quere determinar se unha das varianzas é maior ou menor ca outra.

Como exemplo, unha plataforma de comercio electrónico quere analizar a variabilidade do tempo de resposta do seu sistema de recomendación en dous servidores distintos. Obsérvanse os tempos de 10 usuarios distintos en cada servidor:

- Servidor 1: $560, 510, 535, 575, 520, 570, 545, 530, 565, 525$
- Servidor 2: $510, 480, 485, 520, 490, 500, 495, 488, 505, 492$

Supoñendo que os tempos son normais, queremos contrastar se existen diferenzas entre as varianzas dos tempos de resposta dos dous servidores.

```
x1 <- c(5.6, 5.1, 5.35, 5.75, 5.2, 5.7, 5.45, 5.3, 5.65, 5.25)
x2 <- c(5.1, 4.8, 4.85, 5.2, 4.9, 5, 4.95, 4.88, 5.05, 4.92)

var.test(x1, x2, conf.level = 0.95)
##
##  F test to compare two variances
##
## data:  x1 and x2
## F = 3.4564, num df = 9, denom df = 9, p-value = 0.07882
## alternative hypothesis: true ratio of variances is not equal to 1
## 95 percent confidence interval:
##   0.8585332 13.9156468
## sample estimates:
## ratio of variances
##            3.45645
```

O resultado do contraste proporciona p-valor de 0.07882. Comparando o p-valor co nivel de significación α, observamos que:

- Se $\alpha < 007882$, non se rexeita a hipótese nula.
- Se $\alpha \geq 007882$, rexeitaríase a hipótese nula.

En particular, para valores habituais como $\alpha = 005$, non se rexeita a hipótese nula. Polo tanto, non hai evidencia estatística suficiente para concluír que existan diferenzas significativas entre as varianzas das dúas poboacións (para un nivel de significación $\alpha = 005$ os datos son compatibles coa hipótese de igualdade de varianzas).

Se se quixeran formular contrastes unilaterais, estes poderían realizarse tamén mediante o argumento `alternative` da función `var.test()`, indicando se se desexa comprobar se a varianza dunha poboación é maior ou menor ca da outra.

5.5. Contrastes para unha proporción

Ademais dos intervalos de confianza que vimos no tema anterior, tamén podemos realizar **contrastes de hipóteses sobre unha proporción**.

Consideramos o contraste:

$$H_0 : p = p_0 \qquad \text{fronte a} \qquad H_1 : p \neq p_0$$

Na teoría, os contrastes sobre proporcións realízanse empregando a aproximación normal do estimador. En R, a función `prop.test()` implementa este contraste de forma automática. O argumento `correct = FALSE` emprégase para desactivar a corrección por continuidade que se aplica ao aproximar a distribución binomial pola normal, facendo que o procedemento sexa máis fiel á formulación teórica vista na aula.

Como exemplo, consideremos un modelo de clasificación de imaxes. Avaliáronse 50 imaxes, das cales 42 foron clasificadas correctamente. Queremos contrastar se a proporción de acerto é significativamente distinta do 90 %:

```
n <- 50
x <- 42

prop.test(x, n, p = 0.9, alternative = "two.sided", correct = FALSE)
##
##  1-sample proportions test without continuity correction
##
## data:  x out of n, null probability 0.9
## X-squared = 2, df = 1, p-value = 0.1573
## alternative hypothesis: true p is not equal to 0.9
## 95 percent confidence interval:
##  0.7148578 0.9166258
## sample estimates:
##    p
## 0.84
```

Á vista do resultado de `prop.test()`, para un nivel de significación $\alpha = 005$, non existe evidencia estatística suficiente para rexeitar a hipótese nula sobre a proporción.

Observa que intervalo de confianza para a proporción que devolve `prop.test()` non coincide exactamente co intervalo aproximado calculado na sesión de laboratorio anterior. Isto débese a que non se constrúe de forma idéntica á aproximación normal explicada na teoría.

Igual que nos outros casos, poderíase empregar o argumento `alternative` da función `prop.test()`, se o que se desexa é contrastar se a proporción de acerto é significativamente maior ou menor que un valor dado.

5.6. Contrastes para a diferenza de proporcións

Nalgúns casos estamos interesados en contrastar hipóteses sobre a diferenza de proporcións $p_1 - p_2$ de dúas poboacións. Para iso dispoñemos de dúas mostras independentes:

- Unha mostra de tamaño n_1 procedente da poboación 1.
- Unha mostra de tamaño n_2 procedente da poboación 2.

As mostras considéranse independentes, é dicir, os individuos de cada poboación son distintos.

Consideramos o contraste:

$$H_0 : p_1 = p_2 \qquad \text{fronte a} \qquad H_1 : p_1 \neq p_2$$

Como exemplo, comparamos dous modelos de clasificación de imaxes:

- Modelo A: 42 acertos en 50 imaxes
- Modelo B: 45 acertos en 50 imaxes

Queremos contrastar se a proporción de acerto é significativamente diferente nos dous modelos. En R realizamos o contraste mediante:

```r
n1 <- 50
x1 <- 42  # Modelo A
n2 <- 50
x2 <- 45  # Modelo B

prop.test(c(x1, x2), c(n1, n2), alternative = "two.sided", correct = FALSE)
##
##  2-sample test for equality of proportions without continuity correction
##
## data:  c(x1, x2) out of c(n1, n2)
## X-squared = 0.79576, df = 1, p-value = 0.3724
## alternative hypothesis: two.sided
## 95 percent confidence interval:
##  -0.19130296  0.07130296
## sample estimates:
## prop 1 prop 2
##   0.84   0.90
```

Á vista do resultado de `prop.test()`, para un nivel de significación $\alpha = 005$, non existe evidencia estatística suficiente para rexeitar a hipótese nula. Polo tanto, non hai evidencia

estatística suficiente para concluír que existan diferenzas significativas entres as proporcións de clasificación correcta dos dous modelos.

De novo, como en todos os casos anteriores poderíase empregar o argumento `alternative` da función `prop.test()`, se o que se desexa é contrastar se a proporción de acerto é significativamente maior ou menor no modelo A que no modelo B.

5.7. EXERCICIOS

Boletín de exercicios

Descargar (PDF)

Arquivos de datos

StudentPerformanceFactors.csv Este ficheiro foi obtido de Kaggle. https://www.kaggle.com/datasets/sadiajavedd/students-academic-performance-dataset.
heart_rate_data.csv Este ficheiro foi obtido de Kaggle. https://www.kaggle.com/datasets/abdullahmalmutairi/pre-and-post-exercise-heart-rate-analysis.

6Introdución á regresión linear

Nesta práctica aplicaremos os conceptos de regresión lineal estudados na teoría utilizando o entorno de programación R. O obxectivo é aprender a axustar modelos lineais, interpretar os seus coeficientes e realizar contrastes de significación sobre eles a partir de mostras de datos.

6.1. O modelo de regresión lineal simple

No modelo de regresión lineal simple considérase que a variable resposta Y depende dunha única variable explicativa X, e supóñese que a relación entre ambas pode aproximarse mediante unha recta. O modelo escríbese como

$$Y = \beta_0 + \beta_1 X + \varepsilon,$$

onde β_0 é o **intercepto** da recta, β_1 é a **pendente** que describe como cambia Y cando cambia X, e ε representa o **termo de erro**, que recolle a variabilidade que o modelo non pode explicar.

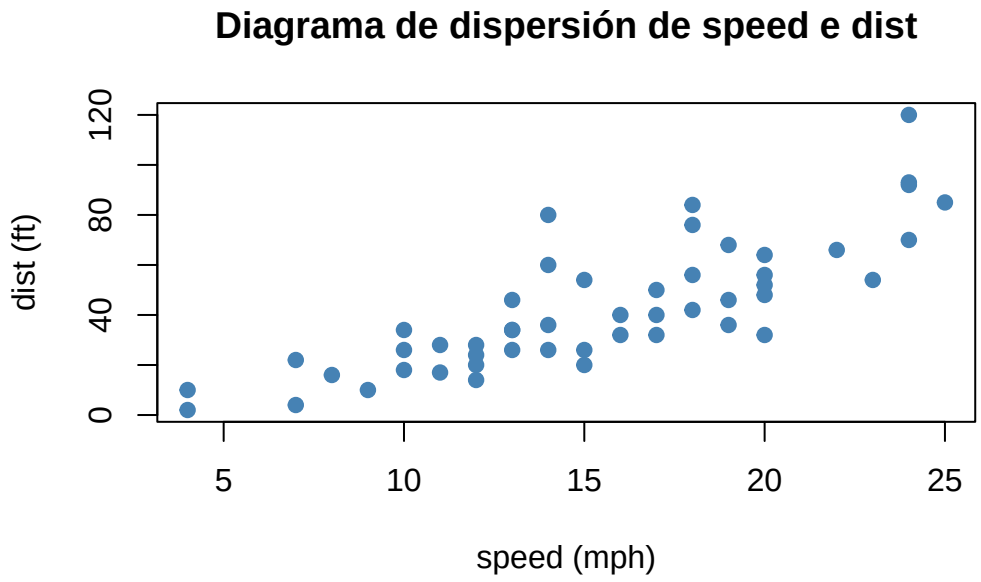
Para ilustrar a regresión lineal utilizaremos o conxunto de datos de R `cars`, que recolle información sobre a velocidade dos coches e a distancia de freado. En particular, consideraremos como variable dependente (Y) a distancia de freado (`dist`) e como variable explicativa ou independente (X) a velocidade (`speed`).

Diagrama de dispersión

Antes de axustar o modelo, representamos graficamente a relación entre as variables:

```
data(cars)
x <- cars$speed
y <- cars$dist

plot(x, y, main = "Diagrama de dispersión de speed e dist", xlab = "speed (mph)",
    ylab = "dist (ft)", pch = 19, col = "steelblue")
```



Cálculo de medidas resumo, covarianza e correlación

Antes de axustar o modelo de regresión, é útil analizar a relación entre as variables mediante medidas resumo e asociación lineal.

```
summary(x)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     4.0    12.0    15.0    15.4    19.0    25.0
summary(y)
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    2.00   26.00   36.00   42.98   56.00  120.00
cov(x, y)
## [1] 109.9469
cor(x, y)
## [1] 0.8068949
```

6.2. Estimación do modelo de regresión en R

Na práctica, os parámetros os parámetros β_0 e β_1 do modelo non son coñecidos e deben estimarse a partir dunha mostra, formada por pares de observacións da forma $(x_1, y_1), \ldots, (x_n, y_n)$. As estimacións resultantes adoitan denotarse por $\hat{\beta}_0$ e $\hat{\beta}_1$, e a recta estimada escríbese como

$$\hat{Y} = \hat{\beta}_0 + \hat{\beta}_1 X$$

Esta recta coñécese como **recta de regresión**, e permítenos estimar o valor medio da variable resposta para distintos valores da variable explicativa.

Estimadores de mínimos cadrados dos coeficientes da recta de regresión

$$\hat{\beta}_1 = \frac{s_{xy}}{s_x^2} \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

Podemos entón calcular os estimadores dos parámetros do modelo de regresión lineal de forma explícita a partir das expresións teóricas. No noso exemplo, estes valores poden calcularse directamente en R:

```
b1hat <- cov(x, y)/var(x)
b0hat <- mean(y) - b1hat * mean(x)
b1hat
## [1] 3.932409
b0hat
## [1] -17.57909
```

Na práctica o axuste do modelo realízase de forma automática mediante a función `lm()`. A súa sintaxe básica é:

```
lm(formula, data = ...)
```

onde:

- `formula` indica a relación entre variables no formato `Y ~ X`,
- `data` é o conxunto de datos no que se atopan as variables.

Se non se usa `data`, hai que referenciar explicitamente as variables no entorno de traballo. No noso caso, podemos axustar o modelo de regresión lineal simple do seguinte xeito:

```
modelo <- lm(dist ~ speed, data = cars)
```

Ou de forma equivalente como:

```
modelo <- lm(cars$dist ~ cars$speed)
```

A función `lm()` devolve un obxecto de clase `lm`, que contén toda a información do axuste:

```
class(modelo)
## [1] "lm"
names(modelo)
##  [1] "coefficients"  "residuals"     "effects"       "rank"
##  [5] "fitted.values" "assign"        "qr"            "df.residual"
##  [9] "xlevels"       "call"          "terms"         "model"
```

Os coeficientes estimados obtéñense con:

```
coefficients(modelo)
## (Intercept)  cars$speed
##  -17.579095    3.932409
```

Os valores axustados polo modelo $\hat{y}_i$ son:

```
fitted.values(modelo)
```

e os residuos do modelo $y_i - \hat{y}_i$

```
residuals(modelo)  # equivalente a cars$dist-fitted(modelo)
```

A función `summary()` devolve un resumo estatístico completo do modelo de regresión.

```
summary(modelo)
##
## Call:
## lm(formula = cars$dist ~ cars$speed)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -29.069  -9.525  -2.272   9.215  43.201
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) -17.5791     6.7584  -2.601   0.0123 *
## cars$speed    3.9324     0.4155   9.464 1.49e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 15.38 on 48 degrees of freedom
## Multiple R-squared:  0.6511, Adjusted R-squared:  0.6438
## F-statistic: 89.57 on 1 and 48 DF,  p-value: 1.49e-12
```

En primeiro lugar, mostra a fórmula empregada e un resumo dos erros do modelo. Tamén aparecen columnas:

- `Estimate`: valor estimado do coeficiente
- `Std. Error`: erro estándar da estimación
- `t value`: estatístico t
- `Pr(>|t|)`: p-valor

En regresión lineal, os contrastes úsanse para comprobar se os coeficientes do modelo son estatisticamente significativos. Para cada coeficiente β_i, realízase o seguinte contraste:

$$H_0 : \beta_i = 0 \qquad \text{fronte a} \qquad H_1 : \beta_i \neq 0$$

Por exemplo, no caso do contraste sobre β_1, a hipótese nula significaría que a variable X non ten efecto sobre Y. A decisión baséase no p-valor, da mesma forma que vimos no tema pasado.

No modelo de regresión lineal simple, o RSE (Residual standard error) calcúlase como:

$$RSE = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-2}}$$

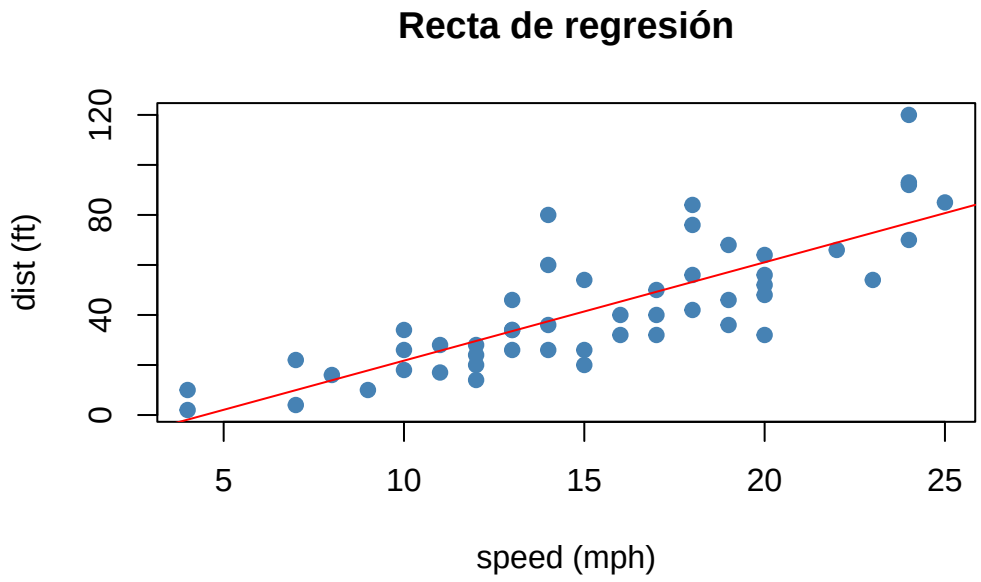
Trátase dunha estimación da desviación típica do erro. O resumo tamén devolve o coeficiente de determinación da recta R^2, que no caso do modelo de regresión lineal simple coincide con r_{xy}^2

```
R2 <- cor(x, y)^2
R2
## [1] 0.6510794
```

Podemos rerpresentar a recta de regresión

```
plot(x, y, main = "Recta de regresión", xlab = "speed (mph)", ylab = "dist (ft)",
    pch = 19, col = "steelblue")

abline(modelo, col = "red")
```



6.3. Predición

Unha vez axustado o modelo de regresión lineal, pódense facer predicións para novos valores x da variable explicativa X. A predición realízase substituíndo o valor de x na ecuación da recta. Estas predicións representan o valor esperado de Y para un determinado valor de X.

Como exemplo, facemos a predición para un coche que viaxa a 17 mph:

```
modelo <- lm(dist ~ speed, data = cars)
novo_dato <- data.frame(speed = 17)
predict(modelo, newdata = novo_dato)
##        1
## 49.27185
```

6.4. EXERCICIOS

Boletín de exercicios

Descargar (PDF)